ATLAS Note
ANA-IDTR-2026-01-INT1
13th November 2026
Draft version 0.3

End-to-End Primary Vertex Finding with Deep Learning at the HL-LHC

Mattia Mauro[a], Rocky Bala Garg[a], Lauren Tompkins[a], Qi Bin Lei[b], Trey Warner[c]

[a]*Stanford University,* [b]*UCSC,* [c]*Swarthmore*

This note presents PV-Finder, a neural-network method for reconstructing primary vertices from ATLAS tracks. We focus on High-Luminosity LHC (HL-LHC) conditions with an average pile-up of $\langle\mu\rangle$=200. PV-Finder takes reconstructed track parameters as input and predicts a one-dimensional histogram along the beamline. The vertex positions are found from peaks in this histogram.

PV-Finder gives ~~(83.16 ± 0.08)%~~ [MM: (83.06±0.08)%] efficiency, compared with ~~(79.80 ± 0.09)%~~ [MM: (79.85 ± 0.09)%] for the Adaptive Multi-Vertex Finder (AMVF). Both use a fixed ~~0.1694 mm~~ [MM: 0.17 mm] matching window, and merged truth vertices count as found. PV-Finder gives ~~0.60 ± 0.09 fewer fake and split vertices per event~~ [MM: 1.01 ± 0.09 fewer fake vertices per event]. Its vertex-vertex resolution is ~~0.229 ± 0.003 mm~~ [MM: 0.217 ± 0.003 mm], compared with 0.305 mm for AMVF. [MM: Pre-GNN numbers: frozen s8 model, to confirm with the final recompute at the 13 November peak cuts.] The network time for an earlier PV-Finder model was approximately constant over the tested pile-up range of 0 to 80. We also present a graph neural network that assigns tracks to the reconstructed PV-Finder vertices.

© 2026 CERN for the benefit of the ATLAS Collaboration.
Reproduction of this article or parts of it is allowed as specified in the CC-BY-4.0 license.

Contents

1 Introduction **6**

2 Method **8**
2.1 Track Selection . . . 8
2.2 Model Construction . . . 8
2.2.1 From tracks to a vertex histogram . . . 8
2.2.2 Eliminating the KDE representation . . . 9
2.2.3 Target histograms and the resolution model . . . 9
2.2.4 Peak finding . . . 11
2.3 Model training and optimization . . . 13
2.4 Performance metrics . . . 14
2.4.1 Vertex-vertex resolution . . . 14
2.4.2 Truth matching and vertex categories . . . 15

3 Datasets **18**
3.0.1 HL-LHC PU200 simulation . . . 18
3.0.2 Samples with full $|\eta|$ acceptance . . . 18
3.0.3 Run 2 and Run 3 data . . . 19

4 HL-LHC PU200 results **20**
4.1 Performance relative to AMVF . . . 20
4.1.1 Vertex-vertex resolution . . . 22
4.1.2 Checks of the matching definition . . . 24
4.1.3 Merged, missed and fake vertices . . . 27
4.1.4 Removing fake peaks . . . 27
4.1.5 Satellite peaks . . . 27

5 Inference timing and scaling with pile-up **28**
5.1 Athena Integration and Timing: . . . 28
5.1.1 Athena implementation details: . . . 28
5.1.2 Timing studies . . . 30

6 Validation at low pile-up **32**
6.1 Simulated data . . . 32
6.2 Collision data . . . 34

7 Track-to-vertex association with graph networks **37**
7.1 Task . . . 37
7.2 Graphs, features and labels . . . 37
7.3 Architecture and training . . . 38

7.4 Track selection and evaluation . . . 40
7.5 HL-LHC full-chain performance . . . 41
7.6 Performance at low pile-up and comparison with Run 3 data . . . 46
7.6.1 Comparison with Run 3 collision data . . . 49
7.7 Further checks and outlook . . . 49

8 Conclusions and outlook **51**

Appendices **55**

A Reproduction **55**
A.1 Results-settings matrix . . . 55
A.2 Effect of extended track acceptance . . . 56
A.3 Data conversion . . . 57
A.4 Training . . . 57
A.5 Evaluation . . . 58
A.6 Track-to-vertex association . . . 59
A.6.1 Graph building . . . 60
A.6.2 Training . . . 60
A.6.3 Evaluation . . . 61
A.7 Model checkpoints . . . 62
A.8 Key paths and test sets . . . 62

B Hyperparameters **64**

C Historical model development **66**
C.1 Analytical KDE construction . . . 66

D Additional PU200 studies **67**
D.1 Full comparison at the fixed window . . . 67
D.1.1 Single-track truth vertices . . . 67
D.1.2 Integral cut and binning of the resolution plot . . . 67
D.2 Results with the wider matching window . . . 68
D.3 Check of the reconstruction version . . . 69
D.4 Examples of merged, missed and fake vertices . . . 70
D.5 Cuts to remove fake peaks . . . 74
D.6 Study of satellite peaks . . . 75

E Studies that did not improve performance **77**
E.1 Training without KDE targets . . . 77
E.2 The role of the U-Net . . . 77
E.3 Changes to the architecture and model size . . . 77
E.4 HGTD timing . . . 78

Version Tracking

Version 0.1: initial document
Version 0.2: updates to timing studies (Chapter 5). Version 0.3: added hard scatter plots

Todo list

1 Introduction

Reconstructing the positions of proton-proton collisions along the beam axis is an important part of event reconstruction at the LHC. These positions are called primary vertices (PVs). The task becomes more difficult as the number of collisions in the same bunch crossing, or pile-up, increases. The typical average pile-up was around 25-30 during Run 2 and reached about 60 during Run 3. At the High-Luminosity LHC (HL-LHC), it is expected to reach about 200. At this level, the computational cost of vertex reconstruction becomes an important limitation. This motivates the study of methods whose cost grows more slowly with pile-up.

ATLAS currently reconstructs primary vertices with the Adaptive Multi-Vertex Finder (AMVF) [1]. AMVF finds and fits several vertex candidates in each event, and its execution time increases with pile-up [2]. PV-Finder is a deep-learning alternative developed in earlier LHCb and ATLAS studies [3, 4]. It takes reconstructed track parameters as input and predicts a one-dimensional histogram of the vertex density along the beamline. The vertex positions are then found from peaks in this histogram. A timing study of an earlier model found that its network time changed little over pile-up 0 to 80. The production model still needs a full timing study at PU200. The fixed-size network makes PV-Finder worth studying as pile-up increases.

The PV-Finder architecture has changed over several stages. The original method used an analytical kernel density estimate (KDE) of the track positions as input to a convolutional network [5]. A later version replaced the analytical calculation with a multilayer perceptron (MLP), giving a two-network tracks-to-KDE and KDE-to-histogram pipeline [6]. In this work, the MLP and U-Net are combined into one end-to-end model. The MLP is first trained directly on the target histogram, so an intermediate KDE is no longer needed during training or inference.

The current model is trained using a dataset of 5.17×10^6 simulated $t\bar{t}$ events. On the PU200 subset of a separate flat-pile-up evaluation sample, its efficiency is ~~3.36 ± 0.06~~ [MM: 3.20 ± 0.06] percentage points higher than AMVF with a common ~~0.1694 mm~~ [MM: 0.17 mm] matching window. This efficiency counts merged truth vertices as found. PV-Finder also gives ~~0.60 ± 0.09 fewer fake and split vertices per event~~ [MM: 1.01 ± 0.09 fewer fake vertices per event] and a smaller vertex-vertex resolution. Increasing the size of the network and changing the optimizer or loss did not improve the results. [MM: The frozen HL-LHC model, s8, is the smaller of the networks tested (439 593 parameters), with no measurable loss: -0.15 ± 0.04 points in efficiency against the 3.55M-parameter model.] The main remaining problems are low-multiplicity vertices that do not produce a peak and close vertex pairs that are reconstructed as one peak. About 13% of adjacent truth-vertex pairs are separated by less than the measured PV-Finder vertex-vertex resolution of ~~0.229 mm~~ [MM: 0.217 mm; the 13% was computed at 0.229 mm and is to be recomputed]. These pairs are particularly difficult to separate.

We also present a graph neural network for assigning tracks to the reconstructed PV-Finder vertices. Each event is represented as a graph containing tracks, candidate vertices, and the possible connections between them. The network gives a score to each track-vertex connection. The full PV-Finder and GNN chain is

compared with AMVF using the same track-based vertex classification. It is tested at PU200 and at lower pile-up, and it is also applied to Run 3 collision data.

Section 2 describes the vertex-finder model and its training, and Section 3 describes the datasets. Section 4 presents the PU200 results and the comparison with AMVF, while Section 5 gives the timing measurements. Section 6 presents the low-pile-up validation, and Appendix E discusses approaches that did not improve the results. Section 7 describes the GNN track association. The appendices contain the settings and commands used to reproduce the results.

2 Method

This section describes how PV-Finder turns reconstructed tracks into vertex positions. Section 2.1 gives the track selection. Sections 2.2.1 to 2.2.4 describe the network, training targets and peak-finding algorithm. Section 2.3 summarizes the training and model development.

2.1 Track Selection

Due to the fact that the ATLAS inner detector will be replaced with the ITK for the HL-LHC runs, different track selections are used for Run 2/3 and the HL-LHC studies. For the Run 2 and Run 3 samples (Section 3.0.3) track satisfy the tight primary requirements and $p_{\mathrm{T}} > 500$ MeV, $|\eta| < 2.5$ and $|z_0| < 200$ mm. Here z_0 is the longitudinal impact parameter at the point of closest approach to the beamline. No requirement on d_0 is applied.

For the HL-LHC samples (Section 3.0.2), the requirements are $p_{\mathrm{T}} \geq 0.9$ GeV, $|d_0| \leq 1$ mm, $\sigma(d_0) \leq 0.35$ mm, $|z_0| \leq 200$ mm and $|\eta| < 4$, in addition to all of the requirements in Table 5 and 6 in the the ITk performance study [7].

2.2 Model Construction

2.2.1 From tracks to a vertex histogram

PV-Finder represents the primary vertices in an event as a one-dimensional histogram. It has 12 000 bins spanning $z \in [-240, 240]$ mm, with a bin width of 0.04 mm. The event is divided into twelve consecutive 40 mm intervals of 1 000 bins each. The same network processes each interval separately. For a given interval, the input contains the tracks whose z_0 lies inside its bounds. The bounds are also given to the network as input features. The twelve outputs are then concatenated to form the full histogram. The target histogram is split at the same interval boundaries. Peak finding (Section 2.2.4) is applied to the full histogram.

The model has two components, shown in Figure 2.1. The first is a masked MLP. Each track is represented by seven features: d_0, z_0, their uncertainties and covariance, and the lower and upper bounds of the interval. Since every interval is 40 mm wide, the two bounds are redundant: one bound, or the interval centre, would contain the same information. Both are kept because the trained checkpoints use the original seven-feature input. The model does not use hit-level information. Padding tracks are masked, and the contribution from each valid track is summed. The model can therefore process a variable number of tracks and is invariant to their ordering.

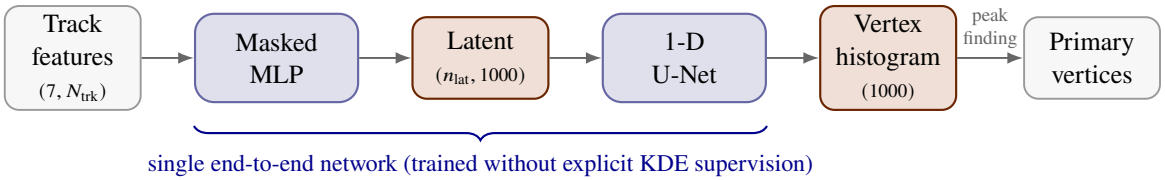


Figure 2.1: End-to-end PV-Finder architecture. A masked MLP maps the variable-length set of track features for a 40 mm interval to a latent representation of size $n_{\text{lat}} \times 1000$, which a one-dimensional U-Net refines into a 1 000-bin vertex-density histogram. Vertex positions are obtained by peak finding. The MLP and U-Net form one trainable system.

The network itself does not set a maximum number of tracks, and no tracks are removed in the samples used here. The MLP has five fully connected hidden layers with LeakyReLU activations and a Softplus output. It produces a latent representation of size $n_{\text{lat}} \times 1000$.

The second component is a one-dimensional U-Net [8]. It uses an encoder-decoder with skip connections to produce the 1 000-bin vertex histogram. The earlier two-stage pipeline used an explicit KDE as the latent representation. In the end-to-end model, the MLP and U-Net are trained together and this intermediate representation is learned (Section 2.2.2). Figure 2.2 shows an analytical KDE and an example output from the end-to-end model. Appendix C describes how the analytical KDE is calculated.

2.2.2 Eliminating the KDE representation

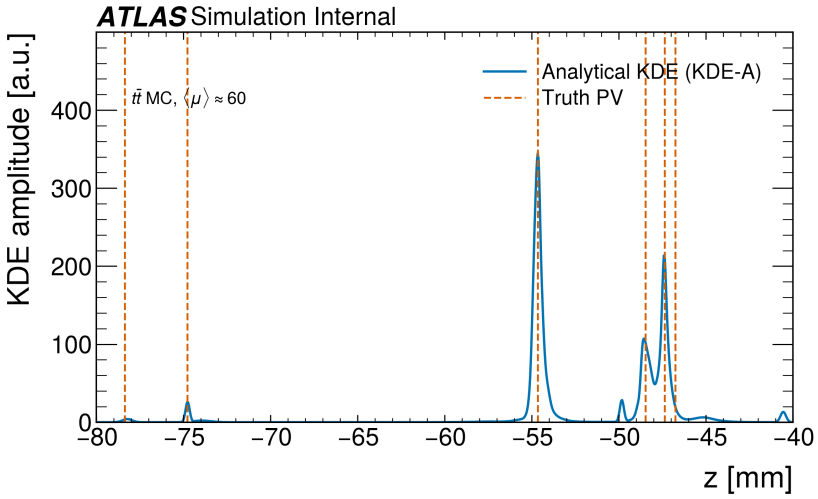
The two-stage pipeline requires a target KDE for every training interval. In the end-to-end model, training from random initialization with only the histogram loss does not work. The network places most of the predicted density in the first bin, and the U-Net spreads it across the histogram. It can then reach a low mean-squared error without learning the correct spatial mapping (Appendix E).

The warm-up avoids this solution by training in two phases. In Phase 1, the MLP is trained on its own against the target histogram. The U-Net is not used. The MLP output is compared directly with the histogram, so the MLP learns the mapping from z_0 to the histogram bins. In Phase 2, the U-Net is added after the MLP and the full model is trained end to end. The loss still compares the predicted histogram with the target histogram, and both the MLP and the U-Net are updated. Both phases use the mean-squared error. No KDE target is used.

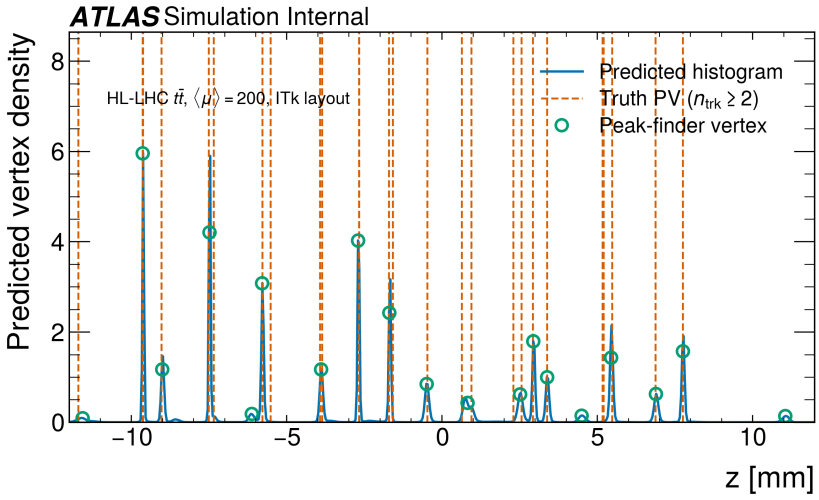
The current HL-LHC model uses this KDE-free warm-up. Computing and storing analytical KDEs for the full PU200 training sample would be too expensive (Section 3.0.1). The low-pile-up reproduction in Section 6 uses the original recipe instead. Its end-to-end model is initialized from separately trained tracks-to-KDE and KDE-to-histogram models and is then fine-tuned.

2.2.3 Target histograms and the resolution model

The training target for each interval is a histogram built from the truth primary vertices using the Gaussian cumulative-distribution method introduced for the PV-Finder [3]. Each truth PV is represented by a Gaussian centered on its z position. The width of the Gaussian is the estimated z resolution of a vertex with that track multiplicity. For vertices with $n_{\text{trk}} \geq 2$, this width is taken from a fit to the measured resolution, as



(a) Analytical KDE (input representation) with truth PVs.



(b) End-to-end output histogram at PU200 with reconstructed PVs.

Figure 2.2: (a) An analytically computed KDE of the track density (solid blue) for one 40 mm interval of a $\langle\mu\rangle \approx 60$ event, the representation used as input in the original two-stage pipeline; dashed orange lines mark the truth primary vertices. Vertices with few tracks deposit little or no density, which is why low-multiplicity vertices are the hardest to reconstruct. (b) The histogram predicted by the end-to-end model (solid blue) for part of a PU200 event; dashed orange lines mark truth vertices with $n_{\text{trk}} \geq 2$ and open circles the vertices returned by the peak finder (Section 2.2.4).

described below. Vertices with fewer tracks are given a single-bin width. The histogram has two channels, aggregating vertices with $n_{\text{trk}} \geq 2$ and $n_{\text{trk}} < 2$ separately, and the first channel is used for vertex finding.

The widths are obtained from HL-LHC or Run 2 simulation, as appropriate. The HL-LHC determination uses 99 800 events and 9 243 820 matched pairs. It has two steps. First, the position residuals are fitted in each track-multiplicity bin. Then, the resulting widths are fitted as a function of multiplicity.

We match AMVF vertices to truth vertices one to one within 2 mm, choosing the closest available pair at each step. We keep matches whose truth vertex has at least two tracks. For each pair, we calculate

$x = z_{\mathrm{AMVF}} - z_{\mathrm{truth}}$ and group these residuals by truth-track multiplicity. Each multiplicity group, labelled k, gets a histogram with 200 bins from −2 to +2 mm. The residual bins are therefore 0.02 mm wide. Groups with fewer than 200 matched pairs are not fitted.

Each residual histogram is fitted with a Gaussian and a flat background,

$$f_k(x) = G_k \exp\left(-\frac{x^2}{2s_k^2}\right) + b_k. \tag{2.1}$$

Here G_k is the Gaussian height, s_k is its width in mm, and b_k is the background level in counts per residual bin. The Gaussian centre is fixed at zero. Within one multiplicity group, b_k is constant across the residual axis x. Each group has its own fitted b_k, so the level can change with track multiplicity. We do not impose a common background level or a formula for its dependence on multiplicity.

The initial background estimate is the mean count in bins with $|x| > 1.5$ mm, with a minimum starting value of 0.1. We then fit G_k, s_k and b_k together using all 200 bins. The heights G_k and b_k are constrained to be non-negative, and $10^{-4} \le s_k \le 1$ mm. The background remains free during the fit; we do not subtract the initial estimate from the histogram.

The 46 points in Figure 2.3 are the fitted widths s_k, plotted at the mean track multiplicity $\bar{n}_k$ of each group. We fit these points with

$$\sigma(n_{\mathrm{trk}}) = A\, n_{\mathrm{trk}}^{-B} + C, \tag{2.2}$$

determining A, B and C together. The weights use the width uncertainties from the residual fits, with a minimum uncertainty of 0.2% of each width. The term $A\, n_{\mathrm{trk}}^{-B}$ falls with multiplicity, and C is a constant offset in the fit. The function describes the measured vertex-position resolution over the track multiplicities in the sample. It is not a physical model, and C should not be read as the resolution at large n_{trk}. It is separate from the background levels b_k. The fit gives

$$A = 0.14318 \text{ mm}, \qquad B = 0.36200, \qquad C = -0.01797 \text{ mm}. \tag{2.3}$$

The fit gives $\chi^2/\mathrm{ndf} = 60.2$ with these point uncertainties. We use the curve as an approximate model for the training widths.

These widths use AMVF vertices matched to truth. We are studying a replacement that does not use AMVF. In simulation, reconstructed tracks would be grouped by their truth vertex, and a weighted average of their z_0 would give the position. The width of that residual against the truth z, as a function of track multiplicity, would set the target width.

When the target histograms are built, the width is required to be at least 0.002 mm. This limit was not reached in the 5.17 million converted events. The smallest calculated width was 0.00255 mm.

2.2.4 Peak finding

Primary vertices are extracted from the predicted PV-Finder output histogram by a deterministic scan, illustrated in Figure 2.4. Moving left to right, the algorithm collects each contiguous sequence of bins whose predicted output is above an amplitude threshold $\tau = 0.01$. This sequence is an above-threshold interval. An interval is accepted as a vertex if it is at least three bins wide and its integral exceeds a threshold. Two vertices close enough that the histogram never drops back below τ between them form a single interval.

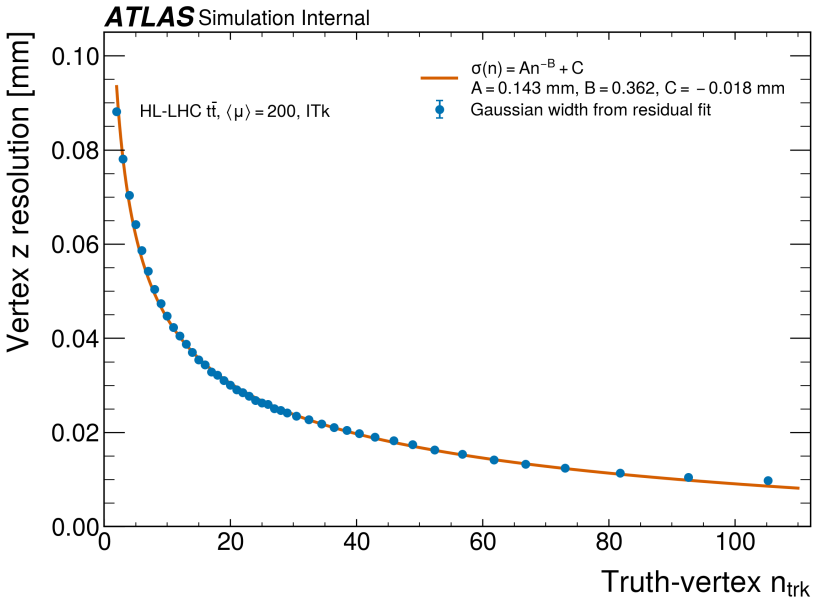


Figure 2.3: AMVF–truth vertex z-resolution as a function of truth track multiplicity for the HL-LHC PU200 ITk sample. Each point is the Gaussian width s_k from a Gaussian-plus-background fit in one track-multiplicity group, plotted at that group's mean multiplicity. The background level is fitted separately in each group. The solid line fits these widths with Equation 2.2, determining A, B and C together.

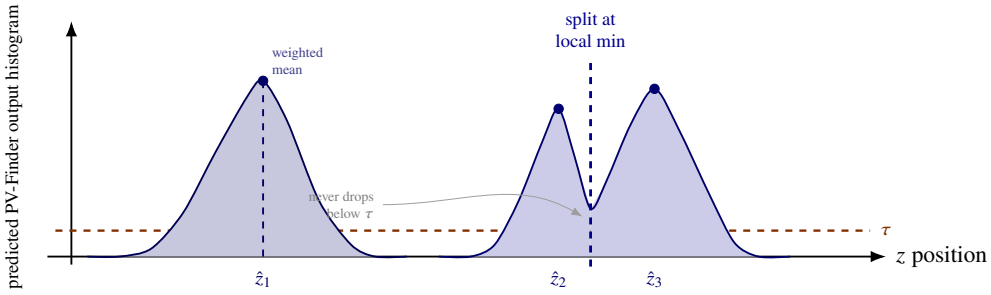


Figure 2.4: Schematic illustration of peak finding on the predicted PV-Finder output histogram. An isolated peak (left) is accepted if it clears the amplitude threshold τ, the minimum width and the integral cut, and its position is the amplitude-weighted mean. Conjoined peaks (right) that never dip below τ are separated at the intervening local minimum.

These conjoined peaks are split at the local minimum that separates them. An optional minimum-height floor on the peak amplitude removes the lowest, broadest spurious peaks (Section 4.1.4).

The vertex position is the local centroid. This is the amplitude-weighted mean of the bins within three bins of the peak maximum, rather than of the full accepted interval. Using the whole interval can pull the position toward a neighbouring peak after a conjoined region is split. The local centroid reduces this bias and is used for the results in this note.

The peak cuts are listed in Table 2.1. The amplitude threshold $\tau = 0.01$ determines where a peak starts and ends. ~~For the HL-LHC efficiency and vertex counts, we require an integral of at least 0.30 and a height of at least 0.03. For the resolution plot, we use a slightly higher integral cut of 0.50 to remove more small~~

~~peaks and make the dip in the pair-separation distribution clearer. All other peak cuts stay the same. The higher cut also removes some real vertices, so we keep 0.30 for the efficiency and vertex counts. The effect of this choice is checked in Appendix D.1.2.~~ [MM: Every result in this note uses one configuration per regime, given in Table 2.2. The two regimes differ only in the density model and in the integral cut: 0.186 for HL-LHC and 0.50 at low pile-up. There is no height cut in either regime, and $\sigma_{\text{vtx-vtx}}$ is fitted on the same peaks as the efficiency, so there is no separate threshold for the resolution. The integral and height are in units of each model's output, so the values are not portable between models. The HL-LHC cut is set to give the fake rate of the earlier 3.55M-parameter model; 0.50 is the Run 2 convention. The HL-LHC cut was consolidated on 13 November 2026 on tuning events only: the integral cut alone (0.186) reproduces the earlier integral plus height cuts (0.184, 0.0184) within 0.01 points in efficiency and 0.02 fakes per event in every μ bin.]

Table 2.1: Peak-finding parameters used for the HL-LHC results. [MM: Superseded by Table 2.2.]

Parameter	Value	Role
Amplitude threshold τ	0.01	start/stop an above-threshold interval
Minimum width	3 bins	reject single-bin spikes
Integral threshold (counts)	0.30	select peaks for efficiency and counts
Integral threshold (resolution)	0.50	remove more small peaks for the fit
Minimum height	0.03	remove low peaks

2.3 Model training and optimization

The production model was trained on the 5.17M-event PU200 sample with $|\eta| < 4$ (Section 3.0.2), using the split in Table 2.3. In Phase 2, we used a learning rate of 10^{-4}, a short warm-up followed by cosine decay, and gradient clipping at 1.0. These settings gave stable training. The higher Phase 2 learning rate of 10^{-3} used in the Run 2 recipe caused training to diverge at PU200.

We tested the model size on an initial sample of 99.8K events. A model with 681K parameters and a larger model with 3.55M parameters gave similar performance. A sweep over the learning rate, optimizer, and loss also gave no clear improvement. These tests suggest that increasing the model size alone did not improve the result. More details are given in Appendix E.

We kept the larger architecture for the later training. This allowed us to change the training sample and target widths without also changing the network. It does not show that 3.55M parameters are needed. A smaller model trained on the current sample is needed for a direct comparison. [MM: This comparison has now been made. The frozen HL-LHC model, s8, has 439 593 parameters (80 channels, one latent channel) and is trained on the same sample with the same recipe, with one epoch per phase. On the two evaluation samples its efficiency differs from the 3.55M-parameter model by -0.15 ± 0.04 and -0.08 ± 0.04 points. s8 is used for every HL-LHC result.]

Table 2.2: [MM: Frozen configuration, one column per regime. Every result in this note uses one of these two configurations. The HL-LHC integral cut (0.186, no height cut) is the 13 November 2026 consolidation; the HL-LHC vertex-finding numbers were computed with 0.184 and a height cut of 0.0184, which give the same results within 0.01 points, and are to be confirmed with the final recompute.]

	HL-LHC	Run 2/3 (low pile-up)
Samples	ITk $t\bar{t}$ simulation, $0 \le \mu \le 200$	Run 2 simulation, Run 2 data, Run 3 data
Density model	s8: U-Net v2, 80 channels, 1×1000 latent, MLP 128×5, 439 593 parameters	e400: U-Net v1, 64 channels, 1×1000 latent, MLP 100×5, 358 713 parameters
Training	Phase 1 (MLP only) 1 epoch, Phase 2 1 epoch; no KDE	Phase 1 50 epochs, Phase 2 400 epochs; no KDE
Amplitude τ, minimum width	0.01, 3 bins	0.01, 3 bins
Integral cut	0.186	0.50
Minimum height	none	none
Position	local centroid, ±3 bins	local centroid, ±3 bins
$\sigma_{\text{vtx-vtx}}$ fit	same peaks, 300 bins over ±6 mm	same
Vertex matching window	0.17 mm	0.17 mm
Hard-scatter window	0.1 mm	0.1 mm
Residual assignment	one to one within 1 mm (measures the residuals that set the window)	same
GNN	v2, epoch 527, 39 521 parameters	$\mu = 60$ v2, epoch 115, 39 521 parameters
GNN score cut t	0.94	0.94
Vertex after the GNN	≥ 2 tracks, for PV-Finder+GNN and AMVF	same
Density training	5.17×10^6 PU200 events (six files); intervals split 96/3/1% (training/validation/unused test)	Run 2 simulation events 0–35 699; validation 35 700–48 449
GNN training	149 820 flat-μ events (`r16443` entries $\ge$ 25 000, `r16638`); validation 7 500 `r16443` events, which also set t	35 700 Run 2 simulation events; validation 12 750
Evaluation before the GNN	`r16443` and `r16638` entries 0–24 999; PU200 ($185 \le \mu \le 215$): 1 920 and 1 888 events	Run 2 simulation events 48 450–50 999 (2 550); 2 500 Run 2 and 2 500 Run 3 collision events
Evaluation after the GNN	`r16443` entries 0–24 999 (1 920 at PU200)	the same 2 550 simulated events; the same 2 500 Run 3 events

2.4 Performance metrics

In order to assess the performance of the algorithm we define several key metrics which are detailed below.

2.4.1 Vertex-vertex resolution

The vertex-vertex resolution $\sigma_{\text{vtx-vtx}}$ measures how well the algorithm separates nearby vertices. For each event, we calculate the differences $\Delta z = z_i - z_j$ between all pairs of reconstructed vertices. The distribution has a dip near zero because some close pairs are reconstructed as one vertex. We fit this distribution with

$$f(\Delta z) = \frac{a}{1 + e^{\,b(\sigma - |\Delta z|)}} + c, \tag{2.4}$$

and define $\sigma_{\text{vtx-vtx}}$ as the fitted parameter σ. At $|\Delta z| = \sigma$, the fitted curve is halfway between its lower and upper levels, c and $a + c$ (Figure 2.5). Some pairs closer than $\sigma_{\text{vtx-vtx}}$ can still be separated. This quantity is also different from the error on a single vertex position. Fake and split vertices can change the shape of the distribution and affect the fit.

Table 2.3: Data and training settings for the production HL-LHC model. [MM: Values for the frozen model s8 in red.]

Setting	Value
Training sample	5.17M PU200 events, $\|\eta\| < 4$
Train/validation/test split	96%/3%/1%
Architecture	~~Redesigned U-Net, 280 channels, 4 latent channels~~ [MM: U-Net v2, 80 channels, 1 latent channel]
Number of parameters	~~3.55M~~ [MM: 439 593]
Batch size	128
Phase 1	~~2 epochs~~ [MM: 1 epoch], learning rate 10^{-3}
Phase 2	~~2 epochs~~ [MM: 1 epoch, checkpoint at step 450 000 of 465 624], learning rate 10^{-4}
Phase 2 schedule	3000-step warm-up, then cosine decay
Optimizer and loss	Adam, mean-squared error
Maximum gradient norm	1.0

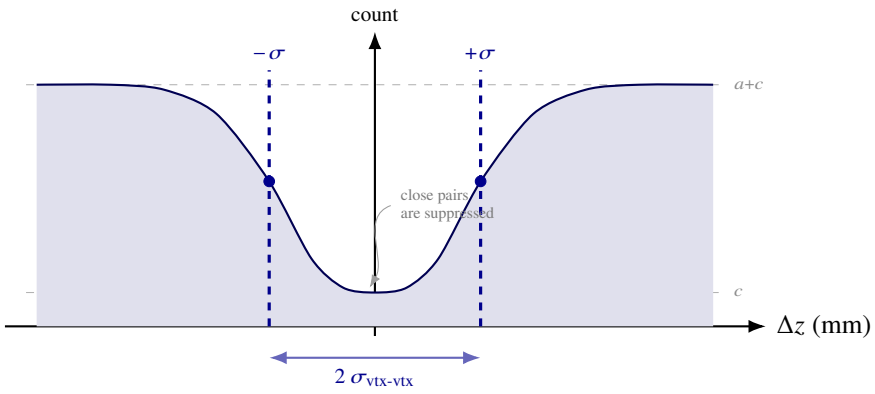


Figure 2.5: Definition of $\sigma_{\text{vtx-vtx}}$ from the pairwise Δz distribution. The sigmoid reaches the midpoint between c and $a + c$ at $|\Delta z| = \sigma_{\text{vtx-vtx}}$. The suppression near zero reflects the loss of close vertex pairs during reconstruction.

2.4.2 Truth matching and vertex categories

We use truth vertices with at least two associated reconstructed tracks passing the sample selection (Section 2.1). We form all reconstructed-truth pairs within a matching window $|z_{\text{reco}} - z_{\text{truth}}| \leq w$, sort them by distance, and assign them one to one, starting with the closest pair. Each vertex can enter only one such assignment.

We then classify the reconstructed vertices:

- *Clean*: assigned to a truth vertex, with no additional unclaimed truth vertex in the window.
- *Merged*: assigned to a truth vertex, with at least one additional unclaimed truth vertex in the window. These neighbours are assigned to the same reconstructed vertex.
- *Split*: unassigned, with a truth vertex in the window that has already been assigned to another reconstructed vertex.
- *Fake*: no truth vertex in the window.

We label the truth vertices separately, using the same assignments:

- *Clean*: matched to a reconstructed peak in the first, one-to-one step. This label stays the same if another truth vertex is later assigned to that peak.

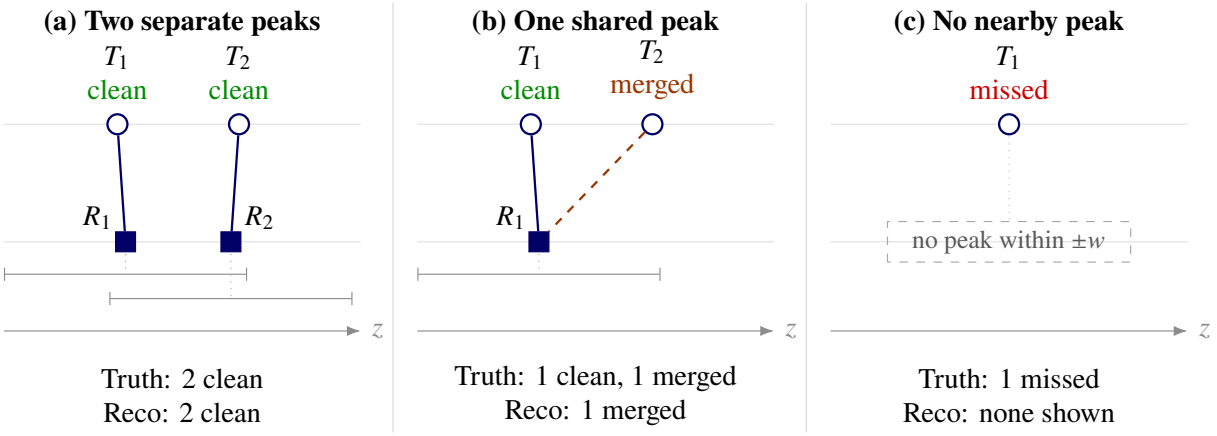


Figure 2.6: Examples of truth-vertex labels. Circles are truth vertices and squares are reconstructed peaks. Solid lines show the first, one-to-one matches; the dashed line shows a truth vertex assigned afterwards. The horizontal bars mark the matching windows $z_{\text{reco}} \pm w$. (a) Each truth vertex has its own peak, so both truth vertices and both peaks are clean, even though the windows overlap. (b) One peak is shared: T_1 is clean, T_2 is merged, and R_1 is merged. (c) No peak is within w of the truth vertex, so it is missed.

- *Merged*: left without a match in the first step, then assigned to a peak that already has a truth match.
- *Missed*: no reconstructed peak lies within the matching window.

For example, suppose two truth vertices share one reconstructed peak. The closer truth vertex is matched first and is labelled clean. The other truth vertex is labelled merged. The reconstructed peak is also labelled merged, because it represents both truth vertices. Thus one merged peak gives one clean truth vertex and one merged truth vertex. Figure 2.6 shows this case alongside two separate matches and a missed truth vertex.

The first matching step matters when nearby truth vertices each have their own reconstructed peak. They remain clean even if their matching windows overlap. An earlier rule checked each peak separately and could label both as merged just because each window contained both truth vertices.

The efficiency reported over a sample is

$$\varepsilon = \frac{\sum_e (N^e_{\text{truth,clean}} + N^e_{\text{truth,merged}})}{\sum_e N^e_{\text{truth}}}. \tag{2.5}$$

The sums run over the selected events. We add the counts before taking the ratio. Each truth vertex assigned to a merged peak counts as found. We also report *one-to-one efficiency*, where each truth vertex must have its own reconstructed match. The fake rate is the number of fake vertices divided by the number of selected events. We use the term surplus vertices when counting fake and split vertices together. [MM: Split vertices are included in this efficiency: a truth vertex inside the window of a split vertex counts as found. This changes no number, because every such truth vertex already has its own one-to-one match, so Eq. (2.5) equals the fraction of truth vertices with at least one reconstructed vertex within w.]

These results depend on the matching window. For the main comparison, we use the same fixed window for both methods. It is based on their single-vertex position precision, as explained below. [MM: The window is $w = 0.17$ mm for every vertex-matching result in this note, in both regimes. At PU200 it is three times

the 68% single-vertex residual (Section 4.1.1, Figure 4.5). For consistency with ITk performance studies, which quote only the hard-scatter vertex, we also give the hard-scatter result with a 0.1 mm window.] We also check how the results change when the window is varied (Section 4.1.2).

3 Datasets

3.0.1 HL-LHC PU200 simulation

The HL-LHC studies use fully simulated $\sqrt{s} = 14$ TeV $t\bar{t}$ events generated with the `mc21` campaign and the ITk detector layout at nominal pile-up $\mu = 200$. The original sample contains 99 800 events, with about 927 reconstructed tracks and 127 stored truth vertices per event. The stored truth vertices have at least one associated reconstructed track; about 99 per event have at least two tracks and enter the efficiency denominator. `ActualNumOfInt` gives the expected number of in-time interactions, denoted by μ. It is not the number of stored truth vertices. It ranges from 190 to 210 and has a mean of 200. The PU200 evaluation uses $185 \leq \mu \leq 215$, which includes the full sample.

A larger set of about 2.9×10^6 events also contains High-Granularity Timing Detector (HGTD) information. The 2.74×10^6 events centered at $\mu = 200$ were used for earlier training studies. Two other datasets contain events over a broad pile-up range ($\mu \approx 0$ to 210). They are kept for evaluation and robustness studies. Only about 3.9% of tracks in these samples have a valid HGTD time because the tracks are restricted to $|\eta| < 2.5$. This fraction rises to 28.8% in the samples with full $|\eta|$ acceptance described below (Appendix E.4). The vertex finder in this note does not use the timing information. The earlier timing study used the $|\eta| < 2.5$ samples. No timing-enabled model has yet been trained on the sample with 28.8% coverage, so that case remains open (Appendix E.4).

3.0.2 Samples with full $|\eta|$ acceptance

The earlier samples include tracks only up to $|\eta| < 2.5$. A new production uses the full ITk acceptance. The production model and all HL-LHC results in Chapter 4 use these samples. For the matched `r16443` comparison, the two productions contain the same events and have the same beam spot and pile-up values. The AMVF vertex positions are also the same. What changes is the reconstructed-track information saved in the ntuples and the resulting truth-vertex track counts.

The PU200 training sample contains 5.17×10^6 events with pile-up close to $\mu = 200$. Two datasets with a flat-μ distribution are used only for evaluation. [MM: This holds for the density model. The GNN v2 is trained on the `r16443` entries from 25 000 on and on `r16638`; entries 0–24 999 of each file are used only for evaluation (Table 2.2).] The new production applies the same track selection over the full detector. The earlier production used a lower p_{T} threshold and looser d_0 selection in part of the central region. A direct comparison of the two productions is given in Appendix A.2.

The forward tracks have worse z_0 resolution than central tracks. Their main effect is to add vertices that do not have two central tracks, rather than to improve the measured vertex position. We therefore refitted the target-resolution model of Section 2.2.3 on the new sample. The resulting `hllhc_alleta` preset is used for training (Table B.3).

Conversion to HDF5. Vertex-finder training reads a flat HDF5 layout rather than ROOT. A two-pass converter stores a padded track tensor $(7, N_{\text{trk}})$ and a two-channel target histogram $(2, 1000)$ for each 40 mm sub-event. It also stores the truth PV z positions for each event. The targets are created during conversion with the resolution model of Section 2.2.3. No KDE is stored because computing it for the full PU200 sample would be too slow.

Most of the padded track tensor is empty. LZF compression reduces the file size by about a factor of nine. The per-file datasets are combined and padded to a common track length. The combined data are shuffled once with a fixed seed and split 0.96/0.03/0.01 into training, validation, and test samples ($59.6 \times 10^6 / 1.86 \times 10^6 / 0.62 \times 10^6$ sub-events).

3.0.3 Run 2 and Run 3 data

The low-pile-up validation uses three datasets. The Run 2 simulated sample is a flat HDF5 file of 51 000 events (12 sub-events each), with precomputed KDEs and target histograms for each sub-event. It is used to train and evaluate the low-pile-up model. The Run 2 collision sample contains 298 639 events with a pile-up peak at $\mu \approx 25$ to 30 and a beam spot near $z = -2.5$ mm. The Run 3 collision sample peaks at $\mu \approx 60$. On collision data there is no MC truth, so AMVF vertices with $n_{\text{trk}} \geq 2$ serve as the reference, and the beam-spot z offset is subtracted before comparison. For the Run 2 and HL-LHC simulated samples, MC truth is used as the reference for both PV-Finder and AMVF.

4 HL-LHC PU200 results

This chapter presents the vertex-finding results at PU200. We first explain how the performance is measured, then compare PV-Finder with AMVF on the same events. We also study merged, missed and fake vertices, and test cuts to remove fake peaks. Here the vertices are compared using their positions. Chapter 7 uses the assigned tracks to compare them.

4.1 Performance relative to AMVF

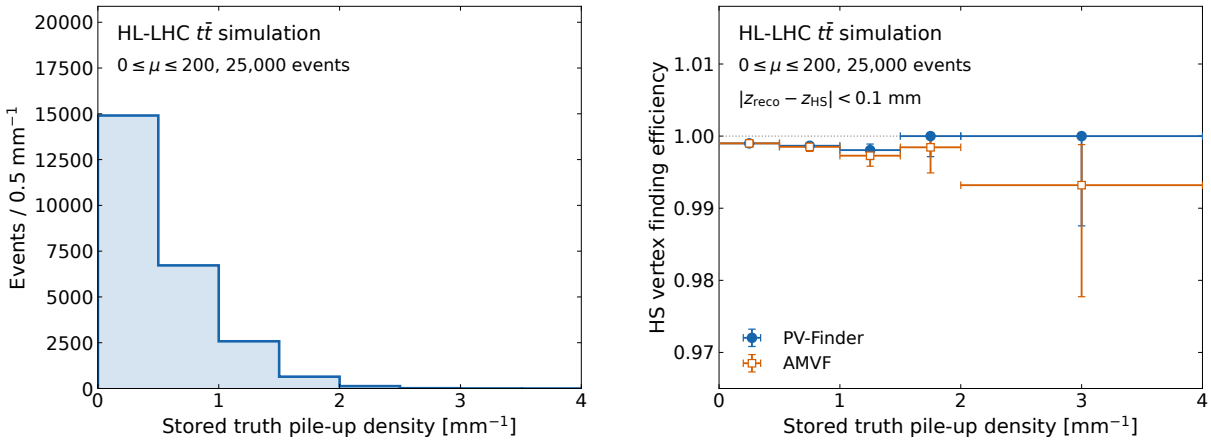
The production model was trained on events with pile-up close to $\mu = 200$. These events come from the 5.17×10^6-event sample with full ITk track acceptance described in Section 3.0.2. The separate flat-μ sample is used only for validation. [MM: The density model never saw it; its first 25 000 entries are used for every HL-LHC result.] The 25 000 validation events cover $\mu = 0$ to 200. The PU200 results below use the 1 920 events that pass $185 \leq \mu \leq 215$. Their actual range is 185 to 200, since this validation sample has no events above $\mu = 200$.

Here μ gives the expected number of pile-up interactions in a bunch crossing. The actual number fluctuates from event to event. A truth PV is a collision position recorded in the simulation. We count only truth PVs with at least two selected reconstructed tracks. In the selected events, $\langle\mu\rangle = 192.5$, while the mean number of these truth PVs is 111.3. There are 213 738 such truth PVs in total.

We compare the two methods using a fixed matching window of ~~0.1694 mm, or about 0.17 mm~~ [MM: 0.17 mm]. This is three times the larger of their 68% position-error values, 56.5 μm (Section 4.1.1[MM: , Figure 4.5: 3×55.6 μm = 0.167 mm for PV-Finder and 3×56.5 μm = 0.169 mm for AMVF, rounded to 0.17 mm]). Both methods use the same events and truth vertices. The window stays fixed when the PV-Finder resolution fit changes.

With this choice, PV-Finder gives ~~(83.16 ± 0.08)%~~ [MM: (83.06 ± 0.08)%] efficiency and AMVF gives ~~(79.80 ± 0.09)%~~ [MM: (79.85 ± 0.09)%] (Table 4.1). The difference is ~~3.36 ± 0.06~~ [MM: 3.20 ± 0.06] percentage points. Merged truth vertices count as found. ~~PV-Finder also gives 0.60 ± 0.09 fewer fake and split vertices per event. Counting these two categories together includes extra peaks from splitting one vertex.~~ [MM: PV-Finder also gives 1.01 ± 0.09 fewer fake vertices per event (18.64 against 19.65; fake fraction 18.48% against 20.08%) and 0.38 ± 0.02 more split vertices (0.56 against 0.18). On the second evaluation sample, `r16638` (1 888 PU200 events), the efficiencies are 83.22% and 80.13% (+3.08 ± 0.06 points). All numbers in this chapter use the frozen model s8 and are to be confirmed with the final recompute at the 13 November peak cuts (Table 2.2).]

The efficiency and vertex counts use ~~an integral threshold of 0.30 and a minimum peak height of 0.03~~ [MM: the frozen configuration of Table 2.2]. The position is a weighted mean within three bins of the peak maximum. No additional classifier is used to reject peaks in this comparison. The classifier study is described in Section 4.1.4. ~~For the resolution plot, the integral cut is raised to 0.50 to remove more small~~



(a) Local truth-vertex density.

(b) Hard-scatter vertex finding efficiency.

Figure 4.1: Hard-scatter vertex finding on all 25 000 HL-LHC validation events over $0 \leq \mu \leq 200$. One HS truth vertex is counted per event. The local density counts stored pile-up truth vertices with $|z - z_{\text{HS}}| < 2$ mm, excluding the HS, divided by 4 mm. No additional track-multiplicity cut is applied to these truth vertices. An event passes if at least one reconstructed vertex satisfies $|z_{\text{reco}} - z_{\text{HS}}| < 0.1$ mm. Vertical error bars in (b) show 68.27% exact binomial intervals; horizontal bars show the bin widths. The interval from 2 to 4 vertices/mm is combined in (b).

~~peaks and make the dip clearer. All other peak cuts stay the same.~~ [MM: The resolution fit uses the same peaks.] Full settings are given in Table A.1.

We also count events with at least one reconstructed vertex within 0.1 mm in z of the true hard-scatter (HS) vertex, using the positional definition in the ITk study [7]. [MM: For consistency with ITk performance studies, which quote only the hard-scatter vertex, the hard-scatter results use this 0.1 mm window; all other results use 0.17 mm.] On the same 1 920 events, PV-Finder finds 1 918 HS vertices and AMVF finds 1 916. The efficiencies are $(99.90^{+0.07}_{-0.14})\%$ and $(99.79^{+0.10}_{-0.16})\%$, respectively, with 68.27% exact binomial intervals. The difference is not statistically significant. This counts one HS vertex per event, while the efficiencies above include all eligible truth vertices. It measures finding the HS position. Selecting it among the reconstructed vertices is a separate task.

Figure 4.1 extends the HS comparison to all 25 000 validation events over $0 \leq \mu \leq 200$. Each event has exactly one stored HS truth vertex, identified by `TruthVertex_type = 1`. No track-multiplicity cut is applied to the HS. With the same reconstruction selections, PV-Finder finds the HS in 24 971 events (99.884%) and AMVF in 24 966 (99.864%). The figure shows the local truth-vertex density and the HS finding efficiency in bins of this density. The highest-density bin contains 147 events and has a larger statistical uncertainty.

Figure 4.2 shows the number of reconstructed vertices over the full pile-up range. The truth selection uses the full ITk track acceptance. The comparison with the earlier central-track sample is kept in Appendix A.2. Results with a wider matching window and another reconstruction of the simulated sample are given in Appendix D.

Table 4.1: PV-Finder and AMVF on the same 1 920 PU200 events, using a fixed ~~0.1694 mm~~ [MM: 0.17 mm] matching window. Truth vertices require at least two selected tracks. Efficiency counts merged truth vertices as found. Fake and split vertices are counted together in the last column. The uncertainties are statistical, from resampling the events. Each resample uses the same events for both methods.

Method	Efficiency [%]	Fake + split vertices/event
PV-Finder	~~83.16 ± 0.08~~ [MM: 83.06 ± 0.08]	~~19.27 ± 0.10~~ [MM: fakes 18.64 ± 0.10, splits 0.560 ± 0.016]
AMVF	~~79.80 ± 0.09~~ [MM: 79.85 ± 0.09]	~~19.87 ± 0.10~~ [MM: fakes 19.65 ± 0.10, splits 0.184 ± 0.010]
Difference	~~+3.36 ± 0.06~~ [MM: +3.20 ± 0.06] points	~~−0.60 ± 0.09~~ [MM: fakes −1.01 ± 0.09, splits +0.376 ± 0.019]

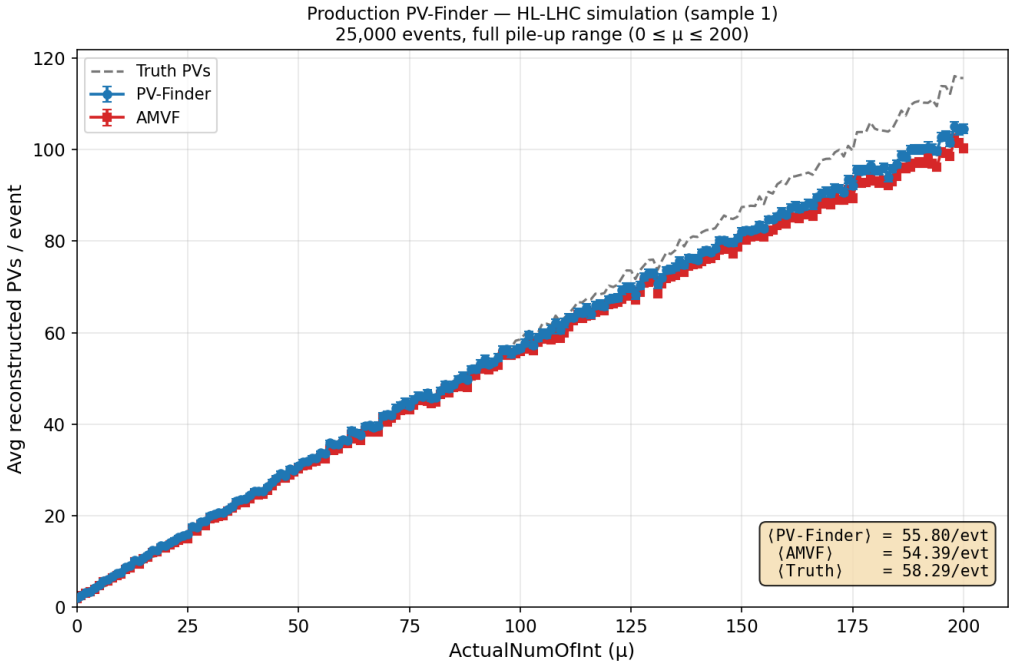


Figure 4.2: [MM: Made with the earlier 3.55M-parameter model; to be replaced by the s8 version.] Number of reconstructed vertices versus μ for PV-Finder and AMVF on the validation sample. This plot uses all 25 000 evaluated events. The PU200 comparison in Table 4.1 uses the 1 920 events passing $185 \leq \mu \leq 215$. The truth line counts vertices with at least two selected reconstructed tracks.

4.1.1 Vertex-vertex resolution

PV-Finder gives a vertex-vertex resolution of ~~0.229 ± 0.003 mm~~ [MM: 0.217 ± 0.003 mm], compared with 0.305 mm for AMVF. Figure 4.3 shows the fit to the reconstructed-vertex pair separations. The uncertainty comes from the fit. ~~As described above, the plot uses the higher integral cut of 0.50. The result with the 0.30 cut is given in Appendix D.1.2.~~ [MM: The fit uses the same peaks as the efficiency. For AMVF the fitted lower level sits at its bound, $c = 0$; a fit weighted by $\sqrt{N}$ gives 0.298 mm and an unbounded fit 0.282 mm, while PV-Finder is stable within 0.001 mm.]

The separation fits use the PU200 events and 300 bins over $-6 < \Delta z < 6$ mm. Each bin is 0.04 mm wide, the same as a bin in the network output. Wider bins describe the sharp change at the sides of the dip less well. The weighted peak positions can lie between output bins, but sharp peaks often have positions close

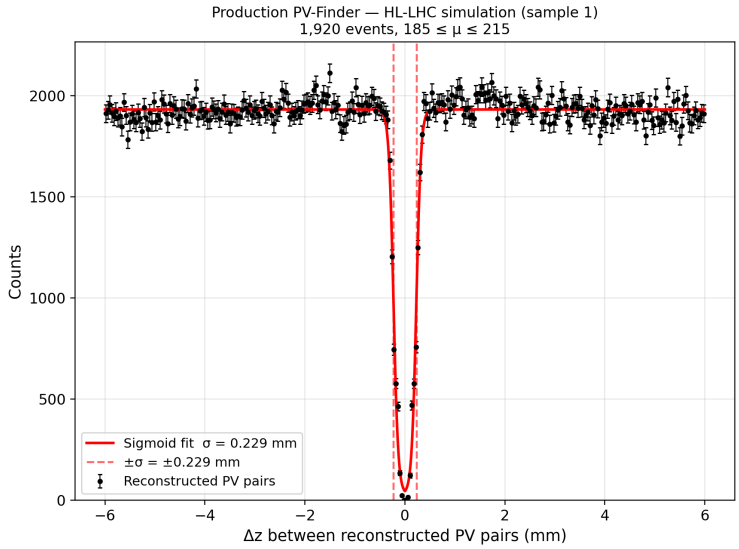


Figure 4.3: [MM: Made with the earlier 3.55M-parameter model at integral 0.50; to be replaced by the s8 version on the counted peaks (0.217 mm).] Pairwise Δz distribution and sigmoid fit for PV-Finder on the PU200 evaluation sample. The fit uses an integral threshold of 0.50 to remove more small peaks and make the dip clearer. It gives $\sigma_{\text{vtx-vtx}} = 0.229 \pm 0.003$ mm. The efficiency and vertex counts use the lower integral cut of 0.30 and a fixed 0.1694 mm truth-matching window.

to the grid. In a separate check using the same saved positions, changing the plotting bins from 0.05 to 0.04 mm largely removed a repeated pattern in the pair counts. No reconstructed positions were changed. Appendix D.1.2 gives this check.

We also measure the error on each reconstructed position, using the ~~0.30 integral cut~~ [MM: same peaks]. For this study, we choose the one-to-one assignment that matches the largest number of vertices within 1 mm of truth. If several assignments give that number of matches, we choose the one with the smallest total distance. This differs from selecting the closest remaining pair at each step, as done for the performance categories.

For 68% of matched vertices, the absolute position difference is below ~~55.05 ± 0.23 μm~~ [MM: 55.58 ± 0.24 μm] for PV-Finder and ~~56.45 ± 0.26 μm~~ [MM: 56.45 ± 0.24 μm] for AMVF (Figure 4.4). These values are measured directly from the residuals. They are not fitted Gaussian widths. Their statistical uncertainties come from resampling whole events. The signed distributions are approximately centred on zero, with similar central widths on either side. A single Gaussian does not describe them well. The two methods find individual vertex positions with similar precision.

The different windows serve different purposes. The 1 mm assignment window allows us to measure position errors without the tighter cut used for efficiency. The main efficiency and fake counts use the fixed ~~0.1694 mm~~ [MM: 0.17 mm] truth-matching window for both methods. It is three times the larger 68% position-error value, rounded ~~to four decimal places~~ [MM: to 0.17 mm (Figure 4.5)]; it is not a Gaussian three-standard-deviation cut. [MM: The 1 mm assignment radius must stay wider than the window, or the derivation of the window would be circular. At low pile-up the same 0.17 mm is about 2.4 times the 68% residual (Chapter 6).] The 2 mm window in Section 2.2.3 is used only to fit the training target widths. Finally, ±6 mm is the range of the pair-separation fit. The fitted separation scale does not set the main truth-matching window.

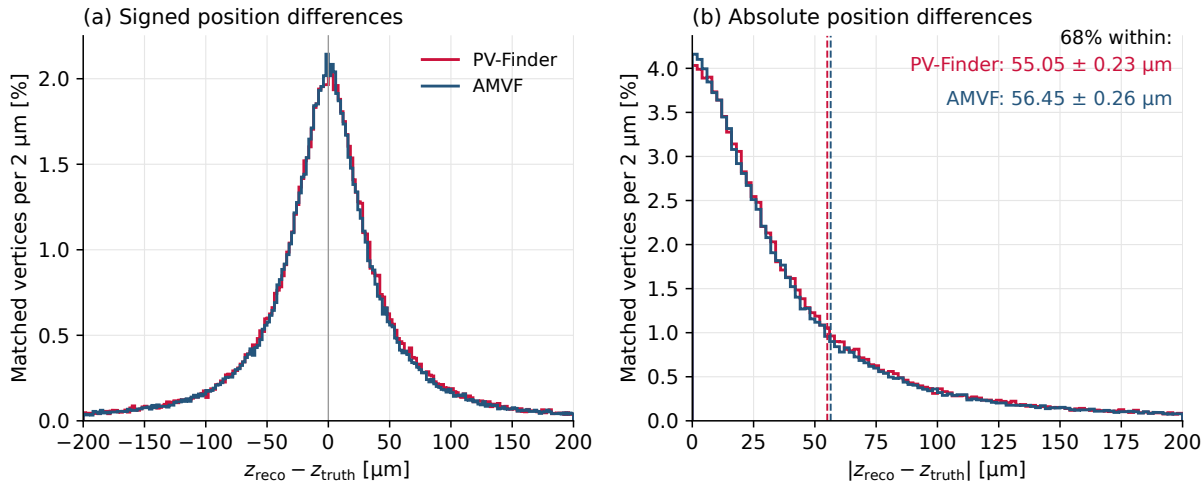


Figure 4.4: [MM: Made with the earlier 3.55M-parameter model; to be replaced by the s8 version (55.58 and 56.45 μm).] Signed (left) and absolute (right) differences between reconstructed and truth vertex positions on the same 1 920 events. Vertices are assigned one to one within 1 mm, and PV-Finder uses the 0.30 integral cut. Each distribution is normalized to all its matched vertices, including those outside the displayed range. The dashed lines mark the 68% values, 55.05 ± 0.23 μm for PV-Finder and 56.45 ± 0.26 μm for AMVF. Their statistical uncertainties come from 2 000 resamples of whole events.

4.1.2 Checks of the matching definition

The efficiency in Table 4.1 counts merged truth vertices as found. If each truth vertex must have its own reconstructed match, the efficiencies are ~~(73.43 ± 0.09)%~~ [MM: (73.36 ± 0.09)%] for PV-Finder and ~~(70.07 ± 0.09)%~~ [MM: (70.10 ± 0.09)%] for AMVF. Both fall by about 9.7 percentage points. The difference is still ~~3.37 ± 0.06~~ [MM: 3.26 ± 0.06] points, almost the same as before. The full comparison, including fake and split counts separately, is in Table D.1 in Appendix D.

[MM: The rest of this section was made with the earlier 3.55M-parameter model at 0.1694 mm. With s8, the efficiency margin is +3.0 to +3.3 points for windows of 0.10 to 0.20 mm (+3.20 at 0.17 mm); the remaining numbers are to be updated with the final recompute.] We also vary the window from 0.1 to 1.0 mm (Figure 4.6). PV-Finder has higher efficiency and fewer fake vertices over this range. At 0.5 mm, its efficiency is 0.86 points higher when merged truth vertices count as found. The difference is 3.01 points when each truth vertex needs its own match. AMVF gains more from including merged truth vertices at this wider window. When fake and split vertices are counted together, PV-Finder gives fewer only for windows below about 0.60 mm.

We make another comparison with the same average number of reconstructed vertices per event. We raise the PV-Finder height cut until it gives 97.87 vertices per event, equal to AMVF. At $w \simeq 0.17$ mm, its efficiency is still 2.89 ± 0.06 points higher. It also gives 3.23 ± 0.09 fewer fake and split vertices per event.

With many vertices in an event, some matches can happen by chance. To check the effect, we shift all reconstructed positions by 3 mm and repeat the matching. Even after this shift, the efficiencies are 18.6% for PV-Finder and 18.4% for AMVF at $w \simeq 0.17$ mm. They rise to 45.4% and 46.7% at $w = 0.5$ mm. These values include merged truth vertices and show that a wider window accepts more chance matches.

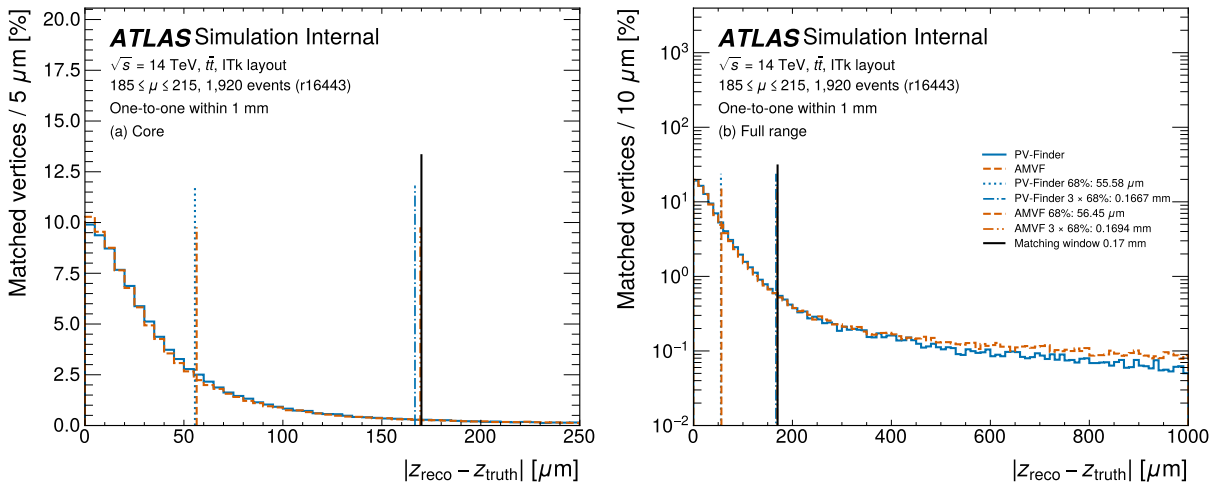


Figure 4.5: [MM: Justification of the 0.17 mm matching window. Absolute residual $|z_{\text{reco}} - z_{\text{truth}}|$ of single vertices on the 1 920 PU200 events (frozen s8 model and AMVF), with vertices assigned one to one within 1 mm: (a) core, linear scale; (b) full range, logarithmic scale. The dotted lines mark the 68% values (55.58 μm for PV-Finder, 56.45 μm for AMVF), the dash-dotted lines three times these values (0.1667 and 0.1694 mm), and the solid line the 0.17 mm window.]

The shift changes the matching problem, so these values do not measure how many matches in the original sample happened by chance. We do not subtract them from the reported efficiencies.

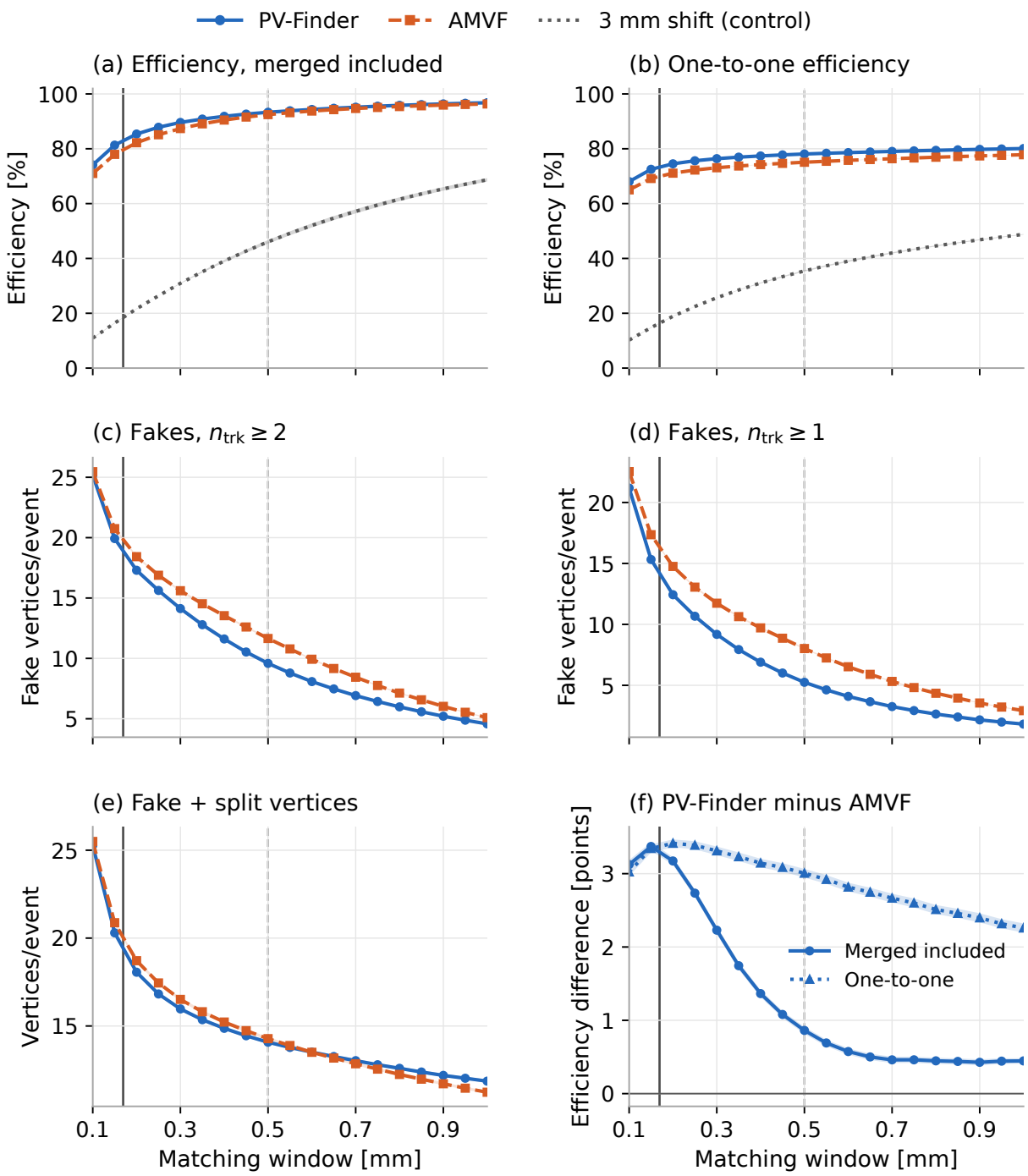


Figure 4.6: Results for different matching windows on the same 1 920 PU200 events. Panels (a) and (b) show efficiency with and without merged truth vertices counted as found. Panels (c) and (d) show fake counts when the truth vertices used for matching must have at least two or one selected reconstructed tracks, respectively. Panel (e) counts fake and split vertices together. Panel (f) shows the efficiency difference between the methods. Coloured bands show statistical uncertainties. The grey curves in (a) and (b) show the 3 mm position-shift check; the grey band spans the values for the two methods. Vertical lines mark the main 0.1694 mm window (solid) and 0.5 mm (dashed).

4.1.3 Merged, missed and fake vertices

Nearby truth vertices can produce one peak and be reconstructed as merged. Missed vertices usually have few tracks and no clear peak nearby. Fake peaks pass the cuts but have no selected truth vertex within the matching window. Some lie near real interactions with only one selected track, which are excluded from the truth reference (Appendix D.1.1).

In the production evaluation sample, 13.4% of neighbouring truth pairs are closer than the measured vertex-vertex resolution of 0.229 mm [MM: (earlier model; s8 gives 0.217 mm, fraction to be recomputed)]. These pairs are difficult to separate. This does not set a limit on the reported efficiency, which counts merged truth vertices as found. The current GNN cannot separate a merged pair because it only assigns tracks to peaks already produced by the finder. Appendix D.4 gives examples from an earlier model and further checks.

4.1.4 Removing fake peaks

[MM: The frozen configuration has no height cut. With s8, an integral cut alone (0.186) gives the same efficiency, fake rate and splits as an integral cut of 0.184 plus a height cut of 0.0184, within 0.01 points and 0.02 fakes per event in every μ bin. At those cuts the height cut was the binding cut for 0.1% of the peaks. The rest of this paragraph refers to the earlier model.] The production finder requires a peak height of at least 0.03. On a subset of 384 PU200 events, this cut removes 1.06 fake vertices per event while reducing efficiency by 0.21 percentage points. This check uses the same 0.229 mm matching window before and after the cut. The window is wider than the one used in the main comparison. Stricter height cuts remove more real vertices because real and fake peaks can have similar heights.

We also tested a gradient-boosted-tree (GBT) classifier using the shape of each peak and information about nearby peaks. It removes more fakes, but also removes some real vertices. It is not used in the main result. The study still needs a test on independent events: the training and test sets can contain peaks from the same event. The cuts, results and sample details are given in Appendix D.5.

4.1.5 Satellite peaks

Some fake or split peaks lie beside a larger peak. We call them satellites. A small rise on the side of a predicted peak can cause the peak finder to split it into two. An earlier study found that these extra peaks produced a bump in the distribution of distances between reconstructed vertices. Calculating each position from the bins near its peak maximum reduces this bump without removing any peaks. We use this local centroid in production. With the production settings, the remaining bump depends on how the baseline is fitted and is difficult to separate from that uncertainty. Appendix D.6 gives the detailed study.

5 Inference timing and scaling with pile-up

At PU200, PV-Finder gives a higher clean-vertex yield and fewer fake vertices than AMVF under the matching rules of Chapter 4. Its running time is another reason to study the method. AMVF iteratively finds and fits vertex candidates. This takes longer as pile-up increases. PV-Finder uses the same network for twelve 40 mm intervals in each event. Once the tracks are stored in fixed-size input tensors, the network size and number of operations stay the same as the number of vertices changes. A full comparison must also include the time to prepare these inputs and find the peaks.

We timed an earlier model on an NVIDIA A100 GPU using one sub-event from each of 12 150 simulated events. The sample contains 150 events in each integer pile-up bin from 0 to 80. We used only the first of the twelve sub-events from each event. The test used batch size one and ten warm-up calls. CUDA was synchronized before and after the network call. We did not time input loading, transfer to the GPU or peak finding.

The mean time was 4.5 ms per sub-event, with a standard deviation of 1.2 ms across calls. The mean changed little over the tested pile-up range (Figure 5.1). This result is for the earlier model and covers only the network call. It does not give the time for a complete event with the production model at PU200.

In a separate profiler run on 1 500 sub-events, the reported GPU operator times added up to about 0.38 ms per call. These times cannot separate GPU work from input preparation in the 4.5 ms measurement. We also need to measure the effect of batching.

The Run 3 tracking-software paper [2] shows that AMVF takes longer to reconstruct primary vertices as pile-up increases on an AMD EPYC 7302 CPU. The ACTS implementation reduces this time by more than a factor of 1.5. The physics results are the same. These CPU measurements cannot be compared directly with the GPU test above. A full comparison at PU200 needs timing measurements of the current algorithms. For PV-Finder, these must include input preparation and peak finding.

The HGTD study in Appendix E.4 uses the arrival times of tracks measured by the detector. We have not yet tested a model using this information on the full-acceptance sample.

5.1 Athena Integration and Timing:

5.1.1 Athena implementation details:

The PV-Finder and the GNN track-to-vertex association algorithms have been implemented within the ATLAS software framework, Athena. This integration also facilitates the comparison of PV-Finder to AMVF on equal terms such that we can time the same events on both algorithms, from raw tracks to vertices with tracks linked.

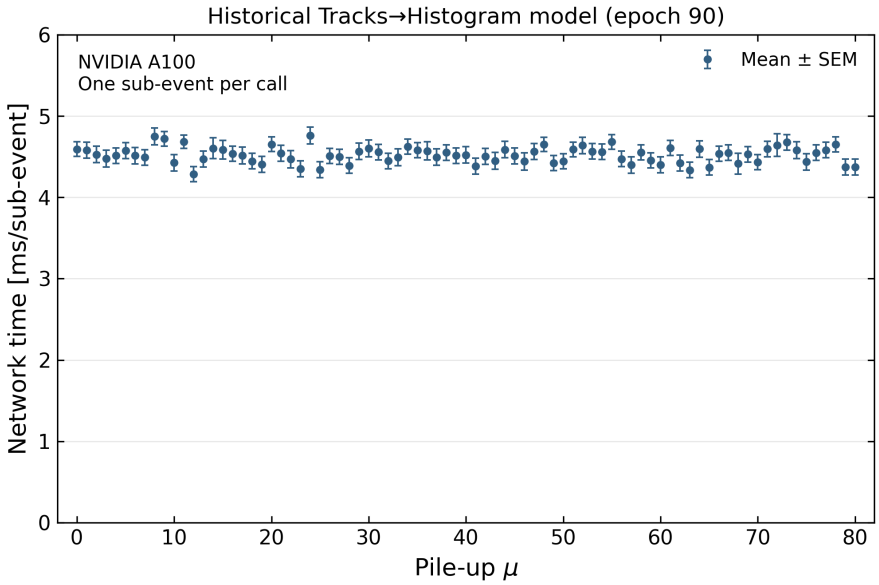


Figure 5.1: Mean network time per sub-event for an earlier PV-Finder model on an NVIDIA A100. Each point is the mean of 150 calls. The error bars show the standard error on the mean. Only the first sub-event of each event was timed. The times exclude input loading, transfer to the GPU and peak finding.

The code is structured as two packages in InnerDetector/InDetRecTools, InDetNNPriVxFinderTool and InDetGNNTrackVertexAssociationTool. The former is an implementation of the IVertexFinder interface, so that one can call it in exactly the same way as AMVF. The latter is a tool that takes a TrackParticleContainer and a VertexContainer reference and links the two according to the GNN Track-to-vertex association algorithm.

Both algorithms are composed of smaller tools: InDetNNPriVxFinderTool is composed of PVFinderTrackSelectionTool, PVFinderTensorBuilderTool, PVFinderInferenceTool, PVFinderPeakFinderTool, PVFinderVertexBuilderTool. InDetGNNTrackVertexAssociationTool is similarly composed of GNNTTVAGraphBuilderTool and GNNTTVAInferenceTool. The flow of these subtools in a call of InDetNNPriVxFinderTool's findVertex method is illustrated in Figure 5.2. The Track-to-vertex association tool is used is a configurable property, as are the other sub-tools.

The PVFinderTrackSelectionTool takes in tracks and applies the TightPrimary cut and extracts the parameters that PV-Finder uses. PVFinderTensorBuilerTool turns this data structure into the tensor that is read by ONNXRuntime to do inference. After inference, peak finding is ported from the python as a tool and then PVFinderVertexBuilderTools converts the PV-Finder-native Peaks object into an xAODVertex, filling in quantities that are not native to PV-Finder such as fit quality with sensible nulls. GNNTTVATrackBuilderTool takes tracks and vertices to produce the graph that is is read by ONNXRuntime to do inference.

Both models' inference is done with OnnxRuntimeInferenceTool. PV-Finder checkpoints were exported with torch.onnx.export at opset 17, while the GNN checkpoints needed the dynamo exporter and opset 20. The ONNX file and the execution provider (CPU, CUDA) are configurable properties of the inference tools, so changing these requires no compilation.

The Athena implementation was verified against the python implementation on fifty events using the PU200

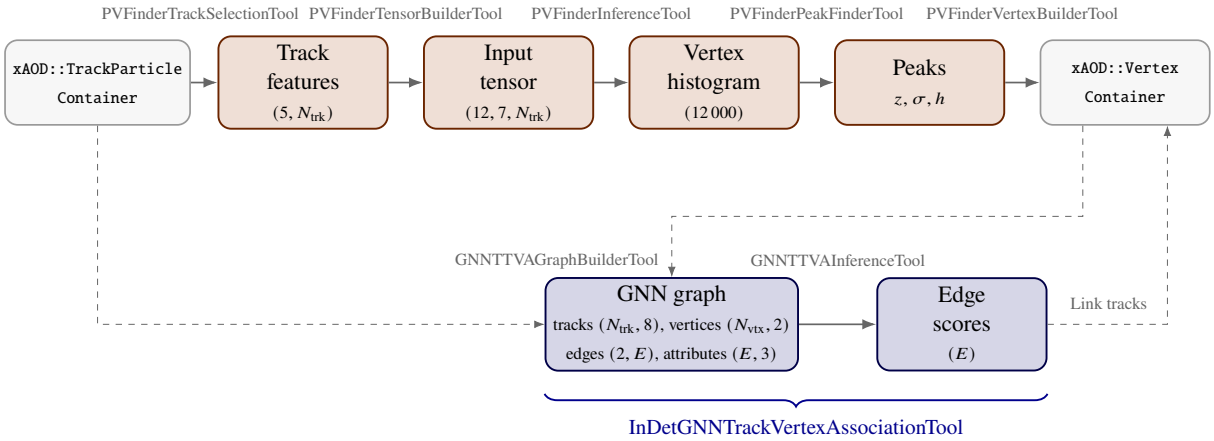


Figure 5.2: Flow of data in PV-Finder Athena tool. Solid lines represent separate tools. The three dashed lines represent the TTVA tool, which is a configurable property, but represented by InDetGNNTrackVertexAssociationTool here.

checkpoint, in which 5037 peaks were found and 962060 edges were considered by the GNN. Using the CPU, for the peaks, the maximum |dz| was 7.7e-06 mm, maximum |dsigma| was 5.9e-08 mm, and bin agreement was exact. This is at the level of floating point rounding. For the GNN, max|dlogit| was 1.8e-03, but no decisions flipped for the 962060 edges at the 0.5, 0.9, 0.99, and 0.999 cutoffs. As for the GPU, apart from one peak where |dz| was 1.6e-3 mm and |dsigma| was 2.0e-3 mm, all peaks agreed to better than 6e-6 mm, with bin agreement and edge agreement exact at each cutoff.

5.1.2 Timing studies

Timing studies were conducted by setting PV-Finder and AMVF as the Vertex Finder tool in InDetPriVxFinder and timing them on the same events from a tracks to the complete vertex container with the associated tracks. The timing is done by a Gaudi auditor hooking the start and end of each algorithm's execute.

Figure 5.3 shows the timing comparison when PV-Finder PU200 model is run with CPU and GPU. All were run on the same lxplus-gpu node with the CPU being Intel Xeon Silver 4216 @ 2.10 GHz and the GPU being NVIDIA Tesla T4. The samples are four run 3 collision data and six run 4 monte carlo, 200-500 events from each. ~~PV-Finder is faster at all pile up ranges, and roughly 9 times faster at $\mu = 200$.~~ [MM: On the GPU, PV-Finder is faster than AMVF at all pile-up values, and roughly 9 times faster at $\mu = 200$. On the CPU, the model timed here (the earlier 3.55M-parameter model, padded input tensor) is slower than AMVF at PU200: 2.4 s per event against about 0.65 s (Table 5.1). The speed-up claim therefore holds on the GPU only.]

Additionally, Table 5.1 breaks down the run time by each tool, averaging each time over 597 events. The CPU inference takes up 98.6% of the run time while with CUDA it is 56.1%. The majority of the non-inference time is accounted for by track selection at 39.3%. The GPU version is also 32 times faster.

[MM: **CPU projection for the frozen chain.** The frozen HL-LHC chain (s8 and the GNN v2) has not yet been timed in Athena. Outside Athena, on one CPU thread of an AMD EPYC 7402, s8 with packed track input (ONNX Runtime) takes 174 ms per PU200 event, 10.9 times faster than the timed model with padded

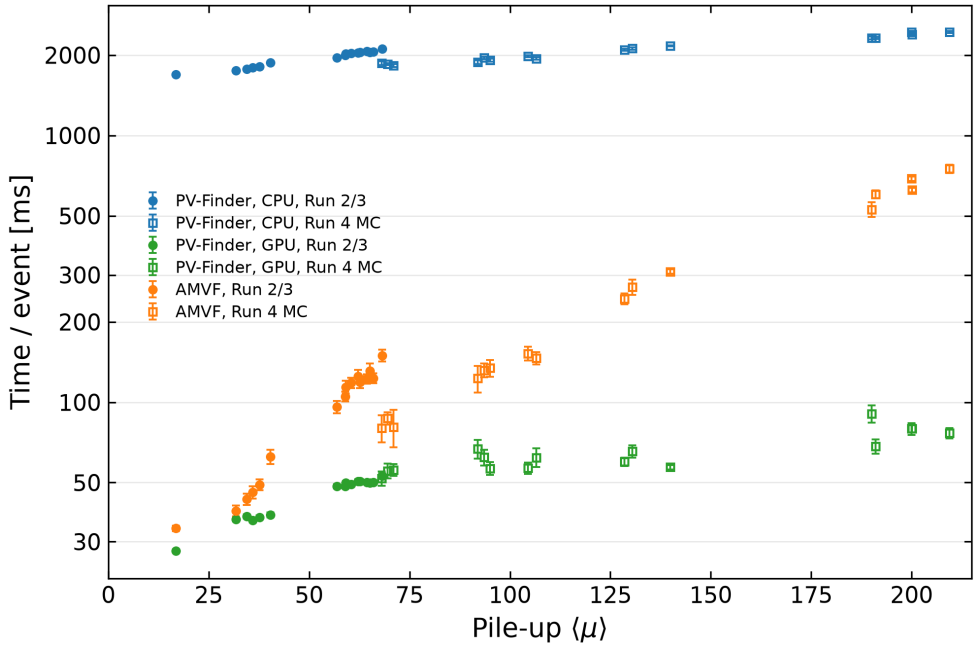


Figure 5.3: Average per-event wall time for AMVF and PV-Finder on both CPU and GPU. 200-500 events from four Run 3 data samples and six Run 4 monte carlo samples are used.

Table 5.1: Per-event timing of InDetNNPriVxFinderTool, averaged over the three Run 4 PU200 samples.

Stage	CPU (ms)	GPU (ms)
Track selection	30.54	29.96
Tensor building	0.10	0.10
Inference	2188.67	37.56
Peak finding	0.05	0.05
Vertex building	0.29	0.22
Track-to-vertex association	181.65	8.30
Graph building	2.83	2.50
GNN inference	177.89	5.20
Edge selection	0.68	0.41
Total	2401.28	76.18

input, and the GNN forward pass takes 98 ms; the measured chain is 277 ms. Scaling the Athena inference time of Table 5.1 by the measured speed-up, and keeping the other Athena steps and the Athena GNN step unchanged, projects the chain to 413 ms per event (range 336 to 456 ms over the speed-up estimates), against about 650 ms for AMVF (600 to 750 ms read from Figure 5.3), i.e. about 1.6 times faster. This is a projection, not an Athena measurement, and it needs a packed-input tensor builder in Athena; with the present padded builder the projection is 566 ms, no clear gain over AMVF.]

6 Validation at low pile-up

We first tested the end-to-end model at Run 2 and Run 3 pile-up, before the PU200 studies. This chapter uses the ~~Run 2 reproduction model. Its tracks-to-KDE and KDE-to-histogram networks are first trained separately, then trained together end to end. This model still uses analytical KDE targets during pretraining.~~ [MM: low-pile-up model e400 (Table 2.2). It is trained with the same two-phase procedure as the HL-LHC model (50 epochs of the MLP alone on the target histogram, then 400 epochs end to end), with no KDE at any step. It is used for every low-pile-up result: Run 2 simulation, Run 2 and Run 3 data, and the GNN chain.] The HL-LHC model uses the histogram warm-up described in Section 2.2.2, which does not need a KDE. Both models use an MLP followed by a U-Net. We changed the network size and U-Net design during the HL-LHC development.

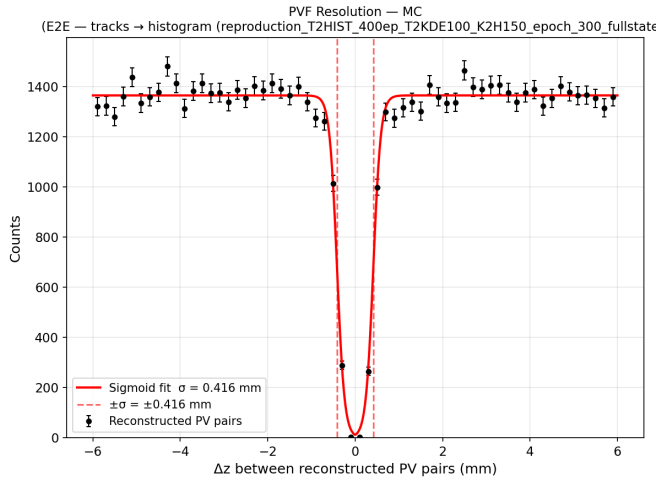
We use 2 550 simulated events from the Run 2 test set and 2 500 events from each collision-data sample. Table 6.1 gives the results for all events in these subsets. We use a peak integral threshold of 0.5 for both the vertex counts and the resolution fit. The peak amplitude threshold is 0.01. We apply no additional minimum-height cut, smoothing, neighbour suppression or GBT filter. [MM: Positions are the local centroid (±3 bins) and the matching window is 0.17 mm, as at HL-LHC. Here it is about 2.4 times the 68% single-vertex residual (72.5 μm for PV-Finder, 66.8 μm for AMVF). The bootstrap uses 2 000 samples.]

The errors on $\sigma_{\text{vtx-vtx}}$ come from the fit covariance. They do not include uncertainty from the fit model or binning, or correlations between vertex pairs in the same event. For reference recovery and peaks per event, we use 10 000 bootstrap samples of whole events. The quoted errors are the standard deviations of these results. The trained model and matching window are fixed. We do not include systematic uncertainties.

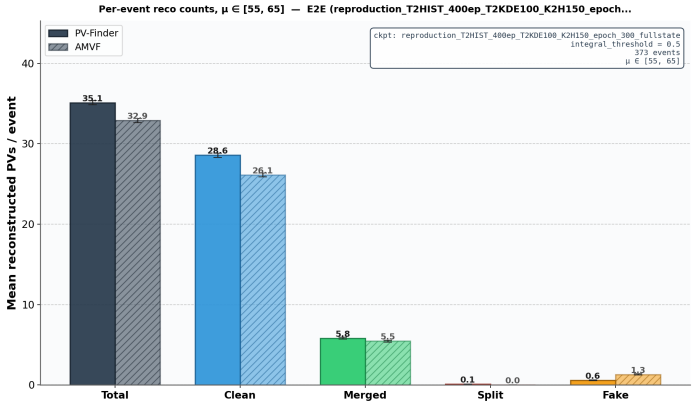
6.1 Simulated data

On Run 2 simulation, PV-Finder has a $\sigma_{\text{vtx-vtx}}$ of ~~0.416 ± 0.011 mm~~ [MM: 0.392 ± 0.009 mm]. Its efficiency is ~~(91.59 ± 0.10)%~~ [MM: (81.55 ± 0.14)%], with ~~0.419 ± 0.014~~ [MM: 2.363 ± 0.040] fake vertices per event. ~~We use the fitted resolution of 0.416 mm as the matching window.~~ [MM: The matching window is 0.17 mm; most of the change from the earlier 91.59% is this narrower window.] The efficiency includes merged truth vertices, as defined in Section 2.4. Most vertices in the category plot are clean. The number of reconstructed vertices increases with pile-up (Figure 6.1). The category plot uses $55 \leq \mu \leq 65$, while the table uses all 2 550 test events.

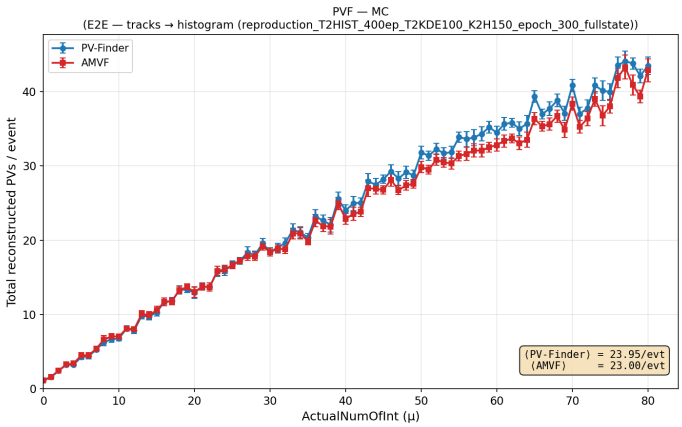
~~In a separate evaluation, AMVF gives a $\sigma_{\text{vtx-vtx}}$ of 0.793 ± 0.017 mm on 2 500 events from the same simulated sample. This evaluation uses vertices with at least two tracks. The PV-Finder resolution is smaller, but the two evaluations do not use exactly the same events. The comparison also depends on the peak cuts and resolution fit.~~ [MM: On the same 2 550 events, with the same code and AMVF vertices with at least two tracks, AMVF gives a $\sigma_{\text{vtx-vtx}}$ of 0.789 ± 0.015 mm, an efficiency of (75.74 ± 0.17)% and 2.487 ± 0.044 fake vertices per event. PV-Finder is 5.81 ± 0.14 points more efficient, with 0.124 ± 0.035 fewer fake vertices per event.]



(a) Resolution.



(b) Categories, $55 \leq \mu \leq 65$.



(c) Reconstructed PVs vs μ.

Figure 6.1: [MM: Made with the earlier Run 2 reproduction model and the 0.416 mm window; to be replaced by the e400 versions.] PV-Finder on the Run 2 simulated test set. (a) Separation between vertex pairs and the fit giving $\sigma_{\text{vtx-vtx}} = (0.416 \pm 0.011)$ mm. The count error bars are $\sqrt{N}$, and the dashed lines mark $\pm\sigma_{\text{vtx-vtx}}$. (b) Reconstructed-vertex categories for $55 \leq \mu \leq 65$. (c) Number of reconstructed vertices versus pile-up for PV-Finder and AMVF. In (b, c), the bars show the standard error on the mean count per event. The $(91.59 \pm 0.10)\%$ efficiency in Table 6.1 uses the full test set.

6.2 Collision data

On collision data, we can measure $\sigma_{\text{vtx-vtx}}$ and yield without truth information. The fitted resolution is ~~0.368 ± 0.009 mm~~ [MM: 0.356 ± 0.010 mm] on Run 2 and ~~0.348 ± 0.009 mm~~ [MM: 0.333 ± 0.008 mm] on Run 3 [MM: (AMVF: 0.808 and 0.813 mm on the same events)]. In both samples, the yield increases approximately linearly with pile-up (Figure 6.2). These resolutions are similar to the simulation result, although the pile-up distributions differ.

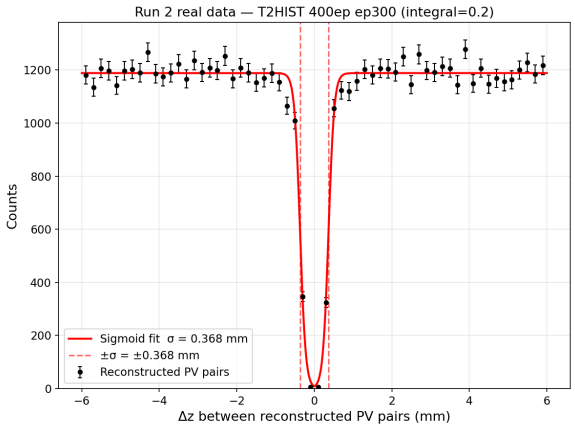
The mean pile-up in the events used for the plots and table is $\langle\mu\rangle = 42.5$ for Run 2 and 50.0 for Run 3. These values differ from the typical pile-up quoted for each run as a whole.

We also compare PV-Finder with AMVF on the same events. The reference uses AMVF vertices with at least two tracks. We subtract the beam-spot z offset (Section 3.0.3). The matching window is ~~the fitted PV-Finder resolution for each sample~~ [MM: 0.17 mm]. Including merged reference vertices, PV-Finder finds ~~(95.31 ± 0.09)%~~ [MM: (94.39 ± 0.10)%] of the AMVF vertices on Run 2 and ~~(94.78 ± 0.09)%~~ [MM: (94.03 ± 0.09)%] on Run 3. These fractions measure agreement with AMVF. Without truth, we cannot tell whether an unmatched AMVF vertex is a missed interaction or an unmatched PV-Finder peak is fake.

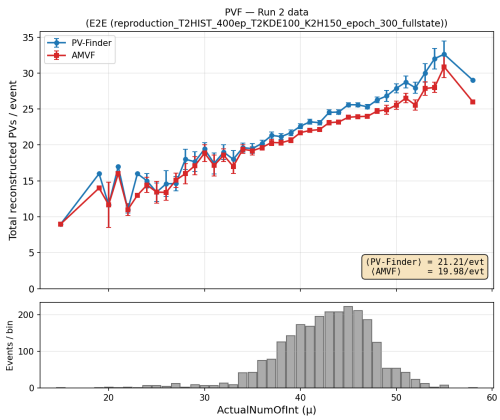
PV-Finder gives ~~1.821 ± 0.029~~ [MM: 3.006 ± 0.040] peaks per event with no AMVF vertex in the window on Run 2, and ~~1.959 ± 0.029~~ [MM: 3.192 ± 0.040] on Run 3. The positional classifier labels these peaks as fake when AMVF is the reference. Split peaks are counted separately. These rates cannot be compared directly with the simulation fake rate, which uses truth vertices. Earlier studies found small peaks near larger ones and tested smoothing and neighbour suppression. We do not use those cuts here. This comparison alone does not explain the extra peaks or show a difference between data and simulation. [MM: **AMVF as reference.** The data numbers measure agreement with AMVF, not efficiency, and they are not comparable with the truth numbers of the simulation. Applying the data convention to Run 2 simulation, with AMVF vertices as the reference instead of truth, gives a recovery of (93.09 ± 0.12)% and 2.998 ± 0.056 peaks per event without a reference vertex, against 94.4 and 94.0%, and 3.0 and 3.2, in data. Against truth the same simulated events give 81.55% and 2.363. The difference between the data and the simulation truth numbers is therefore the choice of reference, not a data/simulation difference. Using AMVF as the reference favours AMVF-like peaks: a true vertex that PV-Finder finds and AMVF misses counts against PV-Finder. Figure 6.3 shows the data panels with the Run 2 simulation overlaid; the pairwise-Δz shapes of data and simulation agree on the plateau ($\chi^2/\text{ndf} = 202/200$ for Run 2 and $197/200$ for Run 3). There is no Run 3 simulation; the Run 3 panels compare with the Run 2 simulation.]

Table 6.1: Low-pile-up validation. The results use all 2 550 events tested with PV-Finder on simulation and all 2 500 events from each collision sample. Reference recovery includes merged vertices. It measures truth efficiency on simulation and agreement with AMVF on collision data. The last column counts peaks with no reference vertex in the matching window. Split peaks are counted separately and excluded from this column. ~~The AMVF resolution comes from a separate 2 500-event subset of the simulated sample.~~ [MM: All rows use the e400 model, the 0.17 mm window and the same events for PV-Finder and AMVF.] Resolution errors come from the fit covariance; errors on recovery and peaks per event come from resampling whole events at a fixed matching window. Peak-finding settings are in Table A.1.

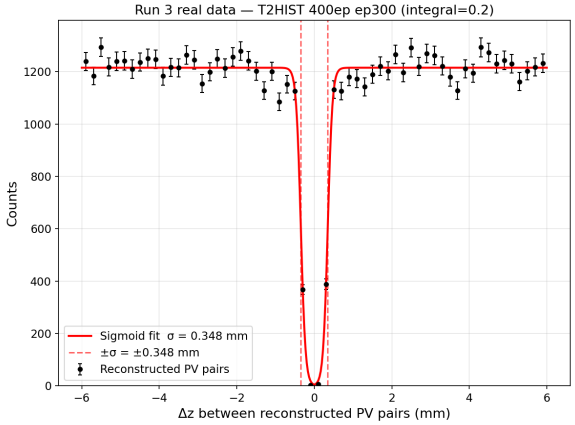
Sample	Reference	$\sigma_{\text{vtx-vtx}}$ [mm]	Reference recovery	No reference/evt
Run 2 simulation	MC truth	~~0.416 ± 0.011~~ [MM: 0.392 ± 0.009]	~~(91.59 ± 0.10)%~~ [MM: (81.55 ± 0.14)%]	~~0.419 ± 0.014~~ [MM: 2.363 ± 0.040]
[MM: Run 2 simulation]	[MM: AMVF]		[MM: (93.09 ± 0.12)%]	[MM: 2.998 ± 0.056]
Run 2 data	AMVF	~~0.368 ± 0.009~~ [MM: 0.356 ± 0.010]	~~(95.31 ± 0.09)%~~ [MM: (94.39 ± 0.10)%]	~~1.821 ± 0.029~~ [MM: 3.006 ± 0.040]
Run 3 data	AMVF	~~0.348 ± 0.009~~ [MM: 0.333 ± 0.008]	~~(94.78 ± 0.09)%~~ [MM: (94.03 ± 0.09)%]	~~1.959 ± 0.029~~ [MM: 3.192 ± 0.040]
AMVF (Run 2 sim.)	[MM: MC truth]	~~0.793 ± 0.017~~ [MM: 0.789 ± 0.015]	[MM: (75.74 ± 0.17)%]	[MM: 2.487 ± 0.044]



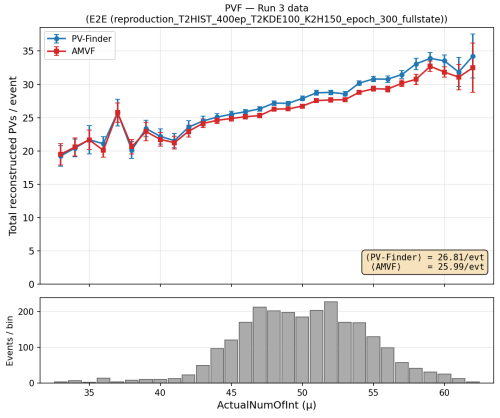
(a) Run 2 resolution.



(b) Run 2 yield.

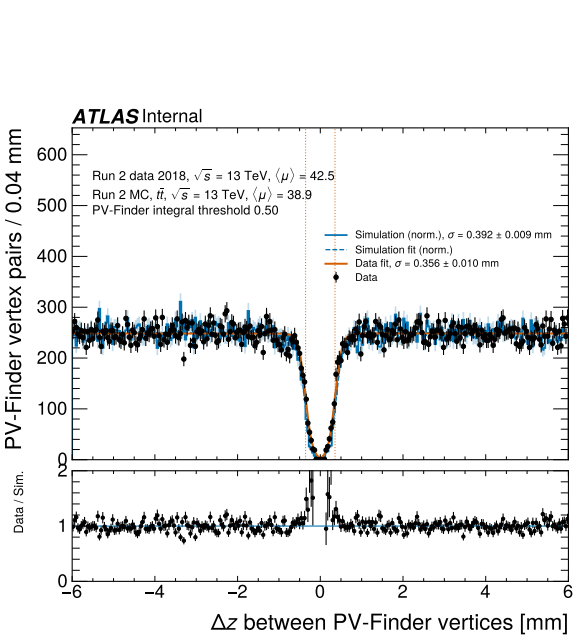


(c) Run 3 resolution.

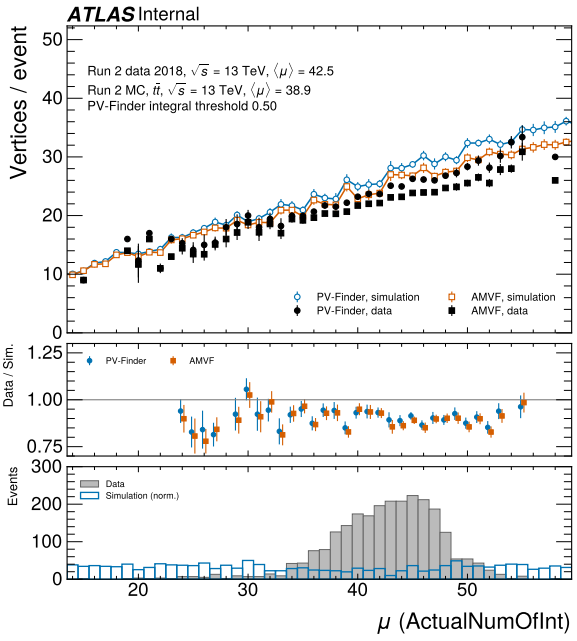


(d) Run 3 yield.

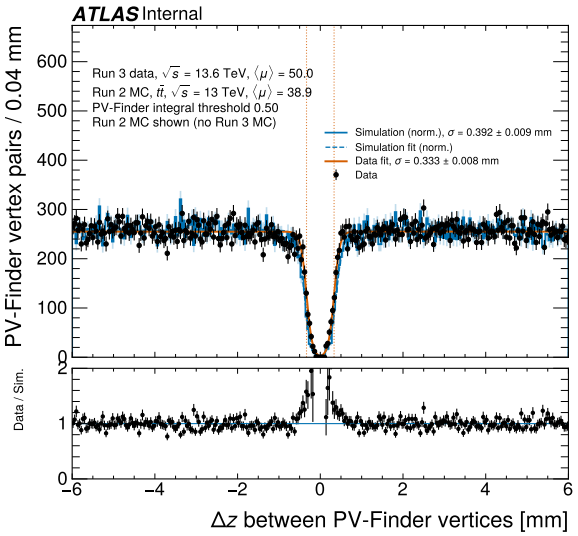
Figure 6.2: Run 2 and Run 3 collision-data validation on the 2 500-event subsets used here. Their mean pile-up is 42.5 and 50.0, respectively. (a, c) Distributions of the separation between PV-Finder vertex pairs and the fits, with $\sqrt{N}$ count error bars. (b, d) PV-Finder and AMVF vertex yields versus pile-up. The error bars show the standard error on the mean yield in each pile-up bin. The pile-up distribution of each sample is shown below its yield plot. AMVF is the reference in this comparison. Agreement with AMVF does not measure truth efficiency. [MM: Proposed replacement with simulation overlaid: Figure 6.3.]



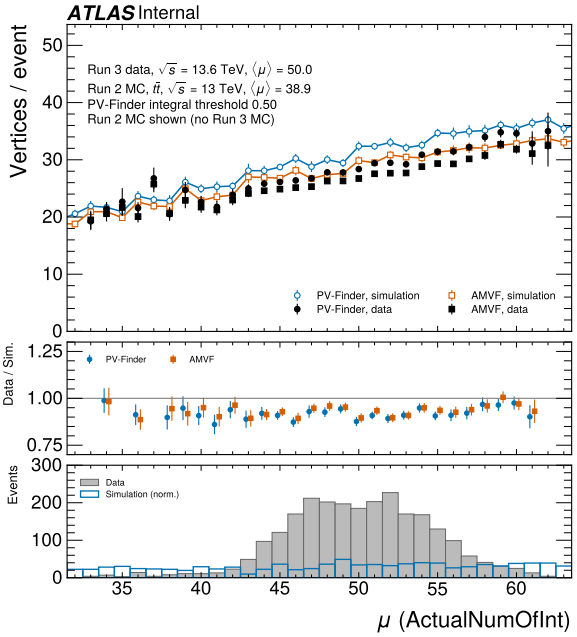
(a) Run 2 resolution.



(b) Run 2 yield.



(c) Run 3 resolution.



(d) Run 3 yield.

Figure 6.3: [MM: Proposed replacement for Figure 6.2, made with the frozen low-pile-up configuration (e400, integral 0.50, 0.17 mm). Each panel overlays the Run 2 simulation ($t\bar{t}$, $\langle\mu\rangle = 38.9$) on the collision data. (a, c) Pairwise Δz of PV-Finder vertices with the sigmoid fits. (b, d) PV-Finder and AMVF vertices per event versus pile-up in data and simulation, the data/simulation ratio, and the pile-up distributions. There is no Run 3 simulation; the Run 3 panels use the Run 2 simulation. AMVF is the reference in data, so agreement with AMVF does not measure efficiency.]

7 Track-to-vertex association with graph networks

7.1 Task

PV-Finder estimates the positions of the primary vertices but does not assign tracks to them. AMVF provides track assignments as part of the vertex fit. For PV-Finder, a separate Track-to-Vertex Association (TTVA) step is therefore needed:

tracks → PV-Finder histogram → peak finding → candidate vertices → TTVA → per-vertex track lists.

The track lists allow the PV-Finder and AMVF chains to be compared using the same track-based definitions, as described in Section 7.4.

We treat TTVA as a binary edge-classification problem. Tracks and candidate vertices are represented as nodes in a graph. The network gives each track-vertex edge a score, and these scores are used to assign each track to at most one vertex. The graph structure allows each score to use information from nearby tracks and vertices.

The network is trained on simulated events, where truth vertices provide the edge labels. During reconstruction, the truth vertices are replaced by PV-Finder peaks. The effect of this difference between training and deployment is studied in Section 7.7. [MM: An updated GNN, trained on PV-Finder candidates instead of truth vertices, is being trained. The GNN results in this chapter use the frozen GNN v2 (epoch 527, $t = 0.94$) and will be updated with the new model.]

7.2 Graphs, features and labels

Each event is represented as a heterogeneous bipartite graph with track nodes and vertex nodes. An edge represents a possible association between a track and a vertex. Figure 7.1 shows how the graph is used to calculate association scores.

Each track node has eight input features:

$$d_0, \quad z_0, \quad \sigma_{d_0}, \quad \sigma_{z_0}, \quad \mathrm{cov}(d_0, z_0), \quad \theta, \quad \phi, \quad p_\mathrm{T}/50\,\mathrm{GeV}.$$

The track momentum is stored in the input ntuples in MeV. Dividing it by 50 GeV avoids passing the much larger raw numerical values to the network and gives a more suitable scale for training. Each vertex node has two features: its z position and its histogram peak height. For the training graphs, the vertex positions come from MC truth. Their heights are calculated using the same Gaussian-CDF procedure used to build the PV-Finder training targets in Section 2.2.3. During reconstruction, the vertex features come from the peaks found in the PV-Finder output histogram.

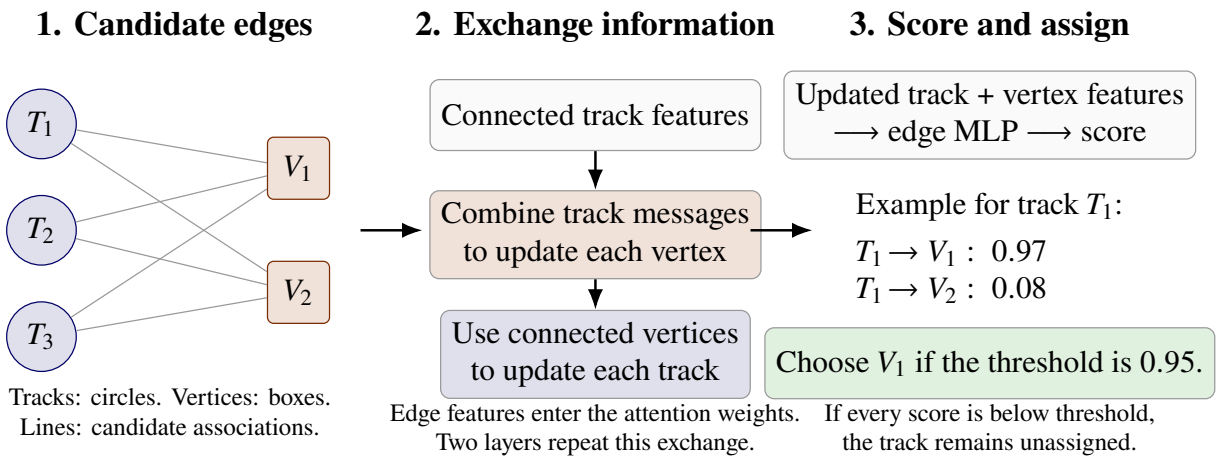


Figure 7.1: Track-to-vertex association with the GNN. A small graph is shown for illustration; at PU200 each track is connected to its 20 nearest vertex candidates. The middle panel shows the two directions of information exchange. Updated node features give one score per candidate edge, and the highest score is used to assign the track if it passes the threshold. Scores in the right panel are illustrative.

Each edge has three features. These are the longitudinal impact-parameter significance,

$$\frac{z_0 - z_{\mathrm{PV}}}{\sqrt{\sigma_{z_0}^2 + \sigma_{\mathrm{PV}}^2}},$$

the transverse impact-parameter significance,

$$\frac{d_0}{\sigma_{d_0}},$$

and the longitudinal distance $|z_0 - z_{\mathrm{PV}}|$. For truth vertices, σ_{PV} is calculated from the number of associated tracks using the resolution model in Section 2.2.3. For PV-Finder vertices, it is taken from the width of the reconstructed peak. The significance includes the track and vertex resolutions, while $|z_0 - z_{\mathrm{PV}}|$ gives the separation in millimetres.

During training, an edge has label 1 if MC truth associates the track with that vertex, and label 0 otherwise. We refer to these as positive and negative edges. These binary edge labels are separate from the Clean, Merged, Split and Fake vertex categories defined in Section 7.4. Those categories are assigned only after the GNN scores have been used to build the track list of each reconstructed vertex.

At low pile-up, each track is connected to every vertex. This is not practical at PU200 because the number of possible edges is much larger. In the current PU200 training sample, an event has on average about 1,366 tracks and 139 vertex nodes. A fully connected graph would therefore have about 190,000 edges. Instead, each track is connected to the 20 closest vertices in $|z_0 - z_{\mathrm{PV}}|$. This gives about 27,300 edges per event. The same choice is used when the model is applied to PV-Finder peaks.

7.3 Architecture and training

The track and vertex inputs first pass through separate linear layers, since the two node types have different input features. Both layers produce a 32-dimensional representation for each node.

Table 7.1: Samples and training settings used for the TTVA models.

	low pile-up	PU200
Training sample	$t\bar{t}$	~~all-hadronic~~ [MM: $t\bar{t}$, flat μ (`r16443`, `r16638`)]
Training events	35,700	~~180,000~~ [MM: 149,820]
Validation events	12,750	~~5,000~~ [MM: 7,500 (1,500 monitor the loss)]
Training files	$2 \times 17,850$	~~$18 \times 10,000$~~ [MM: $12 \times 5,000 + 18 \times 4,990$]
Graph connectivity	fully connected	$k = 20$
Training epochs	120	~~108~~ [MM: 540 (checkpoint 527)]
Full passes over data	60	~~6~~ [MM: 18]
Training graphs	with augmentation	without augmentation

The model then applies two graph-attention layers [9]. In each layer, one operation updates the vertex nodes using information from the connected tracks. A second operation updates the track nodes using information from the connected vertices. Each operation uses four attention heads. The three edge features defined in Section 7.2 enter the attention calculation, so the messages depend on the compatibility between the track and vertex. Residual connections are used between the two layers.

For the final edge prediction, the 32-dimensional representations of the track and vertex are concatenated. They are passed through a multilayer perceptron with dimensions $64 \to 32 \to 32 \to 1$. Its output is one logit per edge. Applying a sigmoid gives the association score used in the track selection. The complete model has 39 521 trainable parameters.

The model is trained with binary cross-entropy. Positive edges are much less common than negative edges, especially at PU200. To account for this imbalance, the positive-class weight is recalculated from the number of positive and negative edges in each batch. Adam is used with an initial learning rate of 10^{-3}. The learning rate follows a cosine schedule down to 10^{-5}, and the gradient norm is clipped at 1.0. Each batch contains 32 event graphs.

The training samples are divided into separate files so that they do not have to be loaded into memory at the same time. During each training epoch, the model processes one file. The files are used in sequence, so a full pass over the data takes two epochs at low pile-up and 18 epochs at PU200. Table 7.1 summarizes the samples and training lengths.

For the low-pile-up model, 70% of the training and validation graphs are modified once when the graph files are built. Some truth-vertex nodes are removed using the measured missed-vertex probability versus track multiplicity. The remaining positions are shifted using the measured reconstructed-truth residuals, and their heights and uncertainties are resampled to resemble reconstructed peaks. Additional vertex nodes are placed near track positions, with heights and uncertainties sampled from unmatched peaks. These added nodes have no positive truth-labelled edges. The track nodes are unchanged.

The distributions used for these changes were measured on the 2 550-event low-pile-up test sample. Its event graphs are excluded from network training, but it was used to choose the augmentation. The resulting evaluation therefore is not independent of all model-development choices. The PU200 model used for the main results is trained on truth graphs without augmentation. The effect of applying similar changes at PU200 is studied in Section 7.7.

7.4 Track selection and evaluation

The GNN gives a score to every track-vertex edge. For each track, we select the vertex with the highest score. The association is kept only if this score is above a threshold t; otherwise, the track remains unassigned. Each track can therefore be assigned to at most one vertex. The value of t is chosen from a threshold scan and can be changed without retraining the model.

After the track selection, each reconstructed vertex is classified from the truth origin of its assigned tracks. The categories are based on the ATLAS primary-vertex classification [10]. In the ATLAS definition, each track contribution is weighted by the weight assigned during the vertex fit. ~~For AMVF, these weights are produced as part of the fit, but they are not stored in the ntuples used here.~~ [MM: For the headline comparison we use, for AMVF, the TTVA track-vertex links (`Nonprompt_Hard_MaxWeight`), which are what analyses use.] We therefore give every track the same weight when evaluating both AMVF and the GNN. [MM: A supplementary ntuple of the same events stores the AMVF fit tracks and fit weights. The two lists differ: at PU200, 23.8% of the tracks are in no AMVF fit, while the TTVA links attach 99.5% of them. Classified with its own fit tracks and fit weights, AMVF has 46.3 Clean vertices per event instead of 34.4, i.e. a clean-vertex yield of about 41.6% instead of 30.9%. This is still about 9 points below PV-Finder+GNN (50.6%; 50.7% if the tracks are weighted by their GNN scores). The headline keeps the TTVA links.] The ntuples retain truth-vertex membership through `TruthVertex_assocTracks`, which lists the reconstructed-track indices associated with each stored truth vertex. Inverting these lists gives the truth-vertex label for each track. This uses all stored truth vertices, including those with only one associated track. The two-track requirement applies separately to the vertex-efficiency denominator.

The ntuples do not retain the track-to-truth-particle link or matching probability used to identify fake tracks at particle level. We can therefore use the stored vertex association, but cannot reproduce that particle-level classification. This describes the information saved in the ntuples; it does not establish whether the upstream simulation contained full truth. Tracks without a stored truth-vertex association form a separate group. In the 1 920-event PU200 evaluation sample, this group contains 27 021 of 2 521 131 input tracks, or 1.07%. The absence of this association alone does not identify why a track is unlabelled.

We use the name *Clean* for the category called *Matched* in the ATLAS definition. For each reconstructed vertex, its assigned tracks are grouped by their truth vertex, with one additional group for tracks without an association. The group containing the largest number of tracks is called the dominant contribution. The vertex is initially classified as follows:

Clean. The dominant contribution comes from a truth vertex and contains at least 70% of all tracks assigned to the reconstructed vertex.

Merged. The dominant contribution comes from a truth vertex but contains less than 70% of the assigned tracks.

Fake. The group with no truth-vertex association is the dominant contribution, or the reconstructed vertex has no assigned tracks.

The Split category is assigned in a second step. If the same truth vertex is the dominant contribution to more than one reconstructed vertex, the vertex with the largest $\sum p_{\mathrm{T}}^2$ keeps its original Clean or Merged classification. The other vertices are classified as Split.

Under this definition, a Fake vertex can still contain some truth-associated tracks. This happens when those tracks come from several truth vertices and the group with no truth-vertex association is larger than each individual truth contribution.

The main performance quantity is the clean-vertex yield,

$$\frac{N_{\text{clean}}}{N_{\text{truth}}},$$

where N_{truth} is the number of truth vertices with $n_{\text{trk}} \geq 2$. The fake rate is

$$\frac{N_{\text{fake}}}{N_{\text{reco}}}.$$

In the main results, vertices with ~~no assigned tracks~~ [MM: fewer than two assigned tracks] are removed before these rates are calculated[MM: , for PV-Finder+GNN and for AMVF alike; their tracks count as unassigned]. ~~Keeping them does not change the clean-vertex yield, but it increases the fake rate.~~ Results that include all reconstructed peaks are labelled separately.

The PV-Finder-only results in Section 2.4 use a positional match between reconstructed and truth vertices. The classification in this section instead uses the truth composition of the assigned tracks. These are different measurements. For the comparison in this chapter, AMVF track lists are passed through the same classification code as the GNN assignments.

7.5 HL-LHC full-chain performance

The PU200 results use the same 1 920 validation events as the finder study in Section 4. The sample has a mean pile-up of 192.5 and contains 213 738 truth vertices with $n_{\text{trk}} \geq 2$. The finder reconstructs ~~193 945 peaks, or~~ about 101 [MM: peaks] per event. The GNN uses $k = 20$ graphs and the track-selection threshold is ~~$t = 0.98$~~ [MM: $t = 0.94$].

The finder efficiency of ~~83.16%~~ [MM: 83.06%] reported in Section 4.1 only asks whether each truth vertex is represented by a nearby reconstructed peak. It also gives credit for merged vertices. For example, suppose two nearby truth vertices produce only one reconstructed peak. The finder metric counts both truth vertices as recovered: one is matched directly, while the other is counted as merged. One peak can therefore account for two truth vertices.

The full-chain classification cannot count the same peak twice. A reconstructed vertex has one list of assigned tracks and can be classified as Clean for only one truth vertex. We therefore repeat the finder measurement with a one-to-one match, in which each peak can be matched to at most one truth vertex within ~~0.5 mm~~ [MM: 0.17 mm]. This gives an efficiency of ~~78.1%~~ [MM: 73.36% (70.10% for AMVF)], which is the relevant starting point for the track-assignment study.

To separate missing peaks from errors in track assignment, we also use a truth-guided assignment, which we call the *oracle*. For each track with a known truth vertex, the track is assigned to the reconstructed peak closest to that truth vertex, provided that the peak is within ~~0.5 mm~~ [MM: 0.17 mm]. Tracks without a truth association, or without a nearby peak, remain unassigned. The resulting vertices are then evaluated with the same Clean, Merged, Split and Fake definitions used for the GNN. This assignment cannot be used on data because it requires MC truth. It is a reference rather than a strict mathematical upper bound.

The truth-guided assignment gives a clean-vertex yield of ~~73.7%~~ [MM: 68.55%]. The GNN reaches ~~68.4%~~ ~~at $t = 0.98$~~ [MM: 50.57% at $t = 0.94$]. Figure 7.2 compares these values with the positional finder match. The successive differences are ~~21.9~~ [MM: 26.6] percentage points from all truth vertices to a separate matched peak, ~~4.4~~ [MM: 4.8] points from the positional match to the truth-guided track assignment, and ~~5.2~~ [MM: 18.0] points from the truth-guided assignment to the GNN result. [MM: Figure 7.3 adds simple baselines on the same PV-Finder peaks. Assigning each track to the nearest peak in z_0 gives 31.3%; requiring in addition $|z_0 - z_{\mathrm{PV}}| < 3\sigma(z_0)$ gives 32.3%, and a cut on the longitudinal significance of Section 7.2, with the peak width as σ_{PV}, gives 32.3% (32.7% at its best cut, $s_{\max} = 2$). AMVF with its TTVA links gives 30.9%, the same as the nearest-z_0 baseline, in every μ bin within 0.4 points. The GNN without a score cut gives 39.4%. The GNN gain is therefore the learned assignment (+8.5 points over AMVF) plus the score cut, which leaves ambiguous tracks unassigned (+11 points), not the use of the significance.] These values are useful diagnostics, but they should not be interpreted as strict bounds on all possible association methods.

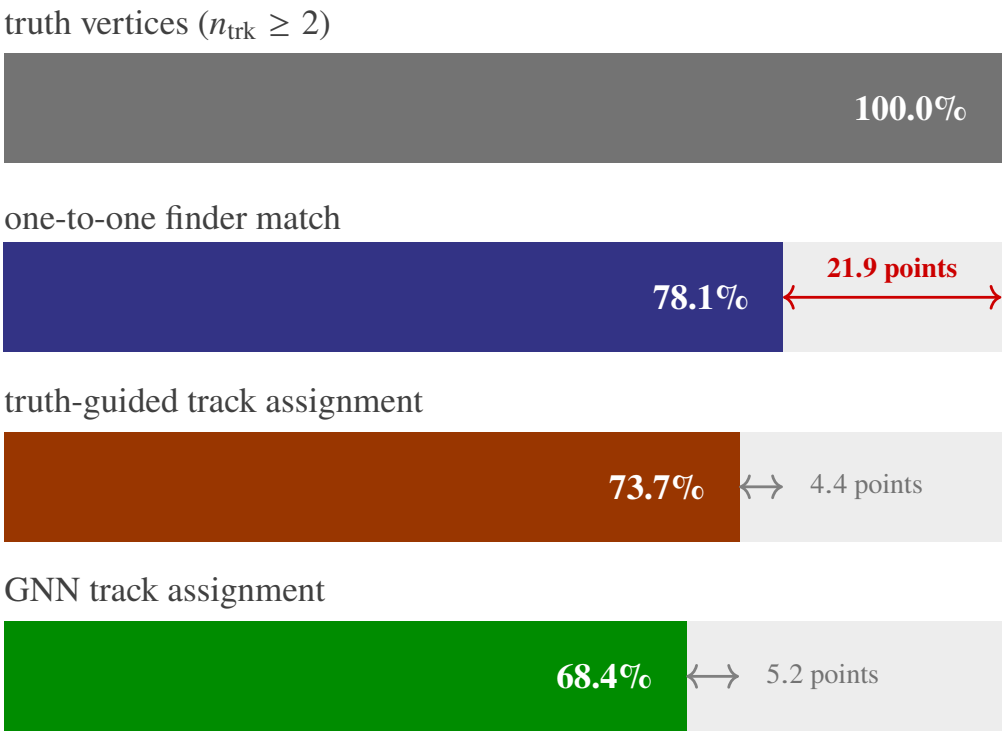


Figure 7.2: [MM: Values from the earlier GNN v4 at $t = 0.98$ and a 0.5 mm window; replaced by Figure 7.3.] Clean-vertex yield at successive stages of the PU200 study. The finder value uses only vertex positions. The truth-guided and GNN values use the assigned tracks and the same vertex-classification rules. The truth-guided result uses MC information and is included as a diagnostic.

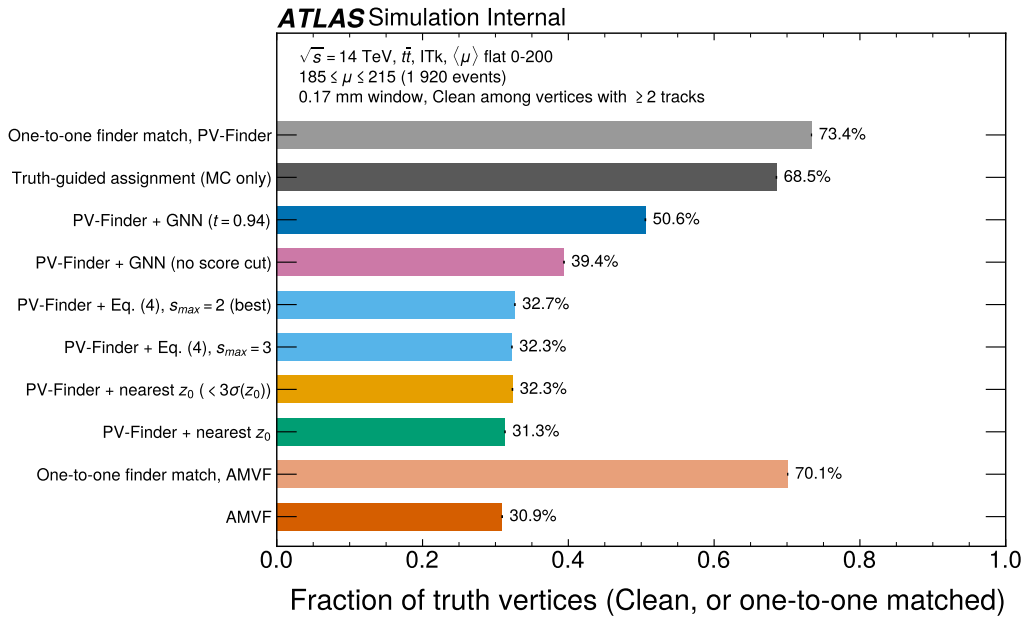


Figure 7.3: [MM: Proposed replacement for Figure 7.2. Fraction of truth vertices with $n_{\text{trk}} \geq 2$ that are Clean (or, for the finder rows, matched one to one within 0.17 mm) on the 1 920 PU200 events, with vertices of at least two tracks. From the top: one-to-one finder match for PV-Finder; truth-guided assignment (MC only); PV-Finder+GNN at $t = 0.94$ and without a score cut; the significance assignment ("Eq. (4)", the paper's numbering; Section 7.2) at $s_{\text{max}} = 2$ and 3; nearest z_0 with and without the $3\sigma(z_0)$ requirement; the one-to-one finder match for AMVF; and AMVF with its TTVA links. All baselines use the same PV-Finder peaks.]

The truth vertices without a separate matched peak have low track multiplicity. Their median is $n_{\mathrm{trk}} = 3$, compared with 9 for matched truth vertices, and 34.8% of them have exactly two tracks. [MM: (Earlier model, 0.5 mm; to be recomputed.)]

Table 7.2: Track-based comparison on the PU200 sample. Vertices ~~without assigned tracks~~ [MM: with fewer than two assigned tracks] are removed before calculating the rates.

Method	Vertices with tracks	Clean	Clean/reco	Clean/truth
AMVF	~~187 960~~ [MM: 187 908]	~~66 127~~ [MM: 66 096]	35.2%	30.9%
PV-Finder+GNN, ~~$t = 0.98$~~ [MM: $t = 0.94$]	~~175 491~~ [MM: 152 795]	~~146 269~~ [MM: 108 086]	~~83.3%~~ [MM: 70.7%]	~~68.4%~~ [MM: 50.6%]

AMVF produces ~~slightly~~ [MM: 23%] more vertices with tracks, but only 35.2% pass the Clean requirement. With the unweighted track counts available in these ntuples, the median fraction of tracks from the dominant truth vertex is 63.2% for AMVF, below the 70% Clean threshold. For the GNN it is 100% [MM: (GNN v4; to be recomputed)]. This explains why the difference in clean-vertex yield is much larger than the difference in the number of reconstructed vertices. [MM: The other categories, as fractions of the reconstructed vertices, are Merged 24.6% against 42.6% and Split 4.6% against 22.1%. At track level, PV-Finder+GNN has an efficiency of 62.9% and a purity of 79.5%, against 68.3% and 68.7% for AMVF (F_1 70.2% against 68.5%).]

~~After empty peaks are removed, the GNN result contains 303 Fake vertices out of 175 491, giving a fake rate of 0.17%. AMVF gives 0.07%. If all finder peaks were counted, including the 18 454 peaks to which the GNN assigns no tracks, the GNN fake rate would be 9.67%.~~ [MM: After candidates with fewer than two tracks are removed (21.3 per event, 17.8 of them with exactly one track), the GNN result contains 69 Fake vertices out of 152 795, a fake rate of 0.05%. AMVF gives 0.07% (135 of 187 908).]

~~For hard-scatter identification, the GNN gives its best result at $t = 0.95$. The vertex with the largest $\sum p_{\mathrm{T}}^2$ is associated with the hard-scatter interaction in 94.4% of events, compared with 80.7% for AMVF.~~ [MM: **Hard-scatter vertex.** We follow the ITk definitions [7]. The HS is reconstructed if a vertex lies within 0.1 mm in z of the true HS: 99.90% of the 1 920 events for PV-Finder and 99.79% for AMVF (Section 4.1). It is reconstructed and selected if, in addition, the HS-matched vertex has the largest $\sum p_{\mathrm{T}}^2$. For AMVF we use its stored primary vertex. In all 25 000 events the stored primary is the largest-$\sum p_{\mathrm{T}}^2$ vertex computed from AMVF's fit tracks, so it is the Athena rule applied to AMVF's own inputs. For PV-Finder+GNN the same rule is applied to the GNN-assigned tracks that are compatible ($\chi^2 < 18.42$) with the fitted vertex described below. At PU200 the HS is reconstructed and selected in (95.05 ± 0.53)% of events for PV-Finder+GNN and (92.92 ± 0.56)% for AMVF, a difference of $+2.1 \pm 0.5$ points. The AMVF value agrees with the ITk paper (about 92–93%). Over the full flat-μ sample the values are 97.1% and 95.1%. Applying the rule to the AMVF TTVA lists instead would give 89.1%; those lists are not what Athena ranks, so this is not used. The GNN puts (87.9 ± 0.2)% of the HS tracks on the HS-matched vertex, against (74.1 ± 0.3)% for AMVF. It also collects more pile-up tracks: the HS-vertex purity is 82.3% against 88.4%, and the HS vertex is Clean in (88.3 ± 0.8)% of events against (92.5 ± 0.6)% for AMVF (-4.2 ± 0.6 points). These two quantities trade against t; at $t = 0.97$ the HS vertex is Clean in 94.6% of events.]

[MM: **Hard-scatter longitudinal resolution.** The PV-Finder peak gives the position of the candidate, not a vertex fit. To measure the HS position, we fit the tracks that the GNN assigns to the HS candidate at t with a three-dimensional adaptive vertex fit with a beam-spot constraint, seeded at the peak position. The resolution is defined as in the ITk study: in events where the HS is reconstructed within 0.1 mm, the residual $z_{\mathrm{reco}} - z_{\mathrm{HS}}$ of the closest vertex, and its width from the iterative RMS within ±3 RMS. On the 1 915 PU200 events where both algorithms reconstruct the HS, it is (9.52 ± 0.26) μm for PV-Finder+GNN

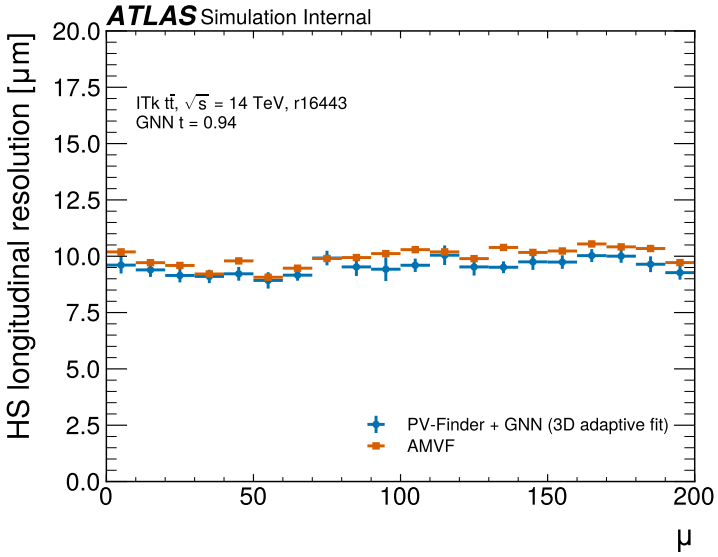


Figure 7.4: [MM: Hard-scatter longitudinal resolution versus μ on the 25 000 flat-μ evaluation events. PV-Finder+GNN: three-dimensional adaptive fit with a beam-spot constraint on the GNN-assigned tracks at $t = 0.94$; AMVF: its vertex. Iterative 3-RMS width of $z_{\text{reco}} - z_{\text{HS}}$ for events with the HS reconstructed within 0.1 mm.]

and (10.24 ± 0.27) μm for AMVF (the ITk paper gives about 10 μm). It is flat in μ (Figure 7.4) and in the local pile-up density.]

7.6 Performance at low pile-up and comparison with Run 3 data

The model is evaluated on the 2 550-event simulated test sample of Chapter 6, with mean pile-up 38.9. It contains 72 189 truth vertices with $n_{\text{trk}} \geq 2$. The distributions of the GNN scores for positive and negative edges are shown in Figure 7.5(a).

~~The threshold scan in Figure 7.5(b) gives a working point of $t = 0.99$ for the vertex classification. At this threshold, 55,897 of the 72,189 truth vertices are reconstructed as Clean. This gives a clean-vertex yield of 77.4%, compared with 60.9% for AMVF.~~ [MM: The GNN graphs are built from the vertex-finding peaks of the same events (e400, integral 0.50). We use the HL-LHC working point, $t = 0.94$, for every low-pile-up result. In the threshold scan it gives the highest HS identification, is within 0.01 points of the best track F_1, and is 1.4 points below the best clean-vertex yield (71.9% at $t = 0.985$). At this threshold, 50 946 of the 72 189 truth vertices are reconstructed as Clean, a clean-vertex yield of (70.57 ± 0.25)%, compared with (60.47 ± 0.28)% for AMVF. PV-Finder+GNN is above AMVF in every μ bin: 87.4, 79.1, 70.3 and 63.8% against 84.2, 71.6, 59.4 and 52.0% for μ in 0–20, 20–40, 40–60 and 60–80.]

~~The GNN assigns tracks to 61,845 PV-Finder peaks. Of these, 13 are classified as Fake, giving a fake rate of 0.02%. A further 4,627 peaks receive no tracks and are removed following the convention in Section 7.4. If these empty peaks are included and classified as Fake, the fake rate is 7.0%. The high score threshold therefore gives a very pure set of vertices with assigned tracks, but it also leaves some PV-Finder peaks without tracks.~~ [MM: Of the 61 102 PV-Finder peaks, 59 874 receive at least two tracks. One of them is classified as Fake (0.002%). The other 1 228 peaks (883 with one track, 345 with none) are removed following the convention in Section 7.4.]

The PV-Finder+GNN chain has a higher clean-vertex efficiency than AMVF in every truth-vertex multiplicity bin shown in Figure 7.5(c). The largest difference is seen for vertices with few tracks. The Merged fraction is ~~7.4%~~ [MM: 13.1%] for PV-Finder+GNN and ~~20.2%~~ [MM: 20.4%] for AMVF.

Table 7.3 also shows the result obtained when the GNN is given truth-vertex positions instead of PV-Finder peaks [MM: (earlier configuration: integral 0.2, $t = 0.98$; not recomputed)]. This provides an ideal-input reference for the same GNN. At $t = 0.98$, it reaches a clean-vertex yield of 89.2%. It should not be interpreted as a general limit for all track-to-vertex association methods.

Using the reconstructed PV-Finder peaks, an oracle assignment based on MC truth reaches 81.9% [MM: (earlier configuration; not recomputed)]. The deployed GNN result of ~~77.4% is 4.5 percentage points below this value~~ [MM: 70.57% is not directly comparable with it]. The difference measures the remaining association loss for the fixed set of reconstructed peaks.

The fraction of reconstructed vertices classified as Clean is slightly higher for the deployed chain than for the truth-vertex graph, ~~90.4%~~ [MM: 90.4% (earlier configuration; now 85.1% at $t = 0.94$)] compared with 88.8%. The truth graph contains all truth vertices, including low-multiplicity vertices that are difficult to associate. The deployed chain contains fewer of these vertices. Its Clean fraction can therefore be higher even though its clean-vertex yield is lower.

The AMVF list contains 398 vertices with no assigned tracks [MM: and 593 with one track]. They are removed using the same rule as for the GNN, leaving ~~56 931~~ [MM: 56 338] vertices. This changes the category fractions, particularly Fake, but leaves the clean-vertex yield per truth vertex unchanged.

Table 7.3: Track-based vertex classification at low pile-up. Vertices with ~~no assigned tracks~~ [MM: fewer than two assigned tracks] are removed. The sample contains 2 550 events and 72 189 truth vertices with $n_{\mathrm{trk}} \geq 2$. Percentages in the first two rows have statistical uncertainties from ~~10 000~~ [MM: 1 000] event-bootstrap samples. Uncertainties for the truth-vertex input have not been evaluated. Raw vertex counts are observed totals. The score thresholds are ~~0.99~~ [MM: 0.94] for the PV-Finder chain and 0.98 for the truth-vertex input. [MM: PV-Finder+GNN and AMVF rows: e400, integral 0.50, vertices with at least two tracks, 1 000 bootstrap samples; the Truth+GNN row is from the earlier configuration.]

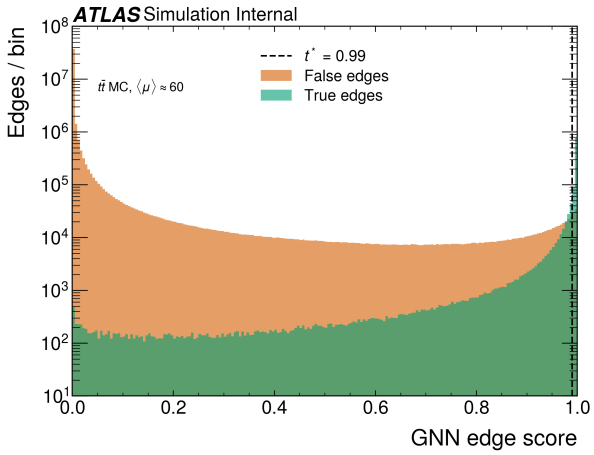
Method	Vertices	Clean [%]	Merged [%]	Split [%]	Fake [%]	Clean/truth [%]
PV-Finder+GNN	~~61 845~~ [MM: 59 874]	~~90.38 ± 0.14~~ [MM: 85.09 ± 0.19]	~~7.42 ± 0.13~~ [MM: 13.12 ± 0.18]	~~2.18 ± 0.06~~ [MM: 1.79 ± 0.06]	~~0.021 ± 0.006~~ [MM: 0.002]	~~77.43 ± 0.24~~ [MM: 70.57 ± 0.25]
AMVF	~~56 931~~ [MM: 56 338]	~~77.23 ± 0.22~~ [MM: 77.48 ± 0.21]	~~20.19 ± 0.21~~ [MM: 20.40 ± 0.20]	~~2.53 ± 0.06~~ [MM: 2.11 ± 0.06]	~~0.054 ± 0.011~~ [MM: 0.004]	~~60.91 ± 0.29~~ [MM: 60.47 ± 0.28]
Truth+GNN	72 544	88.8	7.4	3.8	0.004	89.2

At track level, ~~the highest F_1 score is 0.874 at $t = 0.95$, compared with 0.849 for AMVF~~ [MM: the F_1 score at $t = 0.94$ is $(87.02 \pm 0.11)\%$, compared with $(84.86 \pm 0.12)\%$ for AMVF (efficiency 85.6 against 83.9%, purity 88.5 against 85.9%)]. The F_1 score combines track-assignment efficiency $\varepsilon_{\mathrm{trk}}$ and purity P_{trk} as

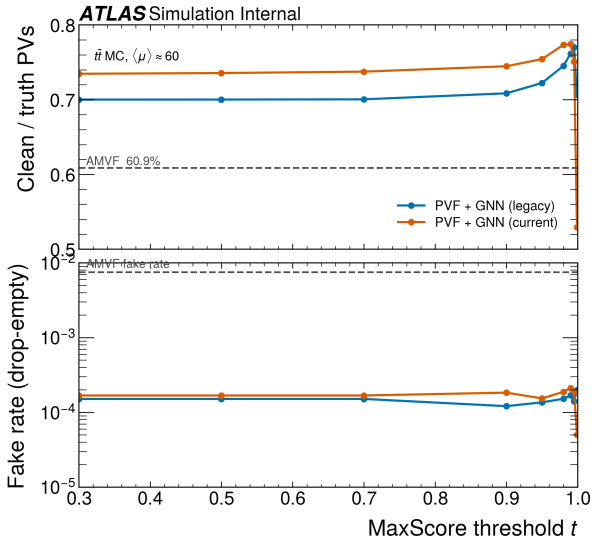
$$F_1 = \frac{2\varepsilon_{\mathrm{trk}} P_{\mathrm{trk}}}{\varepsilon_{\mathrm{trk}} + P_{\mathrm{trk}}}.$$

It is high only when both efficiency and purity are high.

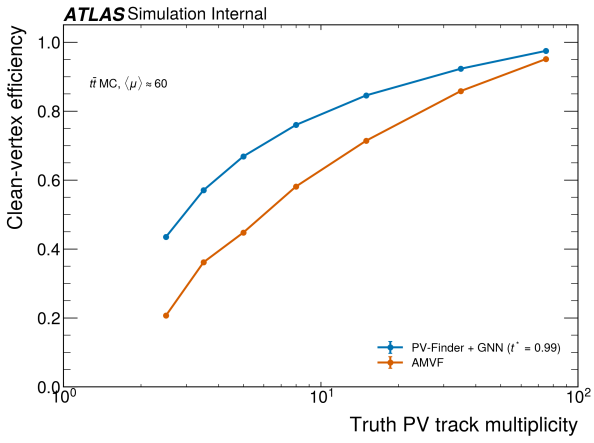
For hard-scatter identification, the vertex with the largest $\sum p_{\mathrm{T}}^2$ is compared with the true hard-scatter vertex. The recorded study on the same 2 550 simulated Run 2 $t\bar{t}$ test events gives ~~$(98.98 \pm 0.20)\%$~~ [MM: $(99.10 \pm 0.18)\%$] for the GNN and ~~$(98.75 \pm 0.22)\%$~~ [MM: $(98.75 \pm 0.21)\%$] for AMVF. ~~These are approximate binomial statistical uncertainties, $\sqrt{p(1-p)/N}$, using the reported success fraction p and $N = 2\,550$.~~ [MM: The selected vertex is counted as correct when the plurality of its truth-labelled tracks comes from the HS, with the same rule for both; uncertainties from 1 000 event-bootstrap samples. AMVF's stored primary vertex gives 98.31%.]



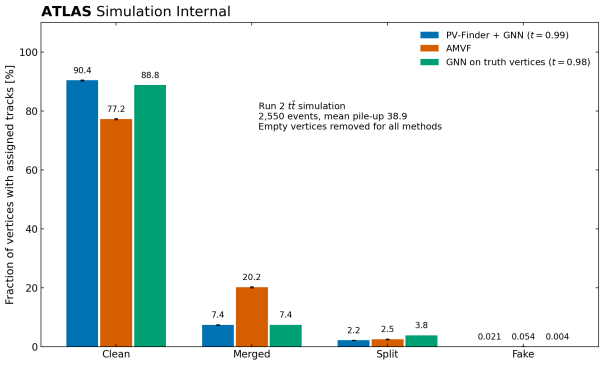
(a) Edge scores by truth label.



(b) Working-point scan.



(c) Clean-vertex efficiency vs n_{trk}.



(d) Category rates.

Figure 7.5: [MM: Made with the earlier configuration (integral 0.2, $t = 0.99$); to be replaced by the e400, $t = 0.94$ versions.] Performance of the TTVA model at low pile-up. The results are shown for the full PV-Finder+GNN chain, AMVF, and the GNN evaluated with truth vertices as input. Panel (d) uses the same removal of empty vertices for all methods. Statistical error bars are shown for the first two methods; they have not been evaluated for the truth-vertex input.

7.6.1 Comparison with Run 3 collision data

The same model is also applied to ~~2,000~~ [MM: the same 2 500] Run 3 collision events [MM: and the same PV-Finder peaks as in Chapter 6 (e400, integral 0.50)]. These data do not provide truth labels, so the track assignments are compared with AMVF. This comparison measures agreement between the two methods and does not determine which assignment is correct.

~~At $t = 0.98$, the GNN assigns 83.4% of the selected tracks, while AMVF assigns 94.8%. Among tracks assigned by both methods, 95.7% are associated with vertices that are within 1 mm of each other. This corresponds to 83.7% of all tracks assigned by AMVF. The vertex with the largest $\sum p_{\mathrm{T}}^2$ agrees between the two methods in 88.7% of events.~~ [MM: At $t = 0.94$, with vertices of at least two tracks for both, the GNN assigns 95.8% of the selected tracks, while AMVF assigns 94.8%. Among tracks assigned by both methods, 94.5% are associated with vertices within 0.17 mm of each other (95.5% within 1 mm). This corresponds to 92.9% (93.9%) of all tracks assigned by AMVF. The vertex with the largest $\sum p_{\mathrm{T}}^2$ agrees within 0.17 mm in 91.3% of events. The GNN gives 28.0 vertices per event and AMVF 26.9.]

7.7 Further checks and outlook

[MM: This section was made with the earlier GNN v4 at $t = 0.98$, a 0.5 mm window and vertices with at least one track; it is kept as a record of that study. The updated GNN (Section 7.1) is trained directly on PV-Finder candidates.] The GNN is trained using truth vertices, but it is applied to reconstructed PV-Finder peaks. The vertex nodes are therefore different during training and inference. The training graphs use the exact truth positions and widths from the resolution model. During inference, the positions, widths and heights are obtained from the reconstructed histogram. In the one-to-one comparison at 0.5 mm, 21.9% of truth vertices cannot be matched to a separate peak, while about 14.0% of the reconstructed peaks remain unmatched. These cases are not present in the standard truth training graphs.

The same GNN reaches a clean-vertex yield of 76.9% when it is evaluated using truth vertices. This is higher than the 68.4% obtained with reconstructed peaks. The difference also includes missing and merged peaks, so it is not a direct measurement of the change in the input distributions. It nevertheless shows that reconstructed peaks make the association problem more difficult.

We tested whether the training graphs could be made more similar to the reconstructed ones. For 70% of the training events, the following changes were applied when the graph files were produced:

- Some truth vertices were removed according to the measured miss probability as a function of n_{trk}.
- The remaining vertex positions were shifted using the measured peak-position residuals.
- The vertex widths and heights were changed to follow the distributions measured from reconstructed peaks.
- Additional vertex nodes with no associated truth tracks were included to represent unmatched peaks.

Two models were then trained on the same 180 000 events, with the same architecture, validation sample and training schedule. One used the modified graphs and the other used the original truth graphs. Both were trained for six complete passes over the data. Their results at $t = 0.98$ are shown in Table 7.4.

Both models give their largest clean-vertex yield at $t = 0.98$. The augmented model produces slightly purer reconstructed vertices and a lower fake rate, but it leaves many more peaks without tracks. Its clean-vertex

Table 7.4: Effect of modifying the PU200 training graphs. The fake rate is calculated after removing peaks without assigned tracks.

Training graphs	Clean/truth	Clean/reco	Fake/reco	Empty peaks
without augmentation	68.4%	83.3%	0.17%	18 454
with augmentation	65.0%	85.9%	0.12%	32 116

yield is therefore 3.4 percentage points lower. The model trained without augmentation also has the lower validation loss, 0.3385 compared with 0.3510, and is used for the main results.

This comparison used one training run for each setup and only six passes over the data. It shows that the tested augmentation does not improve the result under this training setup. It does not establish that augmentation could never help with a longer training or a different construction.

On 300 events from the earlier PU200 sample, the GNN forward pass took a median of 3.8 ms per event, processing one graph at a time. These graphs contained about 917 tracks and 18 300 edges per event. Adding separately measured times for the finder, peak finding, graph construction, GNN and track selection gives about 11 ms per event. This is an estimate from component timings, which exclude some data transfers and input loading. The complete chain has not yet been timed on the current, larger graphs. [MM: CPU timings of the frozen chain are in Chapter 5.]

The PU200 results suggest two main directions for further work. On the finder side, most unmatched truth vertices have few tracks, and nearby truth vertices are sometimes represented by one peak. On the association side, the GNN could be trained directly on reconstructed peaks instead of approximating them through augmentation. The larger available training sample could also be tested with a fixed computing budget. HGTD timing could be added as another edge feature, but this has not yet been studied.

8 Conclusions and outlook

PV-Finder reconstructs primary vertices by mapping reconstructed tracks directly to a one-dimensional histogram of vertex density and finding its peaks. The two-network pipeline has been merged into a single end-to-end model, and the intermediate KDE has been removed from training by warming up the MLP on the histogram target before enabling the U-Net. At low pile-up the model matches AMVF and exceeds it in vertex-vertex resolution (~~0.42 mm, compared with 0.79 mm in a separate AMVF evaluation on the same Run 2 simulated sample~~ [MM: 0.39 mm, compared with 0.79 mm for AMVF on the same Run 2 simulated events, with an efficiency of 81.6% against 75.7% at 0.17 mm]), and is tested on Run 2 and Run 3 collision data.

The production model was trained on the full ITk track sample, using target widths fitted to that sample. On 1 920 held-out PU200 events, its efficiency is ~~(83.16 ± 0.08)%~~ [MM: (83.06 ± 0.08)%], compared with ~~(79.80 ± 0.09)%~~ [MM: (79.85 ± 0.09)%] for AMVF. Both methods use a fixed ~~0.1694 mm~~ [MM: 0.17 mm] matching window. Merged truth vertices count as found. The difference is ~~3.36 ± 0.06~~ [MM: 3.20±0.06] percentage points, and PV-Finder gives ~~0.60 ± 0.09 fewer fake and split vertices per event~~ [MM: 1.01 ± 0.09 fewer fake vertices per event]. Its vertex-vertex resolution is also smaller: ~~0.229 ± 0.003 mm~~ [MM: 0.217 ± 0.003 mm] compared with 0.305 mm. [MM: These are the frozen s8 numbers, to be confirmed with the final recompute.] The efficiency remains higher across the tested matching windows. The fake-plus-split difference changes sign near 0.60 mm (Section 4.1.2). The earlier timing study found little dependence on pile-up up to $\mu = 80$. ~~The complete production chain still needs to be timed at PU200 (Section 5).~~ [MM: In Athena, PV-Finder is about 9 times faster than AMVF at $\mu = 200$ on a GPU; on a CPU the timed model is slower. For the frozen chain on a CPU, a projection from measured component times gives about 413 ms per event against about 650 ms for AMVF (Section 5).]

The chain is completed by the graph-network track-to-vertex association (Section 7), which turns the found peaks into per-vertex track lists. At PU200, the combined PV-Finder+GNN chain gives a clean-vertex yield of ~~68.4%~~ [MM: 50.6% ($t = 0.94$, vertices with at least two tracks)], compared with 30.9% for AMVF under the same track-based classification[MM: , and 41.6% for AMVF classified with its own fit weights]. A truth-guided assignment on the PV-Finder peaks gives ~~73.7%~~ [MM: 68.5%] with a ~~0.5 mm~~ [MM: 0.17 mm] window. This is a diagnostic reference and not a strict upper bound. ~~The GNN also identifies the hard-scatter vertex in 94.4% of events, compared with 80.7% for AMVF.~~ [MM: The hard-scatter vertex is reconstructed and selected in 95.1% of events, compared with 92.9% for AMVF, and its longitudinal resolution is 9.5 μm against 10.2 μm. The HS vertex is Clean less often than for AMVF (88.3% against 92.5%). These numbers use the frozen GNN v2 and will be updated with the new GNN.] Earlier component timings give an estimated 11 ms per event for smaller PU200 graphs, including a median GNN forward time of 3.8 ms. This was not a measurement of the complete chain. A separate training test found that modifying the truth graphs to resemble reconstructed peaks reduced the clean-vertex yield from 68.4% to 65.0% under the tested training conditions.

Bibliography

[1] *Development of ATLAS Primary Vertex Reconstruction for LHC Run 3*, (2019) (cit. on p. 6).

[2] ATLAS Collaboration, *Software Performance of the ATLAS Track Reconstruction for LHC Run 3*, Comput. Softw. Big Sci. **8** (2024) 9, arXiv: 2308.09471 [hep-ex] (cit. on pp. 6, 28).

[3] S. Akar et al., *Advances in developing deep neural networks for finding primary vertices in proton-proton collisions at the LHC*, EPJ Web Conf. **295** (2024) 09003, arXiv: 2309.12417 [hep-ex] (cit. on pp. 6, 9, 66).

[4] ATLAS Collaboration, *Primary Vertex identification using deep learning in ATLAS*, ATL-PHYS-PUB-2023-011, 2023, URL: https://cds.cern.ch/record/2858348 (cit. on p. 6).

[5] *Primary Vertex identification using deep learning in ATLAS*, (2023) (cit. on pp. 6, 66).

[6] Q. B. Lei and R. B. Garg, *Deep Learning for Primary Vertex Identification in the ATLAS Experiment*, Talk at ACAT 2025, Hamburg, Germany, 2025, URL: https://indico.cern.ch/event/1488410/contributions/6561543/ (cit. on p. 6).

[7] ATLAS Collaboration, *Expected tracking performance of the ATLAS Inner Tracker at the High-Luminosity LHC*, JINST **20** (2025) P02018, arXiv: 2412.15090 [hep-ex] (cit. on pp. 8, 21, 45).

[8] O. Ronneberger, P. Fischer and T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, 2015, arXiv: 1505.04597 [cs.CV], URL: https://arxiv.org/abs/1505.04597 (cit. on p. 9).

[9] P. Veličković et al., *Graph attention networks*, arXiv preprint arXiv:1710.10903 (2017) (cit. on p. 39).

[10] ATLAS Collaboration, *Track and Vertex Reconstruction with the ATLAS Inner Detector*, (2026), arXiv: 2605.07585 [hep-ex] (cit. on p. 40).

[11] S. Akar, M. Peters, H. Schreiner, M. D. Sokoloff and W. Tepe, 'Comparing and improving hybrid deep learning algorithms for identifying and locating primary vertices', *21th International Workshop on Advanced Computing and Analysis Techniques in Physics Research: AI meets Reality*, 2023, arXiv: 2304.02423 [hep-ex] (cit. on p. 66).

List of contributions

Mattia Mauro	Analysis, model development, writing
Rocky Bala Garg	Sample preparation, analysis, advising, writing, project conception
Lauren Tompkins	Advising, writing, project conception
Qi Bin Lei	Initial model development, analysis
Trey Warner	Athena Integration, writing

Appendices

A Reproduction

This appendix gives the exact commands, paths and checkpoints needed to reproduce the results. All commands are run from the repository root with the virtual environment active. Heavy jobs are run inside `tmux` with unbuffered output, since the host is shared.

```
source venv/bin/activate
pip install -r requirements.txt
```

A.1 Results-settings matrix

[MM: This table and the commands below describe the earlier configuration (the 3.55M-parameter model, integral 0.30 and height 0.03, 0.1694 mm). The frozen configuration is Table 2.2; this appendix is to be updated after the final recompute.] Table A.1 records, for each main result in this note, the dataset, the file or tag, the truth reference, the model checkpoint and the peak-finding settings (the peak amplitude threshold, the integral threshold and the minimum-height floor), together with whether the post-hoc GBT gate is applied. The same model can move by several percentage points in fake rate when these settings change, so each number in the main text should be read together with its row here. Full checkpoint paths are given in Table A.3.

Table A.1: Evaluation settings for the main results. "Repro. ep300" is the Run 2 reproduction checkpoint and "v4b ep3" the HL-LHC v4b Phase 2 epoch 3 checkpoint (Table A.3). The Run 2 simulation uses `run_eval_pvf.py` (which applies no height floor); all other rows use `run_eval_pvf_run3.py`. "beam-corr." indicates that the AMVF beam-spot z offset is subtracted.

Result	File / tag	Reference	Checkpoint	Peak thr.	Int. thr.	Min. height	GBT
Run 2 simulation	HDF5 (qibin map)	MC truth	Repro. ep300	0.01	0.5	-	off
Run 2 data	Run 2 ROOT	AMVF (beam-corr.)	Repro. ep300	0.01	0.5	0.0	off
Run 3 data	`run3/file_3`	AMVF (beam-corr.)	Repro. ep300	0.01	0.5	0.0	off
HL-LHC production counts	`r16443`	MC truth	production ep2	0.01	0.30	0.03	off
HL-LHC resolution plot	`r16443`	-	production ep2	0.01	0.50	0.03	off
HL-LHC reconstruction check	`r16638`	MC truth	production ep2	0.01	0.30	0.03	off
TTVA PU200 chain	`r16443`	MC truth	production ep2 + v4 ep107	0.01	0.30	0.03	off
TTVA low-pile-up chain	HDF5 (qibin map)	MC truth	e400 + mu60v2 ep115	0.01	0.2	-	off
superseded ($\|\eta\| < 2.5$ pool), kept for provenance							
HL-LHC v4b baseline	`r16438`	MC truth	v4b ep3	0.01	0.2	0.0	off
HL-LHC v4b + GBT	`r16633`	MC truth	v4b ep3	0.01	0.2	0.0	thr 0.3

The main positional comparison uses a fixed 0.1694 mm matching window for both algorithms (Table 4.1). The same production peak settings also give the wider-window results in Appendix D.2; only the matching window changes. The resolution plot uses the higher integral cut of 0.50 to remove more small peaks. The reconstruction check also uses 0.50 for its separation fit and 0.30 for its counts. These are different peak selections from the same model.

Two cautions on this table. The HL-LHC production rows use a peak position taken from a local centroid of half-width 3 bins rather than a weighted mean over the whole split region; that estimator change is worth a substantial part of the $\sigma_{\text{vtx-vtx}}$ improvement and must be set to reproduce the quoted resolution. And the superseded rows use the $|\eta| < 2.5$ pool and a smaller truth denominator, so their higher efficiencies do not establish better reconstruction; they are not comparable with the current rows. `r16438` is additionally inside the v4b training pool.

The two TTVA rows carry a second operating point that the finder rows do not: the MaxScore threshold t on the edge scores, 0.98 at PU200 and 0.99 at low pile-up for vertex classification, and 0.95 for PU200 hard-scatter identification (Section 7.4). The threshold for the recorded low-pile-up hard-scatter result remains to be confirmed. They are also classified by the track-purity taxonomy rather than the positional one, so their Clean/Merged/ Split/Fake counts are not comparable with the rows above; the minimum-width requirement is 3 bins throughout.

A.2 Effect of extended track acceptance

The two `r16443` productions contain the same events. Their `ActualNumOfInt` and AMVF vertex-z values agree event by event. The older ntuple contains tracks with $|\eta| < 2.5$, while the new one contains tracks up to $|\eta| < 4$ and uses a uniform track selection. The comparison is therefore mainly a test of the wider acceptance, although the change in track selection means that it is not a pure $|\eta|$ cut.

Table A.2 uses the same 1 920 held-out events with $185 \leq \mu \leq 215$ and the same peak-finding settings in every row. Running both v4b and production on both ntuples checks the data change with a fixed checkpoint. The comparison between v4b and v5 uses the old ntuple; these two trainings differ only in the target-resolution preset.

Table A.2: Controlled comparison of the track acceptance and target-resolution prescription. Old denotes the $|\eta| < 2.5$ ntuple and full denotes the $|\eta| < 4$ ntuple. Efficiency uses the model's fitted $\sigma_{\text{vtx-vtx}}$ as the matching window.

Checkpoint	Input	Training target	Tracks/event	Truth/event	Clean/event	Fake/event	Efficiency	$\sigma_{\text{vtx-vtx}}$ [mm]
v4b	old	`hllhc`	887.0	95.2	67.5	11.4	90.7%	0.229
v4b	full	`hllhc`	1 313.1	111.3	69.7	13.1	84.9%	0.228
production	old	`hllhc_alleta`	887.0	95.2	69.0	16.5	92.5%	0.234
production	full	`hllhc_alleta`	1 313.1	111.3	71.9	16.3	86.9%	0.229
v5	old	`hllhc_corrected`	887.0	95.2	69.3	17.1	92.2%	0.226

At fixed v4b checkpoint, the full-acceptance input adds 16.2 truth vertices and 2.3 clean vertices per event. At fixed production checkpoint, it adds the same 16.2 truth vertices and 2.9 clean vertices. The efficiency falls by about 5.7 percentage points because the truth denominator grows much faster than the clean yield. The vertex-vertex resolution changes by at most 0.005 mm.

The data-level result gives the same picture. In the full PU200 samples, the number of tracks rises from 921 to 1 361 per event and the number of truth vertices with $n_{\text{trk}} \geq 2$ rises from 98.4 to 115.7. About 19.2 vertices per event pass the two-track requirement only because of forward tracks. For vertices already accepted with central tracks, adding the forward tracks improves an uncertainty-based estimate of the vertex-z resolution by only 0.1 to 0.2%. The forward tracks therefore increase vertex acceptance but add little position information.

The last row of Table A.2 shows the resolution-only training check. At the same operating point, changing the target-resolution preset from v4b to v5 adds 1.8 clean vertices per event but also 5.7 fake vertices. The target width also sets the target peak height, so the peak-height cut must be retuned before treating this as a net improvement. This effect is smaller than the change in the truth population caused by the extended acceptance.

In the matched 1 920-event comparison, the number of truth vertices with $n_{\mathrm{trk}} \geq 2$ rises from 95.2 to 111.3 per event. The clean count grows much less, so the efficiency falls. Forward tracks have poorer z_0 resolution: about 2.8 mm at $|\eta| = 3.7$, compared with 65 μm near $\eta = 0$. With inverse-variance weighting, a track with 2.8 mm resolution has about $1/2000$ of the weight of a central track. The additional truth vertices are therefore difficult for both algorithms.

A.3 Data conversion

The HL-LHC ROOT files are converted to the flat HDF5 layout with a two-pass converter. The preset selected on the command line sets the target widths and is written into the file metadata; the extended-$|\eta|$ pool requires `hllhc_alleta`, since neither earlier preset describes it (Table B.3). The converter requires the preset to be passed explicitly.

```
# One ROOT file -> compressed HDF5, extended-|eta| pool
python -u src/pv_finder/data/root_to_h5.py \
    --input  data/run4_all_etas/Run4_MC21_ITk_LatestJuly2026/
    ATLAS_PVFinderData_601229_e8481_s4494_r16633_PU200.root \
    --output data/run4_all_etas/PU200_alleta_h5/
    ATLAS_PVFinderData_601229_e8481_s4494_r16633_PU200.h5 \
    --resolution-preset hllhc_alleta \
    --max-tracks-per-sub 1536 --max-pv 300 \
    --compression lzf --max-events 0
```

To measure the resolution prescription, first save the reconstructed-truth residuals, then fit their Gaussian cores with a constant background:

```
python -u src/pv_finder/diagnostics/amvf_resolution_vs_ntracks.py \
    --root data/run4_all_etas/Run4_MC21_ITk_LatestJuly2026/
    ATLAS_PVFinderData_601229_e8481_s4494_r16633_PU200.root \
    --out-dir outputs/MM_DD_YYYY_output/amvf_resolution
python -u src/pv_finder/diagnostics/resolution_fit_v2.py \
    --npz outputs/MM_DD_YYYY_output/amvf_resolution/vertex_data.npz \
    --out-dir outputs/MM_DD_YYYY_output/amvf_resolution_corrected
```

For reproducing the trained model, use the stored `hllhc_alleta` preset rather than substituting a newly fitted value.

A.4 Training

The production HL-LHC model trains from the extended-$|\eta|$ PU200 pool of Section 3.0.2, listed in its configuration. Phase 1 warms up the MLP; Phase 2 trains end-to-end with the per-step learning-rate warmup.

```
tmux new -s hllhc_v6
source venv/bin/activate
python -u src/pv_finder/training/train_hllhc_e2e.py \
    -c configs/vertex_finding/config_hllhc_alleta_v6_mse.yml \
    2>&1 | tee outputs/logs/hllhc_alleta_v6.log
# Ctrl+B, D to detach

# Resume Phase 2 from epoch N (restores model, optimizer and scheduler):
python -u src/pv_finder/training/train_hllhc_e2e.py \
    -c configs/vertex_finding/config_hllhc_alleta_v6_mse.yml \
    --resume model_weights/hllhc_alleta_v6_mse_2ep_phase2_epoch_1_fullstate.pth
```

The data must be converted with the `hllhc_alleta` resolution preset and `--max-tracks-per-sub1536`. The conversion logs for the full 5.17M-event training pool show a maximum of 1 103 tracks in one 40 mm interval and no truncation. The fixed width is needed only for batching and storage. If it were exceeded, truncation would be z_0 ordered and would drop the highest-z_0 tracks while the target histogram kept their vertices.

Training is tracked with MLflow under `file:<repo>/mlruns`. The earlier PU200 models use `config_hllhc_pu200_e2e_v2.yml` (v2 wide) and `config_hllhc_pu200_e2e_v3.yml` (v3), and the sweep uses the `config_hllhc_pu200_e2e_v3_runA.yml`, with corresponding runB and runC files.

A.5 Evaluation

The HL-LHC evaluation uses `run_eval_pvf_run3.py`. The production model is a U-Net v2 with 280 channels and four latent channels, so the architecture must be specified explicitly. The integral threshold is 0.30 for counting vertices. The resolution plot uses 0.50 to remove more small peaks and make the dip clearer. The height floor is 0.03, and the summary window covers the discrete pile-up values $185 \leq \mu \leq 215$. The two held-out tags are flat-μ, so the window selects the PU200 slice from them.

```
python -u src/pv_finder/evaluation/vertex_finding/run_eval_pvf_run3.py \
    --root data/run4_all_etas/Run4_MC21_ITk_LatestJuly2026/
    ATLAS_PVFinderData_601229_e8481_s4494_r16443_PU200.root \
    --e2e-model model_weights/hllhc_alleta_v6_mse_2ep_phase2_epoch_2_fullstate.pth \
    --e2e-type v2 --e2e-unet-channels 280 --e2e-latent-channels 4 \
    --e2e-hidden 128 128 128 128 128 \
    --max-events 25000 --mu-min 185 --mu-max 215 \
    --peak-threshold 0.01 --integral-threshold 0.30 --integral-threshold-res 0.50 \
    --min-height 0.03 --centroid-halfwidth 3 --pairwise-bins 300 \
    --output-dir outputs/MM_DD_YYYY_output/eval_v6_r16443
```

The fixed-window numbers of Table 4.1 are not produced by this script, which derives its matching window from PV-Finder's own fitted $\sigma_{\text{vtx-vtx}}$ and applies it to AMVF as well. They come from the dedicated audit, which measures the core position resolution of both algorithms with an optimal (not greedy) assignment, sets the window from AMVF's, and bootstraps the difference over events. Measuring the core resolution greedily biases the width about 10% low, which tightens the window and flatters PV-Finder.

The fixed-window comparison can be repeated using the saved cache, without running the finder again:

```
python -u src/pv_finder/diagnostics/amvf_fair_comparison/run_fair_comparison.py \
    --cache outputs/08_05_2026_output/amvf_fair_comparison/cache_r16443.npz \
    --outdir outputs/MM_DD_YYYY_output/amvf_fair_comparison --tag r16443 \
    --core-multiple 3.0 --core-pct 68 --sensitivity-windows 0.5 \
    --n-boot 400 --n-boot-headline 2000 --seed 20260805
```

To apply the current GBT filter, add the following flags to the finder command. For the filter study use `r16638` and `--max-events6000`, which selects 268 PU200 events. The classifier was trained on peaks from `r16443`; the two reconstruction tags share the event generation and detector simulation.

```
    --gbt-filter-model outputs/08_20_2026_output/peak_classifier_v6/peak_classifier_results.pkl
    \
    --gbt-threshold 0.3
```

This command fits a new resolution and uses it as the matching window. Table D.5 instead replays the saved before/after peak lists at one fixed window per comparison: 0.228983 mm for the height cut and 0.227832 mm for the GBT. The following check reads those stored outputs and reproduces the fixed-window totals without inference:

```
python ../ANA-IDTR-2026-01-INT1/scripts/check_hllhc_results.py
```

The low-pile-up evaluations use the same script (Run 2/3 collision data) or `run_eval_pvf.py` (Run 2 simulation), for example

```
# Run 3 collision data, end-to-end model
python -u src/pv_finder/evaluation/vertex_finding/run_eval_pvf_run3.py \
    --root data/run3/file_3.root \
    --e2e-model model_weights/03_24_2026/
    reproduction_T2HIST_400ep_T2KDE100_K2H150_epoch_300_fullstate.pth \
    --e2e-type v1 --max-events 2500 --mu-min 0 --mu-max 1000 \
    --peak-threshold 0.01 --integral-threshold 0.5 --integral-threshold-res 0.5 \
    --min-height 0.0 --centroid-halfwidth 0 --pairwise-bins 60 \
    --output-dir outputs/MM_DD_YYYY_output/eval_run3
```

Each run saves the evaluation in `eval_results.pkl`. The plots are `resolution_plot.png`, `performance_plot.png`, `stats_histogram.png` and `category_counts_hist.png`.

A.6 Track-to-vertex association

The TTVA models of Chapter 7 live under `src/gnn/` and are reproduced in three steps: build graphs, train, evaluate. Two samples are used, low pile-up (event-keyed HDF5) and PU200 (ROOT), and they take different graph builders.

Known quirk. On the shared host, `gnn.data.h5_to_graphs`, `gnn.evaluation.evaluate_amvf_ttva` and `gnn.diagnostics.plot_ttva_performance` hang at 100% CPU when launched with `python -m` *and* HDF5 plus indices arguments, in a process that does not respond to `SIGKILL`. The identical modules run normally through `runpy`, which is the form used below and in the `scripts/` wrappers.

A.6.1 Graph building

At low pile-up the training graphs are built from the association HDF5, in the event order of the original training index file so that the shuffled 2 550-event test set is excluded by construction. The chain-like augmentation of Section 7.7 is applied at build time with probability 0.7, from parameters measured on the chain itself; the whole dataset is produced by `scripts/build_mu60_v2_shards.sh`.

```
# Augmentation parameters, measured on the mu60 chain
python -u -m gnn.diagnostics.mu60_aug_params \
    --output-dir outputs/MM_DD_YYYY_output/mu60_aug_params

# One training shard (runpy wrapper, see "Known quirk" above)
python -u -c "
import sys, runpy
sys.argv = ['h5_to_graphs',
            '-f', '/share/lazy/qibinlei/recoTracks_incamvfassoc.h5',
            '-i', '/data/home/matmauro/codice/atlas_pvfinder/tracks_to_vertex/
    training_indices_no_test.p',
            '--start-event', '0', '-n', '17850', '--seed', '300',
            '--augment-params', 'outputs/MM_DD_YYYY_output/mu60_aug_params',
            '--aug-prob', '0.7',
            '-o', 'data/mu60/ttva_graphs/mu60v2_train_shard_0.pt']
runpy.run_module('gnn.data.h5_to_graphs', run_name='__main__')
"
```

At PU200 the truth-training graphs come from the ROOT ntuples through `gnn.data.root_to_graphs`; the 18 files of 10k events used for the current model are built by `scripts/build_ttva_v4_shards.sh`. The *chain* graphs, whose vertex nodes are PV-Finder peaks rather than truth vertices, are built by `gnn.data.pu200_chain_graphs`, which in the same ROOT pass runs the production finder, finds peaks at the production operating point, attaches `track.truth_pv` from the truth, computes the AMVF baseline from `RecoVertex_assocTracks` through the identical classifier, and records the per-stage wall times.

```
python -u -m gnn.data.pu200_chain_graphs \
    --root data/run4_all_etas/Run4_MC21_ITk_LatestJuly2026/
    ATLAS_PVFinderData_601229_e8481_s4494_r16443_PU200.root \
    --pvf-weights model_weights/hllhc_alleta_v6_mse_2ep_phase2_epoch_2_fullstate.pth \
    --entry-start 0 --entry-stop 25000 --mu-min 185 --mu-max 215 -d 0 \
    --knn 20 --peak-threshold 0.01 --integral-threshold 0.30 \
    --min-width 3 --min-height 0.03 --centroid-halfwidth 3 \
    --unet-channels 280 --latent-channels 4 --hidden-nodes 128 \
    --output data/run4_all_etas/ttva_graphs/pu200_chain_v6_k20_test.pt \
    --output-dir outputs/MM_DD_YYYY_output/gnn_chain/
```

The gap between truth graphs and chain graphs, which sets the augmentation parameters, is measured by `gnn.diagnostics.chain_gap_decomposition`; it must be re-run and the shards rebuilt whenever the finder or its operating point changes.

A.6.2 Training

Both models train from a YAML configuration. One file is used per epoch and the files are cycled by `gnn.training.shard_loader`.

```
tmux new -s ttva
source venv/bin/activate
# PU200: 108 epochs = 6 passes over 180k events
python -u -m gnn.training.train_ttva -c configs/gnn/config_gnn_ttva_hllhc_v4_noaug.yml
# mu60: 120 epochs = 60 passes over 35.7k events
python -u -m gnn.training.train_ttva -c configs/gnn/config_gnn_ttva_mu60_v2.yml
```

A.6.3 Evaluation

`gnn.evaluation.chain_scan` is the headline tool: it runs the forward pass once per graph, caches the edge scores, and re-applies the MaxScore selection over a threshold grid, classifying each operating point under both vertex conventions of Section 7.4. The `drop_empty` rows are the ones quoted in this note.

```
# Threshold scan on the chain graphs (produces the working-point scan figures)
python -u -m gnn.evaluation.chain_scan \
    -r data/run4_all_etas/ttva_graphs/pu200_chain_v6_k20_test.pt \
    -w model_weights/ttva_gnn_hllhc_v4_noaug/ttva_gat_alleta_k20_v4_noaug180k_epoch_107.pyt \
    --first-event 0 -d 0 -o outputs/MM_DD_YYYY_output/chain_scan/

# Truth-graph evaluation and the learning curve
python -u -m gnn.evaluation.evaluate_ttva_graphs \
    -r data/run4_all_etas/ttva_graphs/pu200_truth_k20_test_r16443.pt \
    -w model_weights/ttva_gnn_hllhc_v4_noaug/ttva_gat_alleta_k20_v4_noaug180k_epoch_107.pyt \
    -t 0.95 -d 0 -o outputs/MM_DD_YYYY_output/truth_eval/
python -u -m gnn.evaluation.evaluate_checkpoints \
    -r data/run4_all_etas/ttva_graphs/pu200_truth_k20_test_r16443.pt \
    -w model_weights/ttva_gnn_hllhc_v4_noaug/ -t 0.95 -d 0 \
    -o outputs/MM_DD_YYYY_output/truth_learning_curve/

# Hard-scatter identification through the chain
python -u -m gnn.evaluation.hs_id_pu200 \
    --graphs data/run4_all_etas/ttva_graphs/pu200_chain_v6_k20_test.pt \
    --root data/run4_all_etas/Run4_MC21_ITk_LatestJuly2026/
    ATLAS_PVFinderData_601229_e8481_s4494_r16443_PU200.root \
    --entry-indices outputs/MM_DD_YYYY_output/gnn_chain/entry_indices.npy \
    -w model_weights/ttva_gnn_hllhc_v4_noaug/ttva_gat_alleta_k20_v4_noaug180k_epoch_107.pyt \
    -d 0 -o outputs/MM_DD_YYYY_output/hs_id/
# The hard-scatter tool reports t = 0.5, 0.95 and 0.995.
```

At low pile-up the AMVF reference is classified by the identical core through `gnn.evaluation.evaluate_amvf_ttva`, and the Run 3 collision-data comparison by `gnn.evaluation.evaluate_ttva_run3`.

```
python -u -c "
import sys, runpy
sys.argv = ['x', '-f', '/share/lazy/qibinlei/recoTracks_incamvfassoc.h5',
            '-i', 'configs/qibin_test_main_indices_v2.p',
            '-o', 'outputs/MM_DD_YYYY_output/amvf/']
runpy.run_module('gnn.evaluation.evaluate_amvf_ttva', run_name='__main__')
"
```

The JIT-compiled peak scan and the vectorized MaxScore selection used for the timing estimate in Section 7.7 are checked against the reference implementations by `gnn.evaluation.verify_fast_paths`, and

`gnn.evaluation.regression_guard` re-checks a stored operating point row by row. The GPU forward pass is nondeterministic at the 10^{-5} level (attention scatter atomics), so a track whose top two scores are closer than that can change vertex between runs; the guard accepts bit-exact rows and knife-edge differences only.

A.7 Model checkpoints

Table A.3 lists the checkpoints used in this note. All are `*_fullstate.pth` files under `model_weights/`, except the TTVA models, which are loaded from the weights-only `*.pyt` files written alongside them. The low-pile-up GNN chain also uses an earlier finder checkpoint at epoch 400, stored in the sibling `atlas_pvfinder/` checkout at `model_weights/e2e_attempt4_mlp_hist/e2e_mlpHist50_e2e400_1latent_mse_phase2_epoch_400.pyt`. Its graph-building integral threshold is 0.2. This differs from the reproduction model and threshold used in Chapter 6.

Table A.3: Checkpoints used in this note.

Role	File	Eval flags
HL-LHC production	`hllhc_alleta_v6_mse_2ep_phase2_epoch_2`	`-e2e-type v2 -e2e-unet-channels 280 -e2e-latent-channels 4`
HL-LHC v4b (superseded)	`hllhc_pu200_e2e_v4b_3ep_280ch_4lat_stepwarmup_phase2_epoch_3`	`-e2e-type v2 -e2e-unet-channels 280 -e2e-latent-channels 4`
HL-LHC v2 wide	`hllhc_pu200_mlp50_e2e400_v2_phase2_epoch_100`	`-e2e-type v1 -e2e-wide`
Run 2 simulation (canonical)	`03_24_2026/reproduction_T2HIST_400ep_T2KDE100_K2H150_epoch_300`	`-e2e-type v1`
Run 2/3 KDE-free	`e2e_mlpHist50_e2e400_1latent_mse_phase2_epoch_130`	`-e2e-type v1`
TTVA PU200 current (v4)	`ttva_gnn_hllhc_v4_noaug/ttva_gat_alleta_k20_v4_noaug180k_epoch_107`	$t = 0.98$ (vertices) $t = 0.95$ (hard scatter)
TTVA PU200 augmented arm	`ttva_gnn_hllhc_v4_aug/ttva_gat_alleta_k20_v4_aug180k`	$t = 0.98$, A/B control
TTVA low pile-up current	`ttva_gnn_mu60_v2/ttva_gat_mu60_v2_aug_epoch_115`	$t = 0.99$

A.8 Key paths and test sets

- Current HL-LHC training pool: six HDF5 files in `data/run4_all_etas/PU200_alleta_h5/` (5 173 600 events), listed in `configs/vertex_finding/config_hllhc_alleta_v6_mse.yml`; split 0.96/0.03/0.01. The eight-file, 2.74M-event pool in `data/run4/PU200_withTiming_h5/` belongs to the earlier v4/v4b models.

- Run 2 simulation HDF5: the flat 51 000-event file, test set `configs/test_main_indices_2550evt.p` (event indices 48 450 to 50 999) with the ROOT index map `configs/qibin_test_main_indices_v2.p`.
- Run 3 collision data: `data/run3/file_3.root` (NPZ cache available).
- TTVA low pile-up: association HDF5 `recoTracks_incamvfassoc.h5` (event-keyed, 51 000 events); training order from `training_indices_no_test.p` (positions 0 to 35 700 train, 35 700 to 48 450 validation) and the shuffled 2 550-event test set `configs/qibin_test_main_indices_v2.p`.
- Current TTVA PU200 training graphs: `data/run4_all_etas/ttva_graphs/v4_shards_pristine/`, with eighteen 10k-event training shards and one 5k-event validation shard. The augmented control uses `v4_shards_augmented/` in the same parent directory. Test graphs are `pu200_truth_k20_test_r16443.pt` and `pu200_chain_v6_k20_test.pt`, both under `data/run4_all_etas/ttva_graphs/`. They contain the same 1 920 events and 213 738 truth vertices with $n_{\text{trk}} \geq 2$. Their ROOT entries are non-contiguous; use the chain builder's `entry_indices.npy` whenever truth is read again.
- The v3 shards and v4b chain graphs in `data/run4/ttva_graphs/` belong to the earlier PU200 study. Older graphs also predate the correction to the vertex-height calculation; use the graph files named above for the current results.
- Outputs are written to dated folders `outputs/MM_DD_YYYY_output/`.

B Hyperparameters

Table B.1 collects the training and model hyperparameters of the HL-LHC vertex-finder configurations referenced in the text, taken verbatim from the configuration files in `configs/vertex_finding/`. All use the mean-squared-error loss and the Adam optimizer (the v3 sweep also tried SGD with momentum). Phase 1 is the MLP-only warmup with the U-Net frozen; Phase 2 is the end-to-end training. The table includes the production model and two earlier models, v3 and v4b, trained on the $|\eta| < 2.5$ samples.

Table B.1: Training and model hyperparameters of the HL-LHC configurations.

	v3	v4b	**production (current)**
Data			
Track acceptance	$\|\eta\| < 2.5$	$\|\eta\| < 2.5$	$\|\eta\| < 4.0$
Events	99.8K	2.74M (8 files)	5.17M (6 files)
Resolution preset	`hllhc`	`hllhc`	`hllhc_alleta`
Train/val/test	0.70/0.25/0.05	0.96/0.03/0.01	0.96/0.03/0.01
Batch size	128	128	128
Phase 1 (MLP warmup)			
Epochs	50	3	2
Learning rate	10^{-3}	10^{-3}	10^{-3}
Phase 2 (end-to-end)			
Epochs	200	3	2
Learning rate	10^{-4}	10^{-4}	10^{-4}
Warmup	5 epochs	3 000 steps	3 000 steps
Schedule	cosine	per-step cosine	per-step cosine
$\eta_{\min}$ frac	0.01	0.01	0.01
Gradient clip	1.0	1.0	1.0
Model			
Architecture	U-Net v2	U-Net v2	U-Net v2
U-Net channels	280	280	280
Latent channels	4	4	4
MLP hidden	$[128] \times 5$	$[128] \times 5$	$[128] \times 5$
Dropout	0.25	0.25	0.25
Parameters	3.55M	3.55M	3.55M

The three models have the same architecture and parameter count. The production model uses the full-acceptance data and the target widths fitted on that sample. The earlier wide-96-channel configuration is recorded in Appendix C.

Table B.2 does the same for the two production track-to-vertex association models of Chapter 7, taken from `configs/gnn/`. Both use binary cross-entropy with a positive-class weight recomputed per batch (the ratio of negative to positive edges in that batch) and the Adam optimizer. One epoch is one shard, so the number of passes over the data is the epoch count divided by the number of training shards.

Table B.2: Training and model hyperparameters of the production TTVA models. The PU200 augmented arm of Section 7.7 differs from the column shown only in the augmentation probability (0.7); every other setting, the file boundaries and the random seed are identical.

	PU200	low pile-up
Data		
Training events	180k	35.7k
Files × size	18×10k	$2 \times 17\,850$
Validation events	5k	12 750
Process	all-hadronic $t\bar{t}$	$t\bar{t}$
Graph connectivity	$k = 20$ nearest	fully connected
Augmentation prob.	None	0.7
Batch size	32	32
Training		
Epochs (passes)	108 (6)	120 (60)
Learning rate	10^{-3}	10^{-3}
Schedule	cosine	cosine
$\eta_{\min}$	10^{-5}	10^{-5}
Gradient clip	1.0	1.0
Model		
Track input features	8	8
Vertex input features	2	2
Embedding dimension	32	32
Message-passing rounds	2	2
Attention heads	4	4
Edge attributes	3	3
Parameters	39 521	39 521
Selection		
Working point t	0.98	0.99
Hard-scatter t	0.95	0.5

The target-histogram resolution model of Equation 2.2 uses the presets in Table B.3, selected on the converter command line and recorded in the output file metadata.

Table B.3: Resolution-model presets, $\sigma(n_{\text{trk}}) = A\, n_{\text{trk}}^{-B} + C$. The production model uses `hllhc_alleta`, refitted on the extended-$|\eta|$ sample; it gives a *wider* σ at fixed n_{trk} than `hllhc`, because n_{trk} there counts low-information forward tracks alongside central ones. The earlier presets must not be reused on that sample.

Preset	A (mm)	B	C (mm)
`hllhc_alleta` (ITk, PU200, $\lvert\eta\rvert < 4$)	0.14318	0.36200	−0.01797
`hllhc` (ITk, PU200, $\lvert\eta\rvert < 2.5$)	0.17898	0.7274	0
`run3` (Inner Detector)	0.23817	0.49491	−0.00079

C Historical model development

PV-Finder was first developed for LHCb [3, 11] and adapted to ATLAS as a two-stage pipeline [5]: a tracks-to-KDE network mapping the track perigee parameters to a kernel-density estimate (KDE) of the vertex density along the beamline, followed by a KDE-to-histogram network refining the KDE into the vertex histogram. Throughout the two-stage era the first stage was trained against an analytical KDE computed directly from the track parameters, and the earliest configurations used the analytical KDE itself as the model input. The end-to-end model of this note (Section 2.2.2) removes the KDE from both training and inference; this appendix records how the analytical KDE was constructed, since it remains the reference input representation for the low-pile-up reproduction baseline and for several diagnostics.

C.1 Analytical KDE construction

Each track's position at the point of closest approach (POCA) to the beam line is characterized by its transverse and longitudinal impact parameters (d_0, z_0) and their covariance matrix

$$\Sigma = \begin{pmatrix} \sigma^2(d_0) & \sigma(d_0, z_0) \\ \sigma(d_0, z_0) & \sigma^2(z_0) \end{pmatrix}. \tag{C.1}$$

The track is modeled as a correlated two-dimensional Gaussian probability density centred at (d_0, z_0),

$$\mathbb{P}(d, z) = \frac{1}{2\pi\sqrt{|\Sigma|}} \exp\left(-\frac{1}{2}\Big((d - d_0), (z - z_0)\Big)^T \Sigma^{-1}\Big((d - d_0), (z - z_0)\Big)\right), \tag{C.2}$$

where (d, z) is a generic point in the radial-longitudinal plane and $|\Sigma|$ is the determinant of Σ. A grid search in the transverse plane with a 20 μm $\times$ 20 μm step locates the region of highest summed probability density. For each z bin, the probability values of all contributing tracks are summed; a track contributes to a bin if the bin lies within $z_0 \pm 3\sigma(z_0)$. Two features are built from this process:

- `KDE-A`: the sum of the track probability values;
- `KDE-B`: the sum of the squares of the track probability values.

Only the relative scale of the probability contributions matters, so normalization constants are dropped. `KDE-B` accentuates the features of `KDE-A`: squaring the per-track probabilities makes genuine peaks, where several tracks overlap, stand out more distinctly from single-track fluctuations.

D Additional PU200 studies

[MM: The studies in this appendix were made with the earlier 3.55M-parameter model and its peak cuts, at 0.1694 mm. They are kept as documentation and their numbers are not updated to the frozen configuration (Table 2.2).] The main result uses one evaluation sample and a fixed 0.1694 mm matching window. This appendix gives the full comparison table and checks with a wider window and another reconstruction of the simulated sample. These comparisons use the same trained finder. The later sections give examples of missed, merged and fake vertices, tests of cuts to remove fake peaks, and the satellite-peak study. Some use an earlier model or different peak cuts, as stated in each study.

D.1 Full comparison at the fixed window

Table D.1 expands the comparison in Table 4.1. It separates fake and split counts and shows the effect of including truth vertices with only one selected track. The main efficiency uses truth vertices with at least two selected tracks.

D.1.1 Single-track truth vertices

The truth selection also affects the fake count. A real interaction with only one selected track is left out of the $n_{\mathrm{trk}} \geq 2$ reference. A peak near it can therefore be counted as fake. Including these single-track truth vertices lowers the fake rate for both methods (Table D.1).

After accounting for estimated chance matches to single-track vertices, we find that they account for 3.98 ± 0.05 vertices per event currently counted as fake by PV-Finder, and 2.73 ± 0.05 by AMVF, at $w \simeq 0.17$ mm. The difference on the same events is 1.25 ± 0.04 per event. At 0.5 mm, the contributions are 3.33 ± 0.05 and 2.36 ± 0.05, with a difference of 0.97 ± 0.04. These estimates depend on the matching window. The main results keep the requirement of at least two selected tracks per truth vertex.

D.1.2 Integral cut and binning of the resolution plot

The efficiency and vertex counts use an integral cut of 0.30. For the resolution plot, we use 0.50 to remove more small peaks and make the dip clearer. Table D.2 compares the two cuts on the same 1 920 events. All other peak settings stay fixed, and the efficiency and fake counts use the same 0.1694 mm truth-matching window.

Raising the cut removes 3.642 ± 0.045 fake vertices per event and reduces efficiency by 0.743 ± 0.019 percentage points. These errors come from 2 000 paired resamples of whole events. The fitted separation scale also changes. The smaller value with the 0.30 cut does not show an improvement in the model:

Table D.1: Full comparison at the fixed 0.1694 mm matching window on the same 1 920 events. Efficiency uses truth vertices with $n_{\text{trk}} \geq 2$. The two fake-rate columns use truth vertices with $n_{\text{trk}} \geq 2$ and $n_{\text{trk}} \geq 1$, respectively. Fake+split uses $n_{\text{trk}} \geq 2$. Uncertainties are estimated by resampling events (bootstrap). Each resample uses the same events for both methods. No GBT filter is applied.

	Efficiency [%]		Vertices/event		
	Merged included	One-to-one	Fake, $n_{\text{trk}} \geq 2$	Fake, $n_{\text{trk}} \geq 1$	Fake+split
PV-Finder	83.16 ± 0.08	73.43 ± 0.09	18.73 ± 0.10	14.00 ± 0.09	19.27 ± 0.10
AMVF	79.80 ± 0.09	70.07 ± 0.09	19.69 ± 0.10	16.16 ± 0.09	19.87 ± 0.10
Difference	+3.36 ± 0.06	+3.37 ± 0.06	−0.95 ± 0.09	−2.16 ± 0.09	−0.60 ± 0.09

Table D.2: Effect of the integral cut on the same PU200 events. The last column uses the same separation-fit procedure for both cuts; its errors come from the fit covariance. The 0.50 central value agrees with the existing resolution plot. The efficiency includes merged truth vertices.

Integral cut	Efficiency [%]	Fake vertices/event	$\sigma_{\text{vtx-vtx}}$ [mm]
0.30	83.16 ± 0.08	18.73 ± 0.10	0.218 ± 0.004
0.50	82.41 ± 0.08	15.09 ± 0.09	0.229 ± 0.003

Table D.3: Results on the main 1 920-event sample with the wider 0.229 mm matching window, taken from the PV-Finder separation fit. Both methods use this same window and truth vertices with $n_{\text{trk}} \geq 2$. Clean, merged and fake refer to reconstructed vertices. Efficiency includes merged truth vertices. The counts use the same 0.30 peak selection as the fixed-window comparison; the PV-Finder separation fit uses the 0.50 selection.

Algorithm	Clean/evt	Merged/evt	Fake/evt	Eff.	$\sigma_{\text{vtx-vtx}}$ [mm]
AMVF	67.7	12.2	17.5	84.0%	0.305
PV-Finder	71.9	11.8	16.3	86.9%	0.229 ± 0.003

extra fake and split peaks can fill the dip. We keep the 0.50 selection for the resolution plot and the 0.30 selection for the performance counts.

We also checked the plotting bin width in a separate 7 680-event sample. The weighted peak positions can lie between output bins. However, sharp peaks still favour positions near the 0.04 mm output grid. Using 0.05 mm plotting bins gives a repeated pattern in the pair counts. When the same pair distances are rebinned at 0.04 mm, the measured modulation falls from 3.88% to 0.04% for the 0.50 peak list. The 0.30 list gives a similar change, from 3.81% to 0.06%. This is a change to the plotted histogram. It does not change the vertex positions or put a lower limit on their precision.

D.2 Results with the wider matching window

An earlier evaluation used the fitted PV-Finder vertex-vertex resolution from the 0.50 integral cut, 0.229 mm, as the matching window for both methods. Table D.3 and Figure D.1 retain these results. The efficiency is 86.9% for PV-Finder and 84.0% for AMVF. These values are higher than those in the main table because the wider window accepts more matches. The model, peak selection and events are unchanged.

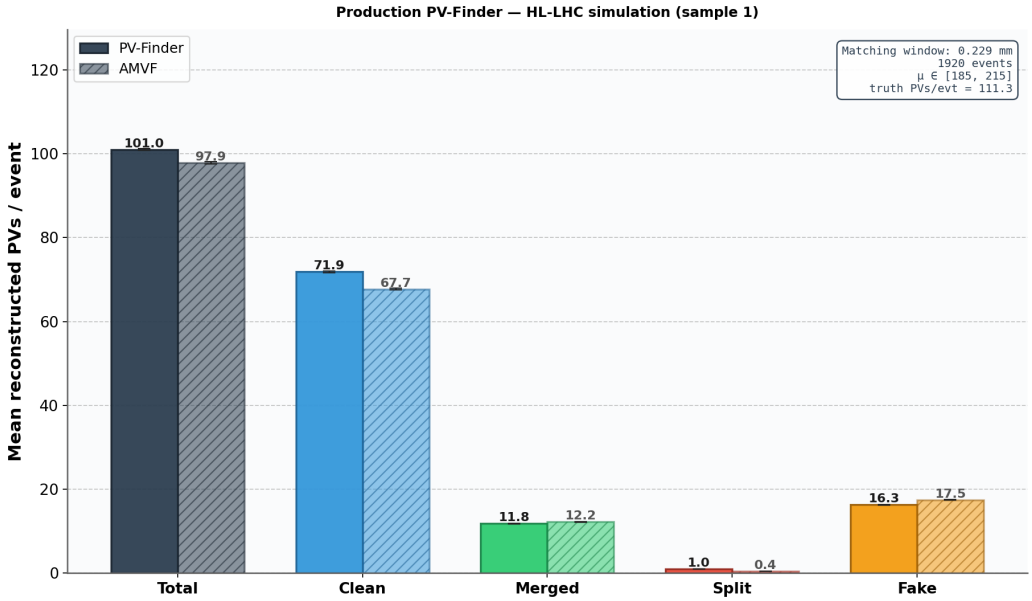


Figure D.1: Reconstructed-vertex categories on the 1 920 PU200 events, using the wider 0.229 mm matching window for both methods. The counts correspond to Table D.3.

Table D.4: PV-Finder results for the two reconstruction versions. Both select $185 \leq \mu \leq 215$, but the evaluated subsets have different sizes. The windows are 0.228983 mm for `r16443` and 0.227832 mm for `r16638`. The main comparison uses only `r16443`, with the fixed 0.1694 mm window. The counts use the 0.30 integral cut and the separation fits use 0.50.

Dataset version	Events	Efficiency [%]	Fake/event	$\sigma_{\text{vtx-vtx}}$ [mm]
`r16443`	1 920	86.9	16.3	0.229 ± 0.003
`r16638`	1 888	86.95	16.10	0.228 ± 0.003

D.3 Check of the reconstruction version

The dataset names `r16443` and `r16638` identify two reconstruction versions of the same generated and simulated events. They describe the data production, not different finder models. The main comparison uses `r16443` for both PV-Finder and AMVF, so both methods are evaluated on the same events. This sample was excluded from finder training.

The second version, `r16638`, is used as a check. The results for PV-Finder are very close in the two versions (Table D.4). Each uses its fitted separation scale as the matching window for this check. Agreement checks the effect of the reconstruction changes. It is not a test on independent simulated events.

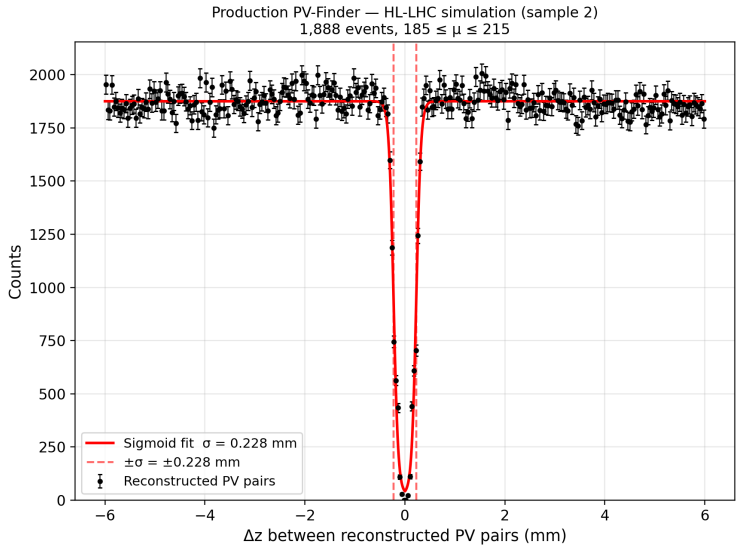


Figure D.2: PV-Finder pairwise Δz distribution for the second reconstruction version. The fit gives 0.228 ± 0.003 mm, close to 0.229 ± 0.003 mm for the main sample. Both fits use an integral threshold of 0.50.

D.4 Examples of merged, missed and fake vertices

The examples in Figures D.3 to D.5 show where the finder makes mistakes. They come from an earlier PU200 model and use a 0.5 mm matching window. The production model's rates are given in Section 4.1.

When vertices merge, the finder produces one peak for two or more truth vertices. In some examples, the tracks or analytical KDE show two nearby groups, but the predicted histogram has only one peak. Missed vertices usually have few tracks and no clear peak nearby. Fake peaks pass the cuts but have no selected truth vertex within the matching window. They can appear in regions with few tracks or beside a larger peak (Appendix D.6).

The median separation between neighbouring truth vertices is 1.16 mm in the production evaluation sample. About 13.4% of neighbouring pairs are closer than 0.229 mm, and 17.2% are closer than 0.30 mm. Separating these pairs is difficult. This does not put a limit on the reported efficiency, which counts merged truth vertices as found (Equation 2.5). The output bins are only 0.04 mm wide, much smaller than the measured vertex-vertex resolution. The bin width alone therefore does not explain the merged pairs.

The tests with different model sizes and training settings did not give a clear improvement on the earlier sample (Appendix E.3). Other tests could change the output binning, targets or loss, or train the finder and track association together. The current GNN cannot separate a merged pair: it only assigns tracks to peaks that the finder has already produced. Timing could help separate nearby interactions. We have not yet trained a model using the larger timing coverage in the new samples (Appendix E.4).

One change to the evaluation reduced the fraction of truth vertices labelled merged from about 22% to 11%. Previously, every truth vertex sharing a peak was called merged. We now call the first matched truth vertex clean and only the additional ones merged, as described in Section 2.4. This changes the truth labels, but leaves the reconstructed categories and efficiency unchanged. It is separate from the change to closest-first matching.

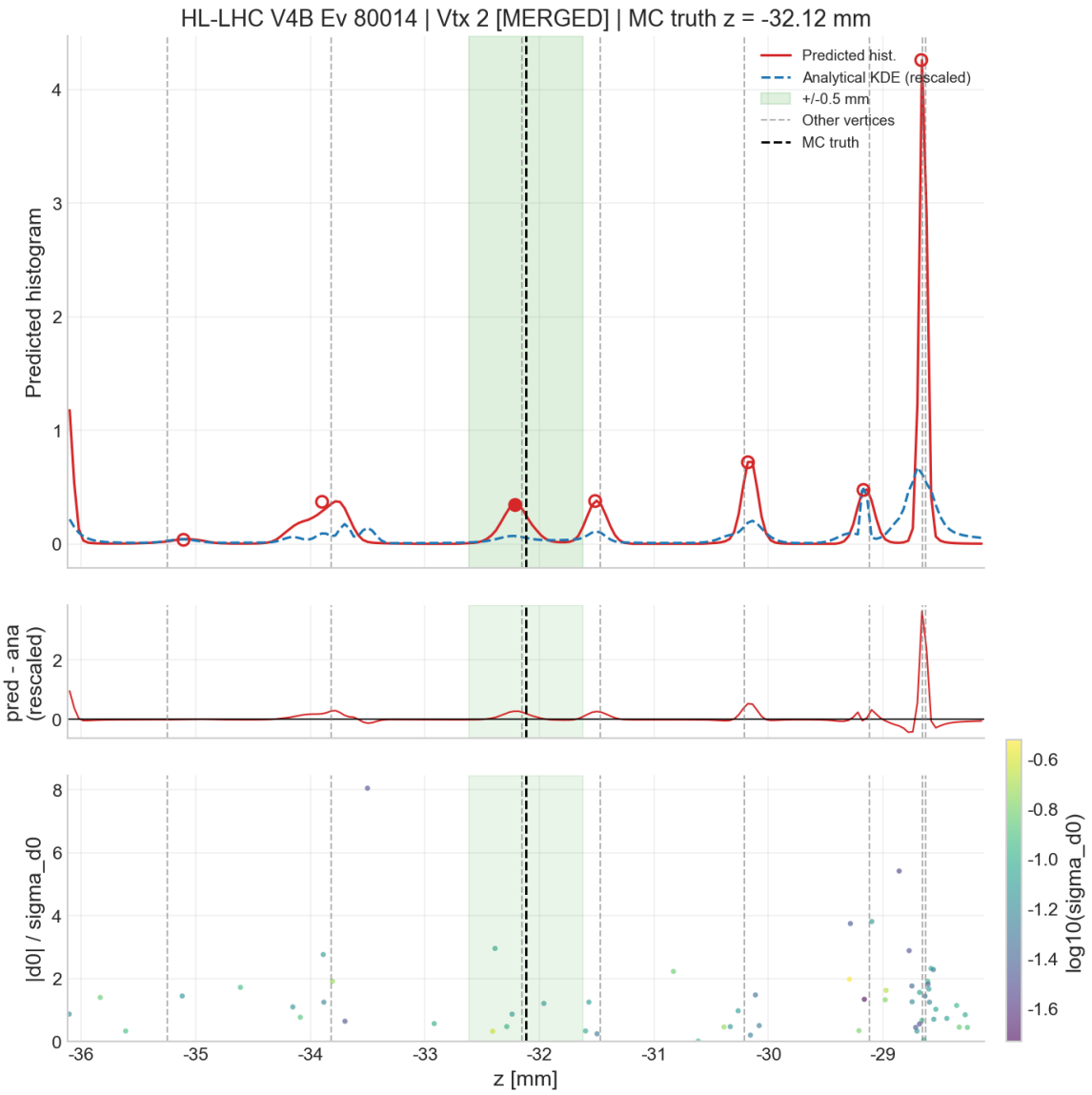


Figure D.3: Two nearby truth vertices sharing a peak in the earlier PU200 model. The top panel shows the predicted histogram in red and its peaks as red circles. The blue dashed curve is the analytical KDE, rescaled for comparison. The black dashed line marks the selected truth vertex, with a ±0.5 mm window around it. Grey lines mark other truth vertices. The middle panel shows the prediction minus the rescaled KDE. In the bottom panel, tracks are shown at their z_0. The vertical axis is $|d_0|/\sigma_{d_0}$ and the colour gives $\log_{10}\sigma_{d_0}$.

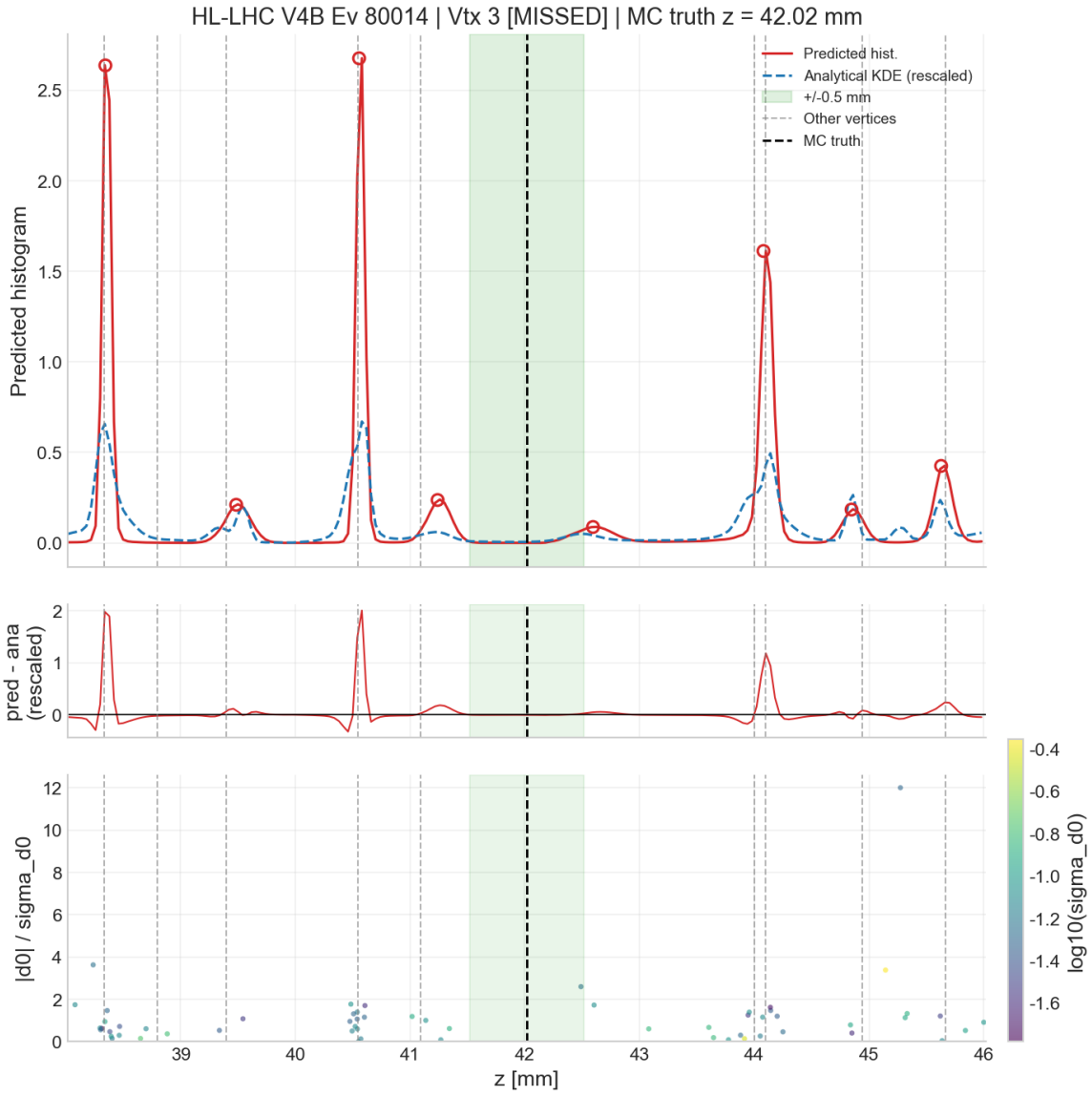


Figure D.4: A missed truth vertex with few tracks in the earlier PU200 model. The panels and lines follow Figure D.3. There are few tracks near the selected truth position and no peak within its window.

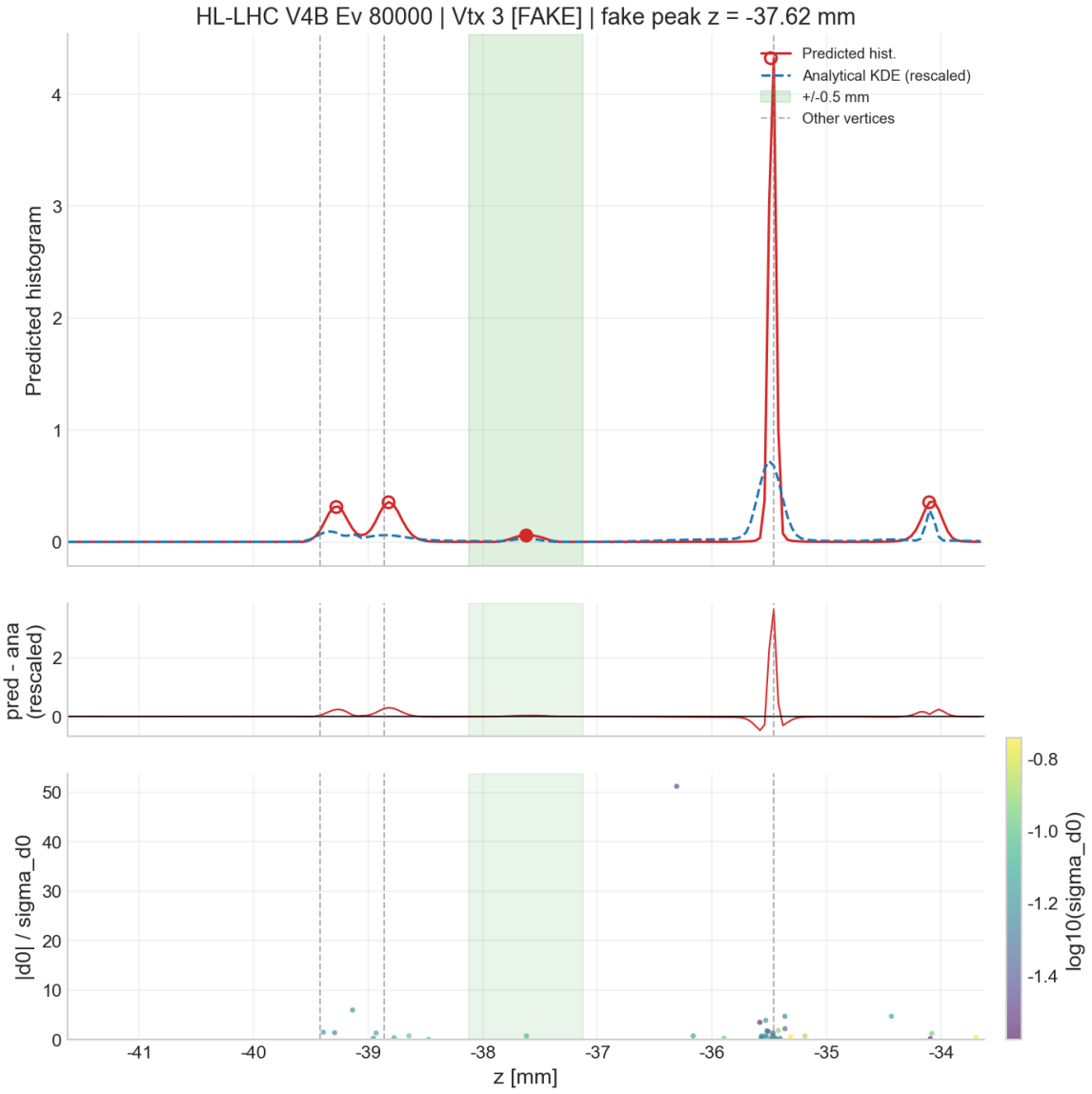


Figure D.5: Example of a fake peak from the earlier PU200 model. The panels follow Figure D.3, with the green ±0.5 mm band now centred on the reconstructed peak. No selected truth vertex lies in this window.

D.5 Cuts to remove fake peaks

We tested two ways to remove fake peaks: a cut on peak height and a gradient-boosted-tree (GBT) classifier. Both use the same trained finder. They reduce the fake rate, but also remove some real vertices. The height cut is used in production. We tested the classifier as an extra cut, using the shape of each peak and information about nearby peaks. Table A.1 gives the settings.

The checks below use smaller subsets and wider matching windows than the main comparison. Each cut is compared before and after on the same events.

On 384 PU200 events, requiring a peak height of at least 0.03 removes 1.06 fake vertices per event. The efficiency falls by 0.21 percentage points (Table D.5). We use the same 0.229 mm matching window before and after the cut.

A stricter height cut removes more real vertices because real and fake peaks can have similar heights. In a separate study, a cut of 0.05 removes 31% of fake candidates and 2.6% of real candidates. At 0.10, it removes 63% and 8.4%, respectively. In this study, a candidate is called real if a selected truth vertex lies within 0.5 mm. The percentages describe how many peaks are removed. They do not measure the change in truth-vertex efficiency.

For the classifier study, we use 340 541 peaks from 6 000 events of the flat-μ sample. With the same 0.5 mm matching rule, 11.6% of these peaks are fake. We randomly split the peaks into 70% for training and 30% for testing. This allows peaks from one event to appear in both sets. A test on independent events would require a split by event, with no test events used in training.

The full classifier uses 23 inputs describing nearby tracks, the peak shape and nearby peaks. It gives an area under the ROC curve of 0.958 on the test set. We also train two versions with fewer inputs. The 15 track inputs alone give 0.946. The eight histogram and nearby-peak inputs give 0.953, close to the full result. Adding track information therefore helps only a little in this test. The local histogram integral has the largest feature-importance score, 0.74. The next largest are the distance to the nearest peak (0.06) and the peak height (0.05).

A score cut of 0.3 on the full classifier keeps 99.0% of real peaks and removes 30.4% of fakes. At 0.5, it keeps 96.1% and removes 63.2%. At 0.7, it keeps 92.1% and removes 83.1%. The classifier using only the histogram gives similar results. At 0.5, it keeps 95.4% of real peaks and removes 62.3% of fakes (Figure D.6). This version needs no track inputs once the finder has produced the histogram.

We then apply this histogram classifier after the production height cut, using a score cut of 0.3. This check uses 268 PU200 events from the second reconstruction of the simulated sample (Appendix D.3) and a fixed 0.228 mm matching window. It removes another 1.38 fake vertices per event, or 8.6%. The efficiency falls by 0.34 percentage points.

The fraction of fakes removed is smaller than in the classifier test above. This check uses different selections, pile-up and matching rules. Some small peaks have also been removed by the height cut. The two reconstruction versions share the generated and simulated events, so these events are related to those used to train the classifier. This check does not provide an independent test.

Removing peaks can change the fitted resolution. If we took a new window from the fit after each cut, the measured efficiency would change for that reason too. Keeping the window fixed in Table D.5 lets us measure the effect of the cut itself.

Table D.5: Efficiency and fake rate before and after each cut. The first two rows use 384 events from `r16443` and $w = 0.228983$ mm. The last two use 268 events from `r16638` and $w = 0.227832$ mm. Both select $185 \leq \mu \leq 215$. Efficiency includes merged truth vertices. Each before-and-after comparison uses the same events and window. These are smaller samples than in Table 4.1.

Selection	Efficiency [%]	Fake vertices/event
No height cut	87.25	17.02
Height ≥ 0.03	87.04	15.96
Height ≥ 0.03	87.29	15.99
Height ≥ 0.03 and GBT score ≥ 0.3	86.95	14.62

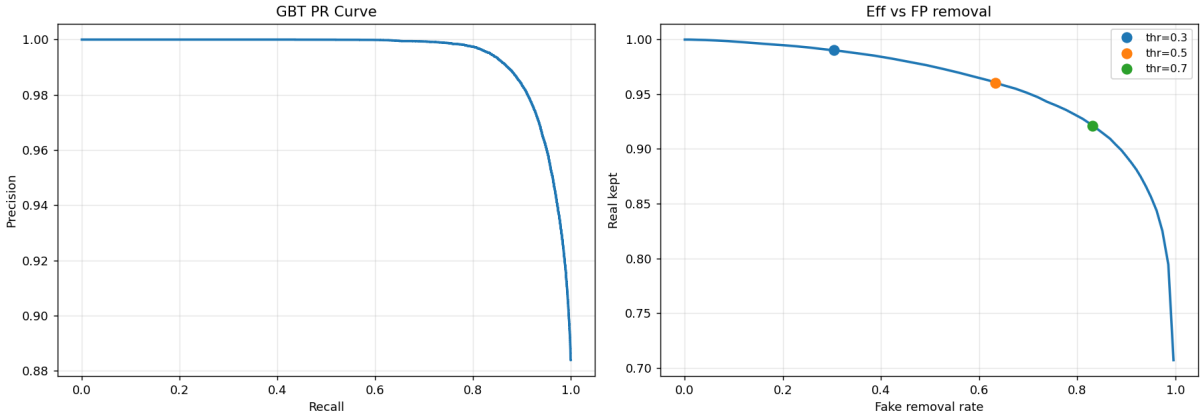


Figure D.6: GBT results for peaks from the production finder. The curves show precision, recall, the fraction of real peaks kept and the fraction of fakes removed. A real peak has a selected truth vertex within 0.5 mm. The test peaks come from the flat-μ sample. The split is made by peak, so an event can contribute to both training and testing.

A possible next test is to penalise fake peaks in the training loss or add a network output that predicts whether a peak is real. These changes have not been tested here. The classifier results also do not show that the histogram contains all the useful track information.

D.6 Study of satellite peaks

Some fake or split peaks lie near a larger peak. We call them satellites. An earlier study found a bump near $|\Delta z| = 0.57$ mm in the distribution of distances between reconstructed vertices. That study used an integral threshold of 0.20, no height cut and a weighted mean over the full peak region to find each position. The bump was easier to see with these settings.

We compared the number of pairs separated by 0.3 to 0.7 mm with a baseline estimated from pairs at larger separations. There were 12.5% more pairs than this baseline predicted. The truth distribution was approximately flat in this range. The extra pairs contained fake or split peaks. Pairs matched to two different truth vertices showed no bump.

In this range, 92.5% of the satellites came from splitting one peak region. The peak finder splits a region when the histogram falls and then rises again. Even a small rise on the side of a peak can therefore give a

second vertex. The same peak finder produced no fake or split peaks when run on the target histograms in the control sample. The problem also depends on the shape predicted by the network.

To measure the size of these rises, we subtract the higher of the two neighbouring minima from the peak height. This difference is called prominence. Its median was 0.029 for satellites in the selected range and 0.751 for peaks matched to truth. Satellite height showed no correlation with the height of the nearby larger peak (Pearson $r = -0.004$). The small peaks therefore do not seem to be copies with a fixed fraction of the larger peak's height. The revised U-Net still favours some positions on the four-bin grid used during upsampling. This is more visible for fake and split peaks, and may contribute to the small rises in the histogram.

Using a different calculation of the peak position reduced the excess from 12.5% to 5.1%, without removing any peaks. A weighted mean over the full region can move the two parts of a split peak further apart. Taking the weighted mean only near each maximum reduces this effect. We use this local centroid in production.

We also tried requiring a prominence of at least 0.005. This removed 75% of the excess, but also lost 1.70 truth-matched peaks per event. It removed about two fake or split peaks for every truth-matched peak lost. We did not use this cut. A height cut removed fewer of these satellites. Rejecting peaks near another peak would also remove real close pairs. In the candidate study, 0.4% of fakes were within 0.3 mm of a real candidate and 12.7% were within 0.6 mm. The median distance was 1.06 mm.

With the production settings, the pair count is 3.6% above the same baseline estimate. The value for truth is 0.4%. Both depend on how the baseline is fitted. Fits that allow a slope and use different ranges at larger separation give an excess of about 0 to 1.7% for PV-Finder. The truth value changes by a similar amount. We therefore cannot clearly separate the remaining bump from the uncertainty in the baseline. AMVF gives −14.8%, a deficit of pairs in this range. Its wider resolution dip still covers this region, so that number alone does not measure how its satellite rate compares with PV-Finder.

E Studies that did not improve performance

We tested several changes to the training and network architecture while developing the end-to-end model. This appendix describes the tests and their results. It also explains why we use the MLP warm-up and the U-Net. The last section describes the HGTD timing study, which remains inconclusive.

E.1 Training without KDE targets

We tested four ways to train the tracks-to-histogram model (Table E.1). When the MLP and U-Net were trained together from random weights, the MLP placed most of its output in the first bin, regardless of the truth position. The U-Net then spread this output across the histogram. The mean-squared error decreased, but the model did not learn the correct vertex positions.

Pretraining the MLP on an analytical KDE before end-to-end training avoided this problem, but still needed KDE targets. We also tested an extra loss at the MLP output. It penalised output concentrated in the first bin and encouraged peaks near the track z_0 positions. This avoided the first-bin solution, but the vertex-finding performance was worse than the KDE-pretrained model.

For HL-LHC, we first train the MLP alone on the target vertex histogram. We then add the U-Net and train the full network end to end. This warm-up lets the MLP learn the positions before the U-Net changes the histogram shape. It avoided the first-bin solution in the tests and does not need KDE targets. The low-pile-up model in Chapter 6 still uses the earlier KDE pretraining.

E.2 The role of the U-Net

The MLP alone found regions containing vertices, but its peaks were too broad to locate them precisely. Adding the U-Net made the peaks sharper. Its convolutions use the local histogram shape, while the encoder, decoder and skip connections combine information over different distances. We therefore kept the U-Net after the MLP warm-up. The intermediate MLP output does not need to reproduce an analytical KDE.

E.3 Changes to the architecture and model size

The original U-Net uses transposed convolutions for upsampling. It reduces each 1 000-bin interval to 125 bins at the narrowest layer. We tested a revised design to reduce artifacts and keep more spatial detail. It replaces transposed convolutions with nearest-neighbour interpolation followed by convolution, and

Table E.1: Training procedures tested during KDE removal. The KDE-pretrained model is included as a reference. The MLP histogram warm-up is used for the HL-LHC models.

Procedure	KDE target?	Observation
Joint training from random initialization	No	Output collapses to first bin
MLP pretraining on KDE, then end to end	Yes	Avoids the collapse
Auxiliary spatial loss at MLP output	No	Avoids collapse, lower performance
MLP histogram warm-up, then end to end	No	Adopted for HL-LHC

reduces the histogram to 250 bins. The first encoder kernel is reduced from 25 to 15 bins. The skip features are added instead of concatenated.

Early resolution plots looked smoother with the revised network. However, the peak selection also changed. A stricter integral threshold removes small peaks and can make the pairwise distribution look smoother. These plots alone therefore do not show that the new architecture removed the artifacts. It still produces small fluctuations related to its upsampling grid (Appendix D.6).

We compared model sizes on the initial 99.8K-event PU200 sample. Increasing the number of parameters from 681K to 3.55M, about a factor of five, gave similar performance. We also tested a higher learning rate with warm restarts, SGD with momentum instead of Adam, and an extra total-variation term in the mean-squared-error loss. None gave a clear overall improvement. Some lowered efficiency. The total-variation term gave fewer false positives, but also lower efficiency.

These tests do not show that the smaller model is large enough for every training sample, or that it has reached the lowest possible loss. The Gaussian target widths also depend on track multiplicity. We kept the larger network for later training, so changes to the sample or targets could be tested with the same architecture. A smaller model still needs to be trained on the current 5.17M-event sample for a direct comparison. Another possible test is a loss that penalises fake, merged or missed vertices more directly. We have not yet tested whether this helps.

E.4 HGTD timing

The HL-LHC ntuples contain per-track information from the High-Granularity Timing Detector (HGTD). Timing could help separate interactions that are close in z, so we studied whether the finder could use it.

The study was inconclusive. In the earlier $|\eta| < 2.5$ samples, only about 3.9% of tracks had a valid time, which limited the test. With full track acceptance, this fraction rises to 28.8%. A model using timing has not yet been trained on that sample. The results in this note therefore do not use timing. Its effect still needs to be tested in the finder or the track-to-vertex association of Chapter 7.